

On the Provenance of a Pre-Complex Body: A Finite Real Response-Algebra Core

Jan Seeck
Independent researcher
jan.seeck1@gmail.com

Finite code snapshot: July 10, 2026
Documentation revision: July 17, 2026

Abstract

This article presents the discrete-finite core of the ComplemeNt Net Architecture (CNNA), initialized by an empty configuration and generated by an explicit deterministic rule family. The rooted carrier is not introduced as a discretization of a pre-existing space or spacetime, but as a finite, deterministic, and exactly inspectable substrate for genealogical provenance and boundary reduction. The structural index b , depth cutoff L , mode m , and update schedule $\mathfrak{s}$ are distinct; reference calculations fix m and $\mathfrak{s}$ by stated commands rather than deriving them from the empty configuration. The term complement has a precise birth-relative meaning: an accepted birth enlarges the finite vertex carrier by a disjoint singleton coordinate, while a causal-layer update enlarges it by the direct sum of all newborn coordinates in that layer. These new carrier directions are then coupled to the existing network by directed parent-child, ancestor, and sibling weights. The implementation produces finite histories, terminal carriers, directed live and record edge layers, and a symmetrized control, without continuum fields, integrals, differential operators, or limits. For fixed inputs it computes carrier states, projectivity and orientation diagnostics, Schur-complement responses, symmetric/skew decompositions, finite J -objects, finite algebra and GNS quotients, local operator checks, sign transport, and live-record backreaction observables. Every reported object is a finite array, edge dictionary, matrix, residual, or generated row.

Dirichlet-to-Neumann terminology refers only to finite weighted networks: the maps are explicit Schur complements with declared boundary/interior partitions and residuals. No analytic boundary calculus is assumed. The calculations distinguish directed live and record structures from the symmetrized control and expose satisfied and failed finite properties. The relation to recent growing-tree models, finite-network effective operators, and tree Dirichlet-to-Neumann constructions is stated by explicit model-separation criteria rather than by a literature-wide novelty claim.

The result is not an infinite or continuum theory. It includes an exact, linearly growing real carrier system with non-unital corner embeddings, while independently reconstructed response, state, GNS, and finite- J objects retain quantified interlevel drift. Infinite completions, convergence claims, and continuum comparisons remain separate problems.

Contents

1	Methodological position and scope	4
2	Recent related work and model separation	6
2.1	Comparison criteria	6
2.2	Recent tree-growth models	6
2.3	Recent boundary-response constructions on trees and finite networks	7
2.4	Claim boundary	9
3	Source surface and execution basis	9
3.1	Inspected source tree	9
3.2	Minimal finite data route	10
3.3	Row status fields	10
3.4	Integrated execution command	10
3.5	Tested numerical environment and licensing	12
3.6	Regeneration boundary for row values	13
3.7	Runtime degrees of freedom	13
3.8	Finite quantities and code locations	13
4	Reference scope for discrete finite operations	17
5	Derivation graph	17
5.1	Global object order	17
5.2	Function-block placement	18
6	Core module map	20
7	Derived-quantity module map	22
8	Core construction initialized by the empty configuration	24
8.1	Finite arithmetic layer	24
8.2	Empty-set growth object	24
8.3	Birth-relative complement meaning of CNNA	28
8.4	Carrier and prefix projections	30
8.5	Whole-network F1/F2 axes and signed current scalar	31
8.6	Symmetric Schur–DtN response on a selected finite cut	32
8.7	Region specification and boundary role compression	34
8.8	Directed response and S/A decomposition	34
8.9	Finite J comparison on an S/A support	35
8.10	Finite response algebra and GNS quotient audit	36
8.11	Regional response object and local support algebras	37
8.12	Compressed carrier/GNS state and projective distribution	38
8.13	Refined boundary/port basis	39
8.14	Entropy functionals and record/live entropy distances	40

9	Finite structural results and exact proof boundary	41
9.1	Well-definedness of the finite growth construction	41
9.2	Positivity of the symmetric finite response	42
9.3	Exact finite GNS quotient	43
9.4	Orientation reversal and the unresolved absolute sign	44
9.5	Separation of backreaction observables	44
9.6	Exact prefix handoff	45
9.7	Growing address-contrast carriers and non-unital corner embeddings	46
9.8	Control-row summary	49
10	Derived quantities as finite row families	49
10.1	Baseline row families	49
10.2	Rank, kernel, coverage, and compressed projectivity	50
10.3	Structural live-record ledgers	50
10.4	Interlevel operator maps and interface rows	50
10.5	Directed transport rows	51
10.6	A/J orientation transport and composition	52
10.7	Local/global J sign relation	53
10.8	Live-record backreaction current	54
10.9	Birth-relative backreaction rows	55
10.10	Branching comparison and row aggregation	56
11	Conclusion	58
12	Outlook	59
13	References	60
14	Source-tree object boundary	62
15	Generated row-file index	64
16	Function-level source inventory	66

1 Methodological position and scope

The present construction is not introduced as a finite-dimensional approximation of an already specified quantum theory. Its question is logically prior: which response-theoretic and algebraic structures can be constructed before a complex Hilbert space, a complex observable algebra, a Hamiltonian, or a continuum geometry is assumed? The dependency order investigated here is

$$\begin{aligned}
& \emptyset \longrightarrow \text{finite } b\text{-ary address growth} \longrightarrow \text{birth-relative real carrier complements,} \\
& \text{carrier complements} \longrightarrow \text{directed record/live weights} \longrightarrow \text{finite Schur boundary response,} \\
& \Lambda^{\text{dir}} \longrightarrow S = \frac{1}{2}(\Lambda^{\text{dir}} + (\Lambda^{\text{dir}})^T), \\
& \Lambda^{\text{dir}} \longrightarrow A = \frac{1}{2}(\Lambda^{\text{dir}} - (\Lambda^{\text{dir}})^T), \\
& (S, A) \longrightarrow \text{finite real algebra} \longrightarrow \text{positive state,} \\
& \text{positive state} \longrightarrow \text{real GNS quotient} \longrightarrow \text{finite complex-structure candidate.}
\end{aligned} \tag{1.1}$$

Here complement is not a metaphor and not a claim about an operator-algebraic commutant. At a birth event it is first the exact set-theoretic and coordinate-linear decomposition of the post-birth carrier into the pre-birth carrier and the newborn coordinate. The directed weights subsequently couple that new direction to the prior network. The skew sector is therefore not supplied by the word complement itself: it can occur only because the coupling rule is directed and because the resulting asymmetry survives the selected Schur reduction. Section 8.3 states these distinctions formally.

Substrate-selection criterion. The carrier choice follows a non-circularity requirement. If continuum geometry, spacetime, the causal and metric structure of spacetime, or complex-linear kinematics are among the structures whose provenance is under investigation, they cannot simultaneously be inserted as primitive input. The finite core therefore starts with neither a manifold, metric, measure, continuum field, differential operator, nor stochastic growth law. It requires instead a deterministic carrier that is finite at every stage, exposes every accepted growth event, has exact genealogical addresses, supports nontrivial branching, and admits explicit finite boundary elimination.

Subject to these requirements, rooted b -ary trees with $b \geq 2$ are among the simplest sufficiently rich carriers. A path carrier is combinatorially simpler but has no sibling branching, whereas an arbitrary graph would add adjacency choices that are not yet derived at the initial stage. The selection is therefore a methodological minimality statement under the stated requirements, not a uniqueness theorem and not a claim that every alternative discrete substrate has been excluded.

Earlier work with post-critically finite Sierpiński-type complexes informed this selection because their finite graph approximants and finite boundary sets support a rigorous theory of compatible resistance and Dirichlet forms, Laplacians, and spectral analysis [17, 18, 19]. That setting supplied a controlled example of hierarchical response theory without requiring a smooth manifold as the starting object and motivated the move to an explicit branching provenance carrier. In the present finite CNNA derivation, however, no Sierpiński gasket or tetrahedron, Euclidean embedding, self-similar continuum set, resistance-form limit, or fractal Laplacian is an input. The rooted tree is the primitive provenance carrier. Any later relation between that tree and a Sierpiński-type complex is a separate gluing or duality problem and must be constructed and checked independently. This order differs from frameworks in which a complex operator algebra, a quantum state space, or prescribed finite-dimensional algebraic connecting data belong to the initial specification. For example, algebraic quantum field theory starts from local observable algebras and their structural

relations [21]; AF constructions start from finite-dimensional C^* -algebras together with embedding data [22]; and finitely correlated spin-chain states start from a finite-dimensional auxiliary algebra and a positive map [23]. These comparisons locate the logical starting point of the present work; they do not assert equivalence with those theories.

A closer methodological comparator is the construction of complex structures on real jet spaces by Finster and Kamran [24]. In the setting of causal variational principles, conserved surface-layer forms are used to define an almost-complex structure, and additional conditions are analyzed under which it gives a canonical complex structure that combines with a scalar product to form a complex Hilbert space and subsequently supports bosonic Fock-space dynamics. The present article neither adopts a causal variational principle nor starts from jet spaces or surface-layer forms. It begins instead with finite real network growth and boundary response and asks whether a skew-response sector, a real algebra, a state, a real GNS quotient, and a finite J -candidate can be obtained along that explicit dependency chain. Consequently, unresolved sign and interlevel-naturality defects are retained as finite obstructions rather than removed by assuming the additional conditions required for a canonical complex structure. The relation between the two approaches is therefore a shared provenance question, not an identity of primitives, constructions, or results.

The distinction does not lie in the individual mathematical tools. Schur complements, finite network response maps, finite matrix algebras, positive functionals, and GNS constructions are established structures. The specific object investigated here is their connection by an explicit finite provenance chain whose interfaces are audited for hidden dependencies. The tree is therefore not interpreted directly as physical space, spacetime, or an observable algebra. It first records genealogical provenance. Each accepted address adds an exactly identifiable carrier direction complementary to the carrier already present at that update; the network character begins when this direction is coupled by the generated directed weights. Candidate observable structure appears only after this complement-coupling step, live-record separation, boundary reduction, and algebra generation.

In particular, the complex unit is not inserted through the scalar field of the initial construction. A real skew-response sector is constructed first, and only then is a finite operator J tested for the algebraic relation associated with a complex structure. The residual transformation $J \mapsto -J$ is retained as an obstruction whenever the available real data do not distinguish the two orientations. Thus a successful finite J test is not itself an absolute orientation theorem.

The finite restriction is deliberate. Every node, edge, response matrix, algebra generator, state, quotient operation, and residual remains explicitly inspectable. No continuum limit, von Neumann closure, modular flow, or type classification is used to justify an earlier stage of the construction. The results should therefore be read as a finite structure theory: they determine what can already be constructed or excluded at finite depth and identify the additional compatibility conditions required for an infinite operator-algebraic handoff.

The novelty claim is correspondingly limited. The article does not claim that its constituent techniques are individually new, that no related provenance-oriented construction exists, or that the finite core already defines a quantum field theory, an inductive-limit C^* -algebra, or a continuum spacetime model. It claims a specific auditable architecture and the finite results proved or computed within that architecture.

Development note. The line of inquiry leading to the present finite core has evolved through more than four years of independent study, implementation, rejected constructions, falsification, and progressive scope reduction. Parts of this development are preserved in public, time-stamped records beginning in late 2021, including contemporaneous forum discussions; some intervening material is no longer completely recoverable. These records document the continuity and changing scope of

the inquiry, not the prior existence of CNNA in its present form. Earlier exploratory formulations considered broader deterministic, fractal, network, open-system, and operator-algebraic interpretations. They are not used as premises or evidence in this article. The present work deliberately retains only the finite definitions, source-level constructions, generated results, and explicit obstructions reproducible from the supplied package. The documented history establishes the provenance of the inquiry, not the validity of its mathematical claims, which stand or fall on the construction and checks presented here.

2 Recent related work and model separation

The phrase *growing tree* covers several mathematically distinct constructions. In one class the finite combinatorial carrier itself changes by the addition, deletion, or rearrangement of vertices and edges. In another class a random subtree, occupied cluster, or labeling grows inside a maximal or already specified carrier. A third class keeps the graph fixed and constructs or reconstructs a boundary-response operator on that graph. These distinctions matter here because the present finite core combines carrier growth, an immutable birth record, later updates of a separate live layer, directed real couplings, and a forward Schur boundary reduction. No single shared noun such as “tree”, “growth”, or “Dirichlet-to-Neumann map” is sufficient to identify the constructions.

2.1 Comparison criteria

The comparisons below use the following explicit questions.

1. What is primitive input: a maximal tree, a finite tree, a metric or quantum graph, or only a root and a branching bound?
2. What actually grows: the carrier, a selected subtree or occupation pattern, a labeling, or only an associated response matrix?
3. Is the evolution deterministic or stochastic, and is its ordering sequential, generation-wise, or synchronous on an active frontier?
4. Are previously accepted vertices and genealogical addresses preserved, or can the earlier tree be rearranged during a growth step?
5. Is there a birth-relative immutable record distinct from a state that is updated by later births?
6. Is a boundary response constructed, and if so, is it a forward finite Schur response or an inverse reconstruction problem?

These criteria are descriptive. They locate the finite CNNA core among adjacent models without asserting that the cited sample exhausts the literature on random trees, branching processes, graph growth, or inverse problems.

2.2 Recent tree-growth models

Marckert constructs a sequence of uniformly distributed rooted d -ary trees in which the passage from size $n - 1$ to size n adds a node and rearranges part of the previous tree [8]. This is a genuine carrier-growth construction and therefore a close combinatorial comparator. Its purpose is uniform random generation, however, and the rearrangement step means that the earlier finite tree is not retained as an immutable genealogical record in the sense used here.

Fredes and Marckert study and survey models of random subtrees of a connected graph, including uniform sampling of an n -vertex subtree of a specified ambient graph [9]. This cleanly illustrates the second meaning of growth used above: the selected subtree changes while the ambient carrier is already present. The present finite core instead treats accepted births as enlargements of the carrier itself; it does not sample an occupied subtree of a pre-existing graph.

Dunlop and Mardin study bounded-generation Bienaymé–Galton–Watson trees with nearest-neighbor ancestor interaction [10]. A random configuration is represented as a rooted subtree of a specified maximal tree, and the offspring variables and coupling determine the probability law. Thus the maximal carrier and stochastic law are primitive, whereas the present construction uses a deterministic accepted-birth sequence on the finite carrier and separates the frozen birth record from the subsequently updated live response.

Bodini, Genitrini, Mailler, and Naima study increasing and strict monotonic Schröder trees generated by evolutionary rules in which labels encode the chronological order of branching events [11]. Their first model selects one leaf at each step, while the strict monotonic model can select several leaves carrying the same chronological label. These are direct comparators for sequential versus synchronized event order. Their chronology is explicit in the labeling rule and supports random-tree enumeration and sampling; it is not a split between an immutable edge record and a later-updated network response.

Bodini, Genitrini, and Nurligareev introduce growing binary trees with internal nodes, active leaves, and dead leaves [12]. Starting from one active leaf at $t = 0$, every active leaf is synchronously replaced at each positive integer time either by a dead leaf or by an internal binary branch. This provides a particularly direct comparator for the `layer_batch` control schedule: both organize a whole active frontier by discrete layers. The cited model adds an extinction rule and develops enumerative and sampling results; it does not construct birth-relative record/live edge layers or a network response that feeds back on previously born vertices.

2.3 Recent boundary-response constructions on trees and finite networks

Gernandt and Rohleder recover a finite weighted tree and its positive edge weights from the Dirichlet-to-Neumann matrix of its graph Laplacian [13]. The tree and symmetric weighted Laplacian belong to a fixed inverse problem, and the boundary matrix is used to reconstruct weighted distances and hence the tree. The present direction is forward rather than inverse: the finite carrier and directed record/live weights are generated first, after which a selected boundary/interior partition is Schur-reduced.

Beard, Stefan, Viator, and Welters formulate effective operators for discrete electrical network problems by block-operator and Schur-complement methods and relate the finite-network effective operator to a discrete Dirichlet-to-Neumann map [14]. This supplies a recent operator-theoretic comparator for finite elimination and variational response. Their graph and constitutive operator are fixed problem data; carrier growth, immutable birth provenance, and a directed record/live split are not part of that construction.

Peng and Song develop sparsified block elimination for Eulerian directed Laplacians, with augmented matrices whose Schur complements encode elimination errors [15]. This confirms that Schur-based elimination has a substantial modern theory beyond symmetric graph Laplacians. The aim there is an approximate linear-system solver on a fixed directed Laplacian class; it is not an exact forward boundary response generated from a birth history, and it has no record/live distinction.

Avdonin, Khmelnytskaya, and Kravchenko give a progressive synthesis of the Weyl matrix, equivalently the transposed Dirichlet-to-Neumann map, of a quantum tree by successively attaching edges and computing the larger response from smaller ones [16]. This is especially relevant

because both the tree and its boundary matrix are enlarged step by step. The quantum-tree setting nevertheless starts from metric edges, Sturm–Liouville equations, potentials, matching conditions, and a spectral parameter. The finite CNNA core starts instead from a real combinatorial carrier and directed finite weights and uses no differential equation or metric-edge input.

Table 1: Model-separation criteria for recent comparators. The entries summarize only the aspects relevant to the present finite construction.

Comparator	Primitive carrier and actual growth	Order or randomness	Provenance and response	Exact separation from the present core
Marckert [8]	Rooted d -ary carrier grows by one node, with rearrangement of prior structure.	Random construction with uniform size- n marginal.	No record/live response split; no Schur–DtN layer.	Close carrier-growth comparator, but previous genealogy may be rearranged and the objective is uniform random generation.
Fredes–Marckert [9]	Selected subtree of a fixed connected ambient graph.	Markov-chain and random sampling models.	Subtree statistics; no carrier-birth record or forward DtN response.	The occupation/subtree grows inside a pre-existing carrier; CNNA enlarges the finite carrier itself.
Dunlop–Mardin [10]	Random rooted subtree of a specified maximal bounded tree.	Stochastic offspring law with generation bound and interaction.	Configuration statistics and recursion; no immutable birth record or forward DtN response.	The maximal carrier and probability law are inputs; CNNA grows the accepted finite carrier deterministically.
Bodini–Genitrini–Mailler–Naima [11]	Labelled Schröder carrier generated by replacing selected leaves.	Sequential one-leaf or synchronized multi-leaf chronological labels; random-tree analysis.	Labels record event chronology; no record/live response split or Schur–DtN layer.	Close event-order comparator, but chronology is an explicit label rule rather than a later network response.
Bodini–Genitrini–Nurligareev [12]	Binary active frontier grows or becomes extinct.	External integer time; synchronous update of every active leaf.	Enumerative profile and sampler; no record/live response split.	Direct comparator for layer_batch , not for the sequential schedule or later directed backreaction.
Gernandt–Rohleder [13]	Fixed finite weighted tree.	No growth dynamics.	Symmetric Laplacian DtN matrix used for inverse reconstruction.	Inverse fixed-tree problem with positive symmetric weights, rather than generated directed forward response.
Beard–Stefan–Viator–Welters [14]	Fixed finite network and constitutive operator.	No carrier growth.	Effective operator and DtN map obtained by Schur methods.	Strong response-theoretic comparator, but no genealogical carrier growth or record/live layering.
Peng–Song [15]	Fixed Eulerian directed Laplacian system.	Algorithmic block elimination and sparsification.	Directed Schur complements encode elimination errors; no birth provenance.	Relevant to directed elimination, but approximate solver theory rather than exact generated boundary response.
Avdonin–Khmelnyskaya–Kravchenko [16]	Metric quantum tree enlarged by attaching edges.	Successive constructive synthesis.	Weyl/DtN matrix of larger tree computed from smaller matrices.	Progressive response growth is close, but metric edges, differential equations, and spectral data are primitive.
Present finite CNNA core	Root plus branching bound; accepted births enlarge the finite real carrier.	Deterministic selector s ; sequential reference mode and synchronous comparison mode.	Immutable record layer, later-updated live layer, directed weights, and forward finite Schur response.	The combined object studied here; stronger infinite, continuum, uniqueness, and literature-wide novelty claims are not made.

2.4 Claim boundary

The cited works show that node-by-node d -ary growth, random subtrees of fixed graphs, interacting branching subtrees, chronology-labelled increasing trees, synchronous binary-front evolution, finite Schur effective operators, directed-Laplacian block elimination, inverse tree DtN reconstruction, and progressive quantum-tree Weyl synthesis are established but distinct constructions. The present claim is correspondingly narrow: the inspected finite source realizes a specific deterministic combination of carrier growth, birth-relative coordinate extension, immutable record data, later live updates, directed coupling, and forward Schur reduction. This section does not prove that no mathematically equivalent formulation exists elsewhere, and it does not use the publication date of a reference as evidence for or against the validity of a mathematical statement.

Revision note. This documentation revision expands the recent-literature comparison and completes DOI or persistent-link metadata in the reference list. It changes neither the finite construction, the Python source, the generated row files, the reported numerical values, nor the finite claim boundary of the July 10 code snapshot.

3 Source surface and execution basis

3.1 Inspected source tree

The inspected source surface is supplied as arXiv ancillary material in `anc/CNNA_structured_core_pkg`. All package paths stated below are relative to that ancillary package root. The reproducible source-surface digest is computed over the Python files in the declared source boundary and excludes the article source, environment records, license, package checks, and other ancillary metadata:

2612012d5149bedca71dbdbe60d0cbd075bbda6171ca008e989482787a002044.

The digest is the SHA256 value of the line-sorted manifest stored in `docs/SOURCE_SURFACE_MANIFEST.txt`. Per-file checksums for the complete ancillary package are stored separately in `CHECKSUMS.txt`. The byte hash of the outer transport ZIP is not used as the mathematical source identity, because an archive may be unpacked or repackaged without changing the declared source files.

The source boundary for this document is the Python source surface

```
src/cnna/__init__.py,  src/cnna/config.py,  src/cnna/io,  
src/cnna/core,  src/cnna/derived_quantities,  src/cnna/object_checks,  
src/cnna/analysis_entrypoints,  run_*.py at package root.
```

The `config.py` file supplies numerical constants and reference selectors used by row construction. The `io` subtree supplies row-file writers and JSON hashing. The `object_checks` subtree is part of the documented source surface where the local-support, Tomita, commutant, split, entropy, and refined-basis row families are generated by their stated commands. Existing files ending in `.md` or `.csv` are not used as mathematical source text. Result files produced before the current growth rule are outside the evidential surface of this document. CSV files may be named later only as outputs generated from the current package by a stated command.

Table 2: Support files in the documented source surface.

Source file	Source object	Finite role
src/cnna/config.py	EPS; REFERENCE_MODE; REFERENCE_STATE; STRUCTURAL_RECORD_STATE; GEN_LABELS; CONTROL_MODES; CONTROL_STATES	numerical tolerance constant, selected reference row labels, generator labels, and comparison selectors used by finite row construction
src/cnna/io/csv_writer.py	write_csv; sha256_json	deterministic CSV field ordering and JSON hash utility for generated row data
src/cnna/io/markdown_report.py	module placeholder	package-level I/O module with no finite object constructor in the current source surface

3.2 Minimal finite data route

The executable route from a finite input tuple to row files is:

$$\begin{aligned}
(b, L, m, \mathfrak{s}) &\longrightarrow \mathcal{G}_{b,L,m,\mathfrak{s}} \longrightarrow (E^{\text{live}}, E^{\text{record}}, E^{\text{sym}}) \\
&\longrightarrow \Lambda_B \longrightarrow M_B = S_B + A_B \longrightarrow (J_B^{\text{fin}}, \mathcal{A}_R, \mathcal{H}_R^{\text{GNS}}) \\
&\longrightarrow \text{local row surfaces and live-record row surfaces.}
\end{aligned}$$

For a fixed command, the row files are finite evaluations of this route. The local-operator command uses the same growth-schedule argument as the integrated command.

3.3 Row status fields

The row files contain text fields that record finite branch outcomes. The following strings occur as data values in the current row surface.

Table 3: Status strings and finite branch meanings.

Status field value	Finite branch meaning
finite_object	the row contains the finite matrix/vector/scalar object for the selected input tuple.
unavailable_rank_below_2	the support used for the finite two-plane object has rank below two, so the two-plane object is not returned.
not_residual_le_tolerance	the row exists and its residual is greater than the tolerance field carried by the row.
coarse_frontier	the row uses a coarse frontier region selected by the directed-frontier cap.
law_fit_eligible=0	the row is present but excluded from the birth-relative fit route by its row fields.
law_fit_eligible=1	the row is present in the birth-relative fit route.

3.4 Integrated execution command

The integrated root entrypoint for the source surface documented here is `run_cnna.py`. It dispatches to `cnna.analysis_entrypoints.run_cnna_analysis.main` and invokes the row functions exported by `cnna.derived_quantities`. The following command generates the row families for the integrated row surface:

```

cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 3 \
  --levels 4 5 6 \
  --modes linear log saturating \
  --states live record sym \
  --max-regions 1 \
  --max-boundary 600 \

```

```

--max-interior 900 \
--directed-frontier-cap 6 \
--algebra-tol 1e-10 \
--algebra-max-iter 8 \
--gns-tol 1e-10 \
--growth-schedule layer_batch \
--outdir outputs/core_derived_documentation_run

```

A smaller finite run with the same entrypoint generates a reduced row surface for layout and source-route checks:

```

cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 3 \
  --levels 4 5 \
  --modes linear \
  --states live record sym \
  --max-regions 1 \
  --max-boundary 250 \
  --max-interior 350 \
  --directed-frontier-cap 5 \
  --algebra-tol 1e-10 \
  --algebra-max-iter 8 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/core_derived_reduced

```

The schedule `layer_batch` constructs all `GrowthStep` objects of one causal depth from the same pre-commit state before committing the births. The schedule `slot_step` applies one slot closure at a time. Both schedules are implemented by `EmptySetGrowthModel`; the command above selects the causal-layer schedule. Other root scripts are single-surface entrypoints. They are not read in this source-surface section. A focused runner is stated only in the later section whose rows it generates.

Command argument meanings. The arguments of the integrated command select finite constructors, row emitters, and numerical caps. They do not introduce additional mathematical objects beyond the source objects listed in Table 5.

Table 4: Arguments of the integrated execution command.

Argument	Value in the command	Finite role in the generated row surface
<code>--branching</code>	3	arity b of the finite rooted tree; each node has at most b child slots and saturation is measured as occupied slots over b
<code>--levels</code>	4 5 6	terminal depths L for the finite approximants; each listed value produces a separate finite growth object and downstream response rows
<code>--modes</code>	linear log saturating	selector for <code>_mode_scale</code> in the birth-weight construction; <code>rule_coefficients</code> depends on <code>--branching</code> only
<code>--states</code>	live record sym	edge-layer readout: <code>record</code> keeps birth edges, <code>live</code> includes later updates, and <code>sym</code> reads the symmetrized edge matrix
<code>--max-regions</code>	1	number of region records emitted per model before region row construction stops
<code>--max-boundary</code>	600	boundary-size cap for Schur-DtN and directed-response matrices

Argument	Value in the command	Finite role in the generated row surface
<code>--max-interior</code>	900	interior-size cap for finite elimination in Schur–DtN and directed-response calculations
<code>--directed-frontier-cap</code>	6	finite frontier level used by directed-region constructors when full terminal boundaries exceed the selected cap
<code>--algebra-tol</code>	<code>1e-10</code>	tolerance used in finite algebra closure, span projection, and residual comparisons involving algebraic bases
<code>--algebra-max-iter</code>	8	iteration cap for the finite multiplication/star closure procedure in the response-algebra layer
<code>--gns-tol</code>	<code>1e-10</code>	tolerance for support, rank, Gram-spectrum, and GNS quotient decisions
<code>--growth-schedule</code>	<code>layer_batch</code>	growth update route: all slots of one causal depth are evaluated from the same pre-commit state and then committed
<code>--outdir</code>	<code>outputs/core_derived_documentation_run</code>	directory receiving the generated CSV row families for the stated command

3.5 Tested numerical environment and licensing

The documented snapshot was tested under Ubuntu 26.04 LTS on Linux 7.0.0-27-generic (x86_64, glibc 2.43), using CPython 3.14.4 built with GCC 15.2.0 and NumPy 2.3.5. Explicit NumPy allocations with `dtype=float` resolve to `numpy.float64`, hence IEEE 754 binary64 arithmetic, in this environment. This arithmetic precision is distinct from the package-level constant $\text{EPS} = 10^{-12}$ and from the algebraic and GNS decision tolerances 10^{-10} used by the stated command.

The numerical backend is identified by runtime mapping, dynamic-link inspection, and the configured Ubuntu package providers. NumPy links to the Ubuntu/Debian reference BLAS and LAPACK implementations, both version 3.12.1, supplied by `libblas3` and `liblapack3` package version 3.12.1-7ubuntu1. After representative matrix multiplication, linear-solve, symmetric-eigenvalue, and SVD calls, `/proc/self/maps` records `/usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.1` and `/usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.1`; `ldd` on the relevant NumPy extension modules resolves the same system links. `threadpoolctl` 3.6.0 detects no thread-pool-managed native library, consistently with the configured reference providers. The complete runtime, linker, package-provider, and NumPy build record is preserved in `ENVIRONMENT.txt`; a concise interpretation is given in `ENVIRONMENT.md`. Package versions and tolerances are fixed, but bitwise identity across other BLAS/LAPACK backends, compiler builds, or processor feature sets is not asserted.

The Python software and software documentation in the ancillary package are distributed under the MIT License in `LICENSE`. That software license does not set the license of this article. Distribution of the article is governed by the license selected on its arXiv record.

The documented semantic edge cases are checked by

```
cd anc/CNNA_structured_core_pkg
python3 package_consistency/check_mathematical_contracts.py
python3 package_consistency/check_finite_structure_theorems.py
python3 package_consistency/check_nested_multiscale_handoff.py
```

The first check covers zero-mass normalization, normalized push-forward semantics, positive and negative GNS cyclicity controls, basis-change and nonfaithful-state quotient behavior, the support-restricted finite- J identity, and the entropy support cutoff. The second check covers finite growth replay, symmetric Schur positivity, exact finite GNS controls, sign-reversal covariance of the finite polar object, separation of backreaction observables, and prefix/carrier handoff. The third check constructs the growing first-order address-contrast carriers, verifies their exact non-unital corner

embeddings and composition, checks common-ambient Schur associativity, and records the separate response, density, GNS, and finite- J naturality residuals.

3.6 Regeneration boundary for row values

The current growth rule changes the finite growth object before every downstream response construction. Therefore numerical statements from earlier packages, reports, CSV files, or prior runs are not inherited. Each numerical statement in later sections is written together with a command of the form displayed above, a generated row family, and a readout from rows generated by the current package. This source-surface section names commands and source routes only; it does not state numerical row values.

3.7 Runtime degrees of freedom

Each runtime parameter is introduced at the first object layer that uses it. Parameters are not used as labels for objects outside the layer where they first enter.

Table 5: Runtime degrees of freedom and first source layer.

Parameter	First layer where it is used	Source role
--branching / b	finite growth record from the empty state	arity of the finite rooted tree
--levels / L	finite growth record from the empty state	terminal level and approximant size
--modes / m	finite birth-weight construction	selector for the implemented scalar map σ_m used on environment load
--growth-schedule / s	finite growth record from the empty state	update schedule: <code>slot_step</code> closes one slot at a time; <code>layer_batch</code> evaluates one causal layer before commit
--states	terminal state and live/record/sym edge layer	selected edge/state layer
--max-regions	region specification	number of emitted region records
--max-boundary	Schur-DtN and directed response	numerical boundary cap
--max-interior	Schur-DtN and directed response	numerical elimination cap
--directed-frontier-cap	directed response and large-level coverage	finite frontier level used for coarse boundary construction
--algebra-tol	response algebra and operator residual rows	tolerance for algebraic residual comparisons
--algebra-max-iter	finite response algebra closure	closure iteration cap
--gns-tol	finite GNS, support, rank, and projectivity rows	tolerance for support/rank decisions

The table is a parameter taxonomy, not a claim that every listed quantity is a physical coupling. In the reference interpretation, b is the varied structural index and L is the finite-size cutoff. The mode m selects one of three implemented birth-load maps, and s selects the update semantics; both alter the generated finite object and are therefore rule-family selectors, not derived consequences of b . State labels, region counts, caps, and tolerances select readout surfaces or numerical decision procedures. Any statement that only b is varied is restricted to a command in which all other selectors are explicitly fixed.

3.8 Finite quantities and code locations

The following table fixes the mathematical symbols used below. A symbol is introduced only at the source location where the corresponding finite object, map, matrix, row family, or residual first appears.

Table 6: Finite quantities used in the mathematical text and their source anchors.

Symbol	Source anchor and code object	Mathematical object described here
b	analysis_entrpoints/run_cnna_analysis.py core/emptyset_growth.py:41--55 -branching; rule_coefficients	finite arity used by the rooted growth constructor and by the coefficient functions of the finite rule.
L	analysis_entrpoints/run_cnna_analysis.py core/emptyset_growth.py:121--456 -levels; EmptySetGrowthModel	terminal level of the finite approximant.
m	analysis_entrpoints/run_cnna_analysis.py core/emptyset_growth.py:131--145 -modes; mode; _mode_scale	selector for the finite scalar map σ_m applied to environment load in the birth-weight construction.
s	analysis_entrpoints/run_cnna_analysis.py core/emptyset_growth.py:124--150 -growth-schedule; growth_schedule	finite update schedule for open birth slots; values: <code>slot_step</code> , <code>layer_batch</code> .
η	core/emptyset_growth.py:129--150 core/emptyset_growth.py:257--260 sibling_linearization; _linearize_open_slots	finite ordering used to linearize slots inside the same causal depth for comparison rows; values: <code>rank</code> , <code>reverse_rank</code> .
ε	config.py:3 EPS	package-level numerical tolerance constant used by source functions that import the package configuration.
ρ_{ref}	config.py:6--8 REFERENCE_MODE; REFERENCE_STATE; STRUCTURAL_RECORD_STATE	reference row selectors used by structural and comparison row families; values: <code>linear</code> , <code>live</code> , <code>record</code> .
$\mathcal{L}_{\text{gen}}$	config.py:10--12 GEN_LABELS	finite list of generator labels used by row surfaces and algebraic residual tables.
s	analysis_entrpoints/run_cnna_analysis.py core/directed_dtn.py:27--47 -states; state_directed_edges	selected edge layer used for response and transport computations; values: <code>live</code> , <code>record</code> , <code>sym</code> .
ε_{alg}	analysis_entrpoints/run_cnna_analysis.py core/response_algebra.py algebra_tol; algebra_closure	finite threshold used for basis insertion and algebra-closure residuals.
ε_{gns}	analysis_entrpoints/run_cnna_analysis.py core/response_algebra.py gns_tol; quotient_support	finite threshold for support rank and GNS quotient decisions.
$\mathcal{G}_{b,L,m,s}$	core/emptyset_growth.py:121--456 EmptySetGrowthModel	finite growth object containing nodes, events, address map, open birth slots during construction, growth steps, record edges, live edges, and node loads.
c_b	core/emptyset_growth.py:20--38 core/emptyset_growth.py:41--55 RuleCoefficients; rule_coefficients	finite tuple of coefficient values computed from b only.
σ_m	core/emptyset_growth.py:186--191 _mode_scale	finite scalar map selected by m and applied to environment load.
$g_{\text{birth}}(v)$	core/emptyset_growth.py:272--286 _make_growth_step; birth_g	birth load computed from $h(p, U_p)$ by the selected scalar map σ_m .
$V_{b,L}$	core/emptyset_growth.py:58--72 core/emptyset_growth.py:121--456 Node; self.nodes	finite node set produced by the constructor.
$\mathcal{E}_{b,L}$	core/emptyset_growth.py:75--96 core/emptyset_growth.py:161--183 Event; self.events	finite ordered event list whose order is a recorded linearization, not the causal-depth order.
$\mathcal{O}(p)$	core/emptyset_growth.py:99--107 core/emptyset_growth.py:242--255 OpenBirthSlot; open_birth_slots	finite set of unoccupied child slots of a parent node; saturation is reached when no such slot remains.
Γ_n	core/emptyset_growth.py:109--118 core/emptyset_growth.py:272--286 GrowthStep; _make_growth_step	finite update record for closing one open slot, carrying causal predecessors, environment load, birth weight, and linearization index.
$U_p, h(p, U_p)$	core/emptyset_growth.py:262--270 _birth_environment_load	occupied sibling set of a parent and its symmetric finite environment load; sibling contributions enter by set sum divided by $ U_p + 1$, not by birth order.
$d(v), \lambda(v), q(v)$	core/emptyset_growth.py:58--72 causal_depth; linearization_index; lapse_weight	causal depth, technical linearization index, and response weight recorded on a node.

Symbol	Source anchor and code object	Mathematical object described here
E^{rec}	core/emptyset_growth.py:155--156 core/emptyset_growth.py:340--344 record_edges	birth-edge dictionary.
E^{live}	core/emptyset_growth.py:155--156 core/emptyset_growth.py:340--368 live_edges	edge dictionary after birth edges and later live updates.
a	core/emptyset_growth.py:58--72 Node.address	finite address tuple of a node.
$\pi_k(a)$	core/projective_maps.py:21--24 prefix	prefix truncation of an address to level k .
Ω_L	core/projective_maps.py:27--29 terminal_carrier	terminal address support at level L with uniform carrier weights.
μ_L	core/projective_maps.py:32--53 terminal_state	normalized finite terminal state built from node load and incident response load.
$(\pi_k)_*\mu_L$	core/projective_maps.py:56--61 push_forward	finite prefix push-forward of a terminal distribution.
C_k and $\Omega_L \setminus C_k$	core/projective_maps.py:64--84 center_subset_prefix; split_distribution	deterministic first-born cylinder and its complement.
(F_1, F_2)	core/signed_current.py:22--62 orthonormal_pair; F1_F2_vectors	real two-vector plane extracted from finite address/current data.
$\omega_{F_1 F_2}$	core/signed_current.py:65--120 orientation_measurement	finite signed scalar on the whole-network F_1/F_2 plane.
B and I	core/schur_dtn.py:45--70 core/schur_dtn.py:73--112 boundary; interior	finite boundary and interior node lists for a selected cut.
K	core/schur_dtn.py:45--70 core/schur_dtn.py:73--112 local Laplacian blocks	finite matrix split into $K_{BB}, K_{BI}, K_{IB}, K_{II}$.
Λ_B	core/schur_dtn.py:23--29 core/schur_dtn.py:45--70 SchurResponse; schur_response	finite Schur-DtN response matrix $K_{BB} - K_{BI}K_{II}^{-1}K_{IB}$ as computed by solve.
R	core/schur_dtn.py:33--42 core/schur_dtn.py:115--156 RegionSpec; build_sibling_regions	finite sibling-cone region record with boundary and interior lists.
Q_R	core/schur_dtn.py:159--198 boundary_role_basis; compress_response	finite role-basis matrix and compression map $Q_R^\top \Lambda_B Q_R$.
M_B^{dir}	core/directed_dtn.py:16--24 core/directed_dtn.py:98--166 DirectedSchurResponse; directed_schur_response	finite directed boundary response matrix.
S_B and A_B	core/directed_dtn.py:402--405 split_symmetric_skew	symmetric and skew parts $(M + M^\top)/2$ and $(M - M^\top)/2$.
J_B^{fin}	core/directed_dtn.py:408--455 j_finite_object_from_SA	finite support-restricted comparison object built from S_B and A_B .
$\mathcal{A}_R$	core/response_algebra.py core/response_algebra.py AlgebraBasis; build_algebras	finite matrix algebra basis generated by response matrices.
ρ_R	core/response_algebra.py core/region_response.py:52--97 positive_density_from_response; positive_support_density_from_response	positive finite density matrix extracted from response data.
$\varphi_R(X)$	core/response_algebra.py state_value; state_gram	finite state functional and its Gram matrix over the algebra basis.
$\mathcal{H}_R^{\text{GNS}}$	core/response_algebra.py GNSData; measurement_gns	finite quotient-support GNS data, full-basis representation residuals, and explicit cyclic-span rank.
$\mathcal{L}_R$	core/region_response.py:101--151 core/algebra_support.py:85--323 RegionalResponse; local_operator_algebras	regional response object and finite local support algebras.
P_U	core/algebra_support.py:169--218 role_support_indices; role_support_projector	diagonal projector for a selected role support.
$\mathcal{C}(\mathcal{A})$	core/algebra_support.py:426--449 commutant_algebra_basis; basis_to_algebra	finite commutant nullspace and corresponding algebra basis.
τ_R	core/algebra_support.py:452--636 finite_tomita_star_metrics	finite star-map/Tomita metric data over the generated algebra and density.

Symbol	Source anchor and code object	Mathematical object described here
$\hat{\mu}_R$	core/compressed_state.py:24--117 compressed_state_object; compressed_projective_distribution	compressed carrier/GNS distribution over finite role/address keys.
Q_R^{ref}	core/refined_port_basis.py:36--138 refined_boundary_role_basis; refined_support_algebra	refined finite boundary/port basis and its support algebra.
$S(\rho)$	core/entropy_quantities.py von_neumann_entropy_no_epsilon	finite entropy from positive eigenvalues of a density matrix.
$D(\rho \sigma)$	core/entropy_quantities.py quantum_relative_entropy_no_epsilon	finite relative entropy with explicit support handling.
$\text{JSD}(\rho, \sigma)$	core/entropy_quantities.py jensen_shannon_divergence_no_epsilon	finite Jensen–Shannon divergence of two density matrices.
$\delta_{L \rightarrow L'}$	derived_quantities/operator_stack_residuals.py:23--289 derived_quantities/operator_stack_interface_residuals.py:93--314 interlevel_operator_rows; interface row transforms	finite residuals after projection to common labels.
$T_{L \rightarrow L'}$	derived_quantities/directed_current_transport.py:68--112 derived_quantities/directed_current_transport.py:191--341 boundary_prefix_transport_matrix; transport rows	finite boundary-prefix transport matrix and composition comparison.
$\sigma_J^{\text{loc}}, \sigma_J^{\text{glob}}$	derived_quantities/j_orientation_projectivity.py:159--439 derived_quantities/local_global_j_sign_relation.py:139--334 orientation and sign row families	finite local and global sign assignments compared by row functions.
$B_L = A_L^{\text{live}} - A_L^{\text{record}}$	derived_quantities/backreaction_current.py backreaction_current_rows	finite live-record skew-response difference on the same model, region, and level.
$\alpha = L - L_{\text{birth}}$	derived_quantities/backreaction_birth_relative.py birth_relative_backreaction_rows; birth_relative_zero_crossing_rows	explicit sibling-root birth-relative age and finite interpolation of the signed skew-norm difference.
Δ_{lin}	derived_quantities/birth_linearization_invariance.py:65--112 birth_linearization_invariance_rows	finite address-level residual comparing the implemented rank and reverse_rank linearizations; it samples the order-dependence surface of slot_step and layer_batch .
$r_{\bullet}$	derived_quantities/baseline.py:21--254 derived_quantities/computed_status_rows.py:26--149 row-family functions; aggregation rows	generated row-family outputs attached to previously constructed finite objects.

4 Reference scope for discrete finite operations

The CNNA core presented here is a discrete-finite construction. Its primitive generated objects are finite rooted address trees, finite weighted edge surfaces, finite carrier arrays, and finite matrices. The references in this document are used only to identify standard background for finite trees, finite graphs, algebraic graph operations, Schur complements, finite matrix decompositions, and finite-dimensional operator-algebraic terminology [1, 2, 3, 4, 5].

These references are not external axioms for the CNNA construction. The live, record, sym, carrier, finite J , and backreaction objects are generated from an empty initial configuration by the explicit finite rules implemented in the source. The branching index b enters the coefficient map, while L , the mode selector m , the update schedule $\mathfrak{s}$, region selectors, caps, and numerical tolerances have separately declared roles. The cited literature documents established mathematical language in which some of these finite operations can be described. No continuum space, no integral calculus, no differential operator, and no analytic boundary-value problem is introduced in the finite core.

The term Dirichlet-to-Neumann is used here in the discrete network sense: a finite boundary response matrix obtained by eliminating interior vertices through a Schur complement. In this document, the object is entirely finite and matrix-based. Boundary sets, interior sets, invertibility checks, solve residuals, and compressed responses are computed directly from the generated finite graph data. The relevant comparison literature is finite resistor-network response theory, not a continuum field theory [6, 7]. For weighted tree graphs, Gernandt and Rohleder likewise identify the discrete Dirichlet-to-Neumann matrix with a Laplacian Schur complement and study what tree and edge-weight information can be reconstructed from it [13]. Their inverse problem starts from a symmetric positively weighted tree and a leaf boundary; the present construction instead generates directed record/live weights and uses the Schur map in the forward provenance chain. The citation therefore supplies a close tree-DtN comparator, not an identity of problems or conclusions.

5 Derivation graph

5.1 Global object order

The documentation follows the first-use order of the objects. When one Python module contributes to several object layers, only the function block whose inputs are already available is introduced at that point.

Order	Object introduced at first use
0	$\emptyset$
1	finite growth record with open slots and GrowthStep
1a	birth-relative set and coordinate complement of each accepted new address; causal-layer complement for layer_batch
1b	directed record/live coupling of the new carrier directions to parent, ancestors, and sibling slots
2	birth-linearization residual rows
3	terminal carrier and prefix maps
4	whole-network F1/F2 signed current
5	symmetric Schur–DtN response
6	region specification and role compression
7	directed response and <i>S/A</i> split
8	finite <i>J</i> comparison
9	finite response algebra and GNS data
10	regional response and local support algebras
11a	compressed carrier/GNS state layer
11b	refined boundary/port basis
12	entropy quantities
13	interlevel residuals and directed transport
14	local/global <i>J</i> sign comparison
15	live-record backreaction current
16	birth-relative transforms
17	row aggregation

Derived row modules do not introduce primitive degrees of freedom. They attach finite measurements to objects constructed earlier in the chain.

5.2 Function-block placement

The placement unit is sometimes a function block rather than a module. The following table records the section order used by the filled source-ordered construction.

Table 7: Derivation layers used for source placement.

Layer	Object layer	Objects introduced	Row functions attached here
0	Source tree and finite notation	package tree; finite arrays; finite dictionaries	none
1	Finite scalar, vector, matrix, and distribution operations	Frobenius norm; total variation; normalization	metric helpers only
2	Finite growth record and complement coupling from the empty state	RuleCoefficients ; Node ; Event ; OpenBirthSlot ; GrowthStep ; EmptySetGrowthModel ; birth-relative carrier complements; record/live directed couplings; growth schedule	genesis, parameter, rule-constant, birth-linearization rows
3	Terminal addresses, carriers, and prefix maps	terminal carrier; terminal state; prefix; push-forward; split distribution	projection, carrier, sector rows
4	Whole-network F1/F2 axes and signed current scalar	orthonormal_pair ; F1_F2_vectors ; orientation_measurement	orientation and history-delta rows
5	Symmetric Schur–DtN response on a selected cut	SchurResponse ; selected boundary/interior response	local response-density and rank/kernel rows
6	Region specification and boundary role compression	RegionSpec ; sibling regions; role basis; compressed response	regional response rows
7	Directed response and <i>S/A</i> decomposition	DirectedSchurResponse ; directed local matrix; <i>S</i> ; <i>A</i>	directed construction, decomposition, skew rows
8	Finite <i>J</i> comparison on <i>S/A</i> support	support-restricted finite comparison from <i>S</i> and <i>A</i>	<i>J</i> snapshot rows
9	Finite matrix algebra and finite GNS data	algebra basis; density; Gram matrix; quotient support	algebra/GNS rows
10	Regional response object and local support algebras	regional support density; local generators; support algebras	local algebra rows when included
11	Finite local algebra checks	commutator, cyclicity, commutant, Tomita, split metrics	check rows when included

Layer	Object layer	Objects introduced	Row functions attached here
12	Compressed carrier/GNS state and projective distribution	compressed state object; compressed projective distribution	compressed carrier rows
13	Refined boundary/port basis	refined role basis; refined regional response	refined-basis rows when included
14	Entropy functionals and record/live entropy distances	entropy, relative entropy, Jensen–Shannon, record/live metrics	entropy rows when included
15	Interlevel operator residuals and interface transforms	label-span projection; matrix log; commutator delta; seam rows	operator residual rows
16	Directed transport and local/global J sign comparison	boundary-prefix transport; sign comparison domains	transport and sign rows
17	Live-record backreaction current	$B_L = A_{\text{live},L} - A_{\text{record},L}$	backreaction rows
18	Birth-relative and branching transforms	birth-relative age; interpolation pair; branching comparison	zero-crossing and branching rows
19	Nested multiscale carrier handoff	first-order address contrasts; growing real carriers; corner embeddings; common-ambient Schur controls	nested carrier, composition, state/GNS, and finite- J naturality rows
20	Row aggregation layer	row-family collection	aggregation rows
S	Support source files	package constants and row-file writers	config and I/O helpers

6 Core module map

Table 8: Core modules in derivation order.

Module	Mathematical object	Role in derivation	Principal code objects
<code>core/math_utils.py</code>	finite numerical primitives	norms, total-variation distances, normalization, row metric extraction	<code>fro</code> ; <code>tv</code> ; <code>normalize_dict</code> ; <code>normalize_distribution</code> ; <code>tv_arr</code>
<code>core/emptyset_growth.py</code>	empty-set growth and complement-coupling object	rooted b-ary address tree with explicit open slots, birth-relative carrier complements, growth steps, update schedule, and record/live directed edge states	<code>class RuleCoefficients</code> ; <code>class Node</code> ; <code>class Event</code> ; <code>class OpenBirthSlot</code> ; <code>class GrowthStep</code> ; <code>class EmptySetGrowthModel</code> ; <code>rule_coefficients</code> ; <code>open_birth_slots</code>
<code>core/projective_maps.py</code>	carrier and prefix projections	terminal probability carrier and finite push-forward maps	<code>prefix</code> ; <code>terminal_carrier</code> ; <code>terminal_state</code> ; <code>push_forward</code> ; <code>center_subset_prefix</code> ; <code>split_distribution</code> ; <code>response_energy_distribution</code>
<code>core/signed_current.py</code>	two-axis signed-current plane	F1/F2 real vectors and orientation measurement	<code>orthonormal_pair</code> ; <code>F1_F2_vectors</code> ; <code>orientation_measurement</code>
<code>core/schur_dtn.py</code>	symmetric Schur-DtN response	finite Laplace restriction, Schur complement, role basis compression	<code>class SchurResponse</code> ; <code>class RegionSpec</code> ; <code>schur_response</code> ; <code>schur_response_from_model</code> ; <code>build_sibling_regions</code> ; <code>normalized_indicator</code> ; <code>boundary_role_basis</code> ; <code>compress_response</code> ; <code>role_for_boundary_node</code> ; <code>atom_key</code>
<code>core/nested_multiscale.py</code>	growing address-contrast carrier	Helmert slot contrasts, nested role basis, corner maps, common-ambient Schur matrix, support graph	<code>class NestedRoleBasis</code> ; <code>class CommonAmbientSchurResult</code> ; <code>nested_address_contrast_basis</code> ; <code>label_inclusion_matrix</code> ; <code>role_intertwining_metrics</code> ; <code>corner_embed</code> ; <code>corner_restrict</code> ; <code>common_ambient_interlevel_schur</code> ; <code>resolved_support_graph_metrics</code>
<code>core/directed_dtn.py</code>	directed Schur-DtN response	non-symmetric local response, S/A decomposition input, finite J object	<code>class DirectedSchurResponse</code> ; <code>state_directed_edges</code> ; <code>directed_local_matrix</code> ; <code>directed_schur_response</code> ; <code>directed_schur_response_from_model</code> ; <code>directed_environment_port_response_from_model</code> ; <code>build_coarse_frontier_sibling_regions</code> ; <code>region_frontier_level</code> ; <code>region_is_coarse_frontier</code> ; <code>signed_role_basis_F1F2</code> ; <code>split_symmetric_skew</code> ; <code>j_finite_object_from_SA</code>
<code>core/response_algebra.py</code>	finite response algebra and GNS data	generator closure, state Gram quotient support, density projection	<code>class AlgebraBasis</code> ; <code>class GNSData</code> ; <code>rank_one</code> ; <code>add_to_basis</code> ; <code>algebra_closure</code> ; <code>project_coeffs</code> ; <code>reconstruct</code> ; <code>span_residual</code> ; <code>positive_density_from_response</code> ; <code>gens_from_response</code> ; <code>build_algebras</code> ; <code>state_value</code> ; ...
<code>core/compressed_state.py</code>	compressed carrier state	role/atom state compression and projective distributions	<code>compressed_state_object</code> ; <code>compressed_projective_distribution</code> ; <code>values_for_keys</code> ; <code>key_is_center</code> ; <code>sector_profile_tv</code>
<code>core/operator_test_config.py</code>	finite parameter iterator	explicit finite scan configuration	<code>class OperatorTestConfig</code> ; <code>iter_growth_models</code>
<code>core/region_response.py</code>	regional response object	region-level density and signed orientation metrics	<code>class RegionalResponse</code> ; <code>sibling_pair_regions</code> ; <code>region_size_meta</code> ; <code>positive_support_density_from_response</code> ; <code>regional_response</code> ; <code>signed_orientation_metrics</code>

Module	Mathematical object	Role in derivation	Principal code objects
<code>core/algebra_support.py</code>	local operator algebra support	role support, commutants, Tomita metrics, node-level embeddings	<code>operator_scale_metadata</code> ; <code>local_operator_generators</code> ; <code>role_coordinate_mapping</code> ; <code>role_support_indices</code> ; <code>role_support_projector</code> ; <code>independently_restricted_operator_algebra</code> ; <code>independent_embedding_metrics</code> ; <code>local_operator_algebras</code> ; <code>max_span_residual</code> ; <code>max_star_residual</code> ; <code>joined_algebra</code> ; <code>commutator_metrics</code> ; ...
<code>core/entropy_quantities.py</code>	finite entropy functionals	tolerance-resolved support entropy, relative entropy, Jensen-Shannon divergence	<code>von_neumann_entropy_no_epsilon</code> ; <code>quantum_relative_entropy_no_epsilon</code> ; <code>jensen_shannon_divergence_no_epsilon</code> ; <code>record_live_entropy_metrics</code>
<code>core/refined_port_basis.py</code>	refined port basis	boundary role refinement and refined support algebras	<code>refined_boundary_role_basis</code> ; <code>refined_regional_response</code> ; <code>refined_support_algebra</code>

7 Derived-quantity module map

Table 9: Derived modules in derivation order.

Module	Mathematical row family	Role in derivation	Principal code objects
<code>derived_quantities/baseline.py</code>	baseline derivation ledger	genesis, projection grammar, carrier states, orientation, region/GNS rows	<code>level_models</code> ; <code>genesis_rows</code> ; <code>projection_grammar_rows</code> ; <code>carrier_state_rows</code> ; <code>orientation_rows</code> ; <code>region_and_gns_rows</code>
<code>derived_quantities/gns_cyclicity_controls.py</code>	finite GNS cyclicity controls	full-algebra positive controls and proper-ambient-subalgebra negative controls	<code>finite_gns_cyclicity_control_rows</code>
<code>derived_quantities/finite_structure_theorem_controls.py</code>	six finite structural controls	replay, Schur positivity, GNS, sign reversal, backreaction separation, and prefix handoff	<code>finite_structure_theorem_control_rows</code>
<code>derived_quantities/nested_multiscale_handoff.py</code>	nested carrier and corner-handoff rows	growing address-coordinate roles, exact carrier embeddings, common-ambient Schur, conditional state/GNS, and finite- J naturality	<code>build_nested_snapshot</code> ; <code>nested_carrier_rows</code> ; <code>nested_handoff_rows</code> ; <code>common_ambient_schur_rows</code> ; <code>nested_multiscale_summary_rows</code>
<code>derived_quantities/birth_linearization_invariance.py</code>	birth-linearization residuals	rank/reverse-rank address-level comparisons for <code>slot_step</code> and <code>layer_batch</code>	<code>birth_linearization_invariance_rows</code>
<code>derived_quantities/rank_kernel_conditions.py</code>	rank-kernel measurements	local response density, coverage, constants, regional rank/kernel rows	<code>local_s_response_density</code> ; <code>measure_rank_kernel_condition_rows</code> ; <code>construction_coverage_rows</code> ; <code>fixed_rule_constants_measurement_rows</code> ; <code>regional_rank_kernel_condition_rows</code> ; <code>terminal_density_state_measurement_rows</code>
<code>derived_quantities/compressed_state_projection_consistency.py</code>	compressed-state projectivity check	compressed carrier distributions and construction track ledger	<code>compressed_carrier_state_rows</code> ; <code>state_construction_track_ledger_rows</code>
<code>derived_quantities/structural_ledgers.py</code>	live-record structural ledger	parameter purity, history delta, carrier effects, state sectors	<code>is_reference_model</code> ; <code>parameter_purity_rows</code> ; <code>live_record_structural_calculation_ledger_rows</code> ; <code>live_record_history_delta_ledger_rows</code> ; <code>carrier_effect_operator_rows</code> ; <code>state_sector_decomposition_rows</code>
<code>derived_quantities/operator_stack_residuals.py</code>	interlevel operator residuals	role projection, GNS/algebra, CE/modular residuals	label-projection helpers; <code>matrix_log_spd</code> ; <code>commutator_delta</code> ; <code>interlevel_operator_rows</code>
<code>derived_quantities/operator_stack_interface_residuals.py</code>	operator-stack seam residuals	seam localization, level scaling, map naturality	<code>operator_stack_seam_localization_rows</code> ; <code>operator_stack_live_record_sym_comparison_rows</code> ; <code>operator_stack_level_scaling_rows</code> ; <code>operator_stack_map_naturality_measurement_rows</code> ; <code>write_operator_stack_residual_report</code>
<code>derived_quantities/operator_stack_consistency_checks.py</code>	operator-stack consistency checks	role dimensions, generator rank, coordinate gauge, response model, skew survival	<code>operator_stack_role_dimension_condition_note_rows</code> ; <code>operator_stack_generator_rank_condition_note_rows</code> ; <code>operator_stack_coordinate_gauge_condition_note_rows</code> ; <code>operator_stack_boundary_response_model_condition_note_rows</code> ; <code>operator_stack_skew_survival_condition_note_rows</code> ; <code>operator_stack_consistency_checks_summary_rows</code> ; <code>write_operator_stack_condition_consistency_report</code>

Module	Mathematical row family	Role in derivation	Principal code objects
<code>derived_quantities/directed_operator_stack.py</code>	directed operator stack	directed response construction, S/A rows, skew survival, finite J comparison	<code>directed_response_construction_rows</code> ; <code>directed_sa_decomposition_rows</code> ; <code>directed_signed_role_survival_rows</code> ; <code>directed_boundary_response_model_comparison_rows</code> ; <code>directed_skew_survival_rows</code> ; <code>directed_operator_transport_rows</code> ; <code>directed_j_finite_object_comparison_rows</code> ; <code>directed_operator_stack_summary_rows</code> ; <code>write_directed_operator_stack_report</code>
<code>derived_quantities/directed_current_transport.py</code>	directed current transport	boundary prefix transport, composition, J-orientation transport, large-L coverage	<code>directed_current_transport_rows</code> ; <code>directed_current_composition_rows</code> ; <code>directed_j_orientation_summary_rows</code> ; <code>write_directed_current_transport_report</code> ; <code>directed_large_L_region_coverage_rows</code> ; <code>directed_large_L_coverage_summary_rows</code> ; <code>write_directed_region_coverage_report</code>
<code>derived_quantities/j_orientation_projectivity.py</code>	J-orientation projectivity	orientation snapshots, interlevel transport, composition, sign scope	<code>j_orientation_snapshot_rows</code> ; <code>j_orientation_transport_rows</code> ; <code>j_orientation_composition_rows</code> ; <code>j_orientation_sign_scope_rows</code> ; <code>j_orientation_projectivity_summary_rows</code> ; <code>write_b_j_orientation_projectivity_report</code> ; <code>a_j_orientation_snapshot_rows</code> ; <code>a_j_orientation_transport_rows</code> ; <code>a_j_orientation_composition_rows</code> ; <code>a_j_orientation_projectivity_summary_rows</code> ; ...
<code>derived_quantities/local_global_j_sign_relation.py</code>	local/global J sign relation	global boundary sign rows and local comparison	<code>global_j_boundary_sign_relation_rows</code> ; <code>local_global_j_sign_comparison_rows</code> ; <code>local_global_j_sign_summary_rows</code> ; <code>write_local_global_j_sign_relation_report</code>
<code>derived_quantities/backreaction_current.py</code>	live-record backreaction current	regional A_live-A_record rows, growth, sensitivity, transport	<code>backreaction_current_rows</code> ; <code>backreaction_growth_profile_rows</code> ; <code>backreaction_region_parameter_sensitivity_rows</code> ; <code>backreaction_transport_rows</code> ; <code>backreaction_current_summary_rows</code> ; <code>write_live_record_backreaction_current_report</code>
<code>derived_quantities/backreaction_birth_relative.py</code>	birth-relative backreaction current	birth-relative ages, zero crossing, branching and region family summaries	<code>birth_relative_backreaction_rows</code> ; <code>birth_relative_zero_crossing_rows</code> ; <code>birth_relative_mode_branching_rows</code> ; <code>birth_relative_region_family_rows</code> ; <code>birth_relative_summary_rows</code> ; <code>write_birth_relative_backreaction_current_report</code>
<code>derived_quantities/backreaction_branching_shift.py</code>	branching zero-crossing shift	branching comparisons of birth-relative crossing locations	<code>read_csv_rows</code> ; <code>deduplicate_mode_branching_rows</code> ; <code>branching_shift_curve_rows</code> ; <code>branching_shift_summary_rows</code> ; <code>write_branching_zero_crossing_shift_report</code>
<code>derived_quantities/computed_status_rows.py</code>	row aggregation	final tabular row aggregation and markdown writer; not a mathematical constructor	<code>operator_stack_residuals_classification_rows</code> ; <code>final_computed_status_rows_full</code> ; <code>write_markdown_full</code>

8 Core construction initialized by the empty configuration

8.1 Finite arithmetic layer

Code map. The module `core/math_utils.py` defines finite numerical primitives: Frobenius norm, total variation distance, probability normalization, and row-metric extraction. These are not CNNA axioms; they are deterministic finite arithmetic functions used by all later measurements.

Definition 8.1 (Finite probability normalization). *For a finite vector or dictionary, the implementation first replaces every negative entry, and for array inputs every non-finite entry, by zero. Writing the resulting values as x_i^+ and $s = \sum_i x_i^+$, the implemented normalization is*

$$\mathcal{N}_\varepsilon(x) = \begin{cases} (x_i^+/s)_i, & s > \varepsilon, \\ (1/n, \dots, 1/n), & s \leq \varepsilon \text{ and } n > 0, \\ (), & n = 0. \end{cases}$$

For a non-empty zero-mass input the fallback is therefore the uniform distribution, not a zero-mass object. Dictionary normalization applies the same non-negative clipping; its keys are retained.

Lemma 8.2 (Total variation is a finite seminorm). *For two finite probability vectors p, q , the implemented quantity*

$$\text{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i|$$

vanishes when $p = q$ and is non-negative.

Proof. Object. The object is the pair of finite arrays used by `tv_arr` or dictionaries used by `tv`. **Proof.** Each summand $|p_i - q_i|$ is non-negative; hence the sum and one-half multiple are non-negative. If $p = q$, each summand is zero. **Counter-check.** The converse requires the index supports to be compared after the common-key completion used by the dictionary implementation. **Conclusion.** The quantity is an exact finite discrepancy measure, not a fitted parameter. $\square$

8.2 Empty-set growth object

Code map. The module `core/emptyset_growth.py` contains `RuleCoefficients`, `Node`, `Event`, `OpenBirthSlot`, `GrowthStep`, and `EmptySetGrowthModel`. The constructor records `empty_state`, creates the root by `birth_root_from_empty`, and then processes finite open child slots. The source separates causal depth, sibling slot rank, technical linearization index, and update schedule. It does not treat the total event-list order as the source temporal object.

Definition 8.3 (Finite CNNA growth approximant with schedule). *For branching arity $b \geq 2$, level $L \geq 0$, mode $m \in \{\text{linear}, \text{log}, \text{saturating}\}$, and growth schedule $\mathfrak{s} \in \{\text{slot_step}, \text{layer_batch}\}$, the finite approximant is*

$$G_{b,L,m,\mathfrak{s}} = (V, \mathcal{E}, E_{\text{record}}, E_{\text{live}}, a, \ell, p, d, \tau, r, \lambda, g, g_{\text{birth}}, q). \quad (8.1)$$

Here V is the finite node list; $a(v)$ is the finite address; $\ell(v)$ is the address level; $p(v)$ is the parent except for the root; $d(v)$ is the recorded causal depth; $\tau(v)$ is the event-list index of the selected finite execution; $r(v)$ is the sibling slot rank; $\lambda(v)$ is the linearization index inside the schedule; $g(v)$ is the current node load; $g_{\text{birth}}(v)$ is the load at the birth update; and $q(v)$ is the recorded response weight `lapse_weight`. The edge dictionaries E_{record} and E_{live} are finite directed weighted dictionaries.

Definition 8.4 (Initial boundary event). *The transition from the empty state to the root is recorded as `birth_root_from_empty`. It is the finite initial boundary event of the implemented construction. It is not derived from a prior response matrix, because before the root there is no finite parent, boundary, sibling set, or conductance environment in the source tree.*

Definition 8.5 (Open birth slot, saturation, and finite growth step). *For a parent node p , `open_birth_slots` returns one `OpenBirthSlot` for each unoccupied rank $r \in \{0, \dots, b-1\}$. A parent is saturated exactly when its `child_by_rank` dictionary has cardinality b . A `GrowthStep` closes one such slot and records the slot, causal predecessors, occupied sibling slots before the update, environment load, birth weight, response weight, and linearization index.*

Definition 8.6 (Growth schedules). *The schedule `slot_step` linearizes open slots and applies one `GrowthStep` immediately. The schedule `layer_batch` first constructs all `GrowthStep` objects for the open slots of one causal depth from the same pre-commit state, then commits the births, and then adds sibling relations by slot rank. Thus `layer_batch` removes slot-sibling update order inside a causal layer; it retains parent-before-child causal order and slot-rank structure.*

Definition 8.7 (Symmetric environment load of an open slot). *For a parent node p and the occupied sibling set U_p read from `child_by_rank`, the function `_birth_environment_load` computes the finite scalar*

$$h(p, U_p) = g(p) + \sum_{a \in \text{Anc}(p)} \frac{g(a)}{(\text{depth}(a, p) + 1)^2} + \begin{cases} \frac{\sum_{s \in U_p} g(s)}{|U_p| + 1}, & U_p \neq \emptyset, \\ 0, & U_p = \emptyset. \end{cases} \quad (8.2)$$

The sibling contribution is a set sum over occupied sibling nodes. It has no factor depending on the enumeration position of a sibling in a previous birth sequence.

Definition 8.8 (Branching-derived rule coefficients). *The function `rule_coefficients` maps the arity b to the tuple c_b with fields and values*

$$\begin{aligned} c_b.\text{env_birth_edge_bias} &= \frac{1}{(b+1)^2}, & c_b.\text{forward_rank_bias} &= \frac{1}{b+3}, \\ c_b.\text{backward_base} &= \frac{1}{2} + \frac{1}{2(b+2)}, & c_b.\text{backward_rank_bias} &= \frac{1}{(b+1)(b+3)}, \\ c_b.\text{ancestor_backreaction} &= \frac{1}{b(b+1)}, & c_b.\text{ancestor_node_load} &= \frac{1}{(b+2)^2}, \\ c_b.\text{sibling_backreaction} &= \frac{1}{b(b+2)}, & c_b.\text{sibling_forward_asym} &= 1 + \frac{1}{b+1}, \\ c_b.\text{sibling_backward_asym} &= 1 - \frac{1}{b+1}, & c_b.\text{sibling_node_load} &= \frac{1}{(b+7)^2}, \\ c_b.\text{child_env_load} &= \frac{1}{(b+7)^2}. \end{aligned} \quad (8.3)$$

The tuple in Equation (8.3) is evaluated from the single integer b .

Definition 8.9 (Mode-scale map and birth weight). *The function `_mode_scale` applies one of the three finite scalar maps*

$$\sigma_m(x) = \begin{cases} x, & m = \text{linear}, \\ \log(1 + \max(0, x)), & m = \text{log}, \\ \frac{x}{1 + |x|}, & m = \text{saturating}. \end{cases} \quad (8.4)$$

For a slot of parent p , the corresponding *GrowthStep* stores

$$g_{\text{birth}} = 1 + \frac{\sigma_m(h(p, U_p))}{1 + \ell(p) + b}. \quad (8.5)$$

The same value is copied to *lapse_weight*. The current load of the new child is then incremented by the separate child-environment update recorded in *child_env_load*.

Definition 8.10 (Base birth-edge weights). For sibling rank $r \in \{0, \dots, b-1\}$ let $\rho_r = (r+1)/b$ and

$$w_0 = 1 + c_b.\text{env_birth_edge_bias} \sigma_m(h(p, U_p)). \quad (8.6)$$

The directed parent-child weights inserted at birth are

$$\begin{aligned} w_{p \rightarrow v} &= w_0(1 + c_b.\text{forward_rank_bias} \rho_r), \\ w_{v \rightarrow p} &= w_0(c_b.\text{backward_base} + c_b.\text{backward_rank_bias} \rho_r). \end{aligned} \quad (8.7)$$

These two directed weights are inserted into both the record and live edge dictionaries.

Proposition 8.11 (Node count of the complete finite approximant). For $G_{b,L,m,s}$ constructed by *EmptySetGrowthModel*,

$$|V| = \sum_{k=0}^L b^k. \quad (8.8)$$

Proof. Object. The object is the list `self.nodes`. **Step 1.** The constructor creates exactly one root node at level 0. **Step 2.** For every node at depth $k < L$, `open_birth_slots` contains exactly the unoccupied ranks of that parent. **Step 3.** The schedules differ in commit order, but both close every open slot of the processed depth exactly once. **Step 4.** A closed slot of a level k parent creates one child at level $k+1$. By induction, level k contains b^k addresses. **Counter-check.** The method `node_count_expected` implements the same finite sum. A mismatch is a construction mismatch for the selected schedule, not a continuum statement. **Conclusion.** The count is an exact finite identity of the generated approximant. $\square$

Proposition 8.12 (Causal depth is separated from linearization index). For non-root nodes, $d(v) = d(p(v)) + 1$ is the causal-depth relation recorded by the slot. The quantity $\lambda(v)$ is the finite ordering index of the selected schedule and is not the parent-before-child relation.

Proof. Object. The object is the pair `OpenBirthSlot` and `GrowthStep`. **Step 1.** An open slot stores `causal_depth = parent.level + 1`. **Step 2.** `_linearize_open_slots` sorts slots by `rank` or `reverse_rank` after causal depth and parent fields. **Step 3.** `GrowthStep.linearization_index` records this schedule index. **Counter-check.** The derived row family `birth_linearization_invariance_rows` compares `rank` and `reverse_rank` by address-level residuals. In this package the `layer_batch` route is the linearization-insensitive route for that comparison, while `slot_step` records finite schedule dependence. **Conclusion.** The source separates causal order from slot linearization. $\square$

Proposition 8.13 (Layer-batch removes sampled slot-linearization dependence at address level). For the implemented comparison $\eta = \text{rank}$ against $\eta = \text{reverse_rank}$, the `layer_batch` route gives identical address-indexed node fields and edge dictionaries up to the reported numerical tolerance. The `slot_step` route need not do so.

Proof. Object. The objects are `OpenBirthSlot`, `GrowthStep`, and the edge dictionaries indexed back to node addresses. **Step 1.** In `layer_batch`, the list of `GrowthStep` records is built before the corresponding children are committed. Therefore each step reads the same pre-commit occupied sibling set for its parent. **Step 2.** The environment load uses the symmetric set expression $\sum_{s \in U_p} g(s)/(|U_p|+1)$, so it does not depend on an order of the occupied siblings. **Step 3.** The birth weights are stored inside the `GrowthStep` record and are then used by `_apply_growth_step`. **Step 4.** Batch sibling relations are added after commit by grouping children by parent and sorting by `sibling_rank`; the relation is a slot-rank relation, not a commit-order relation. **Counter-check.** The package exposes the `rank/reverse_rank` comparison, not a full permutation harness. The `slot_step` route can have non-zero residuals because each slot is applied before the next one is evaluated. **Conclusion.** The causal-layer schedule separates the finite parent-before-child relation from the sampled ordering of slots inside one causal depth. $\square$

Proposition 8.14 (Record-live separation). *At birth, a parent-child directed pair is inserted into both E_{record} and E_{live} . Later ancestor and slot-sibling updates are inserted only into E_{live} and node loads, not into E_{record} .*

Proof. Object. The objects are the dictionaries `record_edges` and `live_edges`. **Step 1.** The birth edge weights $w_{\rightarrow}$ and $w_{\leftarrow}$ are computed from the `GrowthStep` record and inserted into both dictionaries in `_apply_growth_step`. **Step 2.** Ancestor updates and sibling-slot relation updates call `add_edge` only on `live_edges`. **Step 3.** Under `layer_batch`, the live updates of a causal layer use the `GrowthStep` fields evaluated from the frozen pre-commit state; record edges still store only the parent-child birth relation. **Step 4.** The same update changes current node loads g , while g_{birth} remains the value recorded at the birth update. **Counter-check.** `slot_step` and `layer_batch` can produce different live layers because `slot_step` evaluates later slots after earlier commits. Record rows are closer to schedule-independent structure because they omit ancestor and slot-sibling live updates, but their numerical weights still come from the selected schedule. **Conclusion.** The code realizes two distinct finite layers: birth record and current live state. $\square$

Remark 8.15 (The sym edge layer is a readout, not a schedule). The state selector `sym` is implemented by symmetrizing the adjacency read from `live_edges`. It cancels directed skew information at readout level. It is not a separate growth schedule and it does not imply that `slot_step` and `layer_batch` have identical edge weights. The finite comparison represented in `birth_linearization_invariance_rows` compares `rank` against `reverse_rank` inside a selected schedule.

Row-generation command: causal-layer route.

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 3 \
  --levels 2 3 4 \
  --modes linear log saturating \
  --states live record sym \
  --max-regions 1 \
  --max-boundary 250 \
  --max-interior 350 \
  --directed-frontier-cap 5 \
  --algebra-tol 1e-10 \
  --algebra-max-iter 8 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/growth_carrier_f1f2_layer_batch
```

Comparison command: slot-step route.

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
--branching 3 \
--levels 2 3 4 \
--modes linear log saturating \
--states live record sym \
--max-regions 1 \
--max-boundary 250 \
--max-interior 350 \
--directed-frontier-cap 5 \
--algebra-tol 1e-10 \
--algebra-max-iter 8 \
--gns-tol 1e-10 \
--growth-schedule slot_step \
--outdir outputs/growth_carrier_flf2_slot_step_comparison
```

Generated row families. The following readouts use only the two commands displayed above. They are not imported from earlier packages or reports.

Table 10: Growth readout generated under the current growth rule.

Row family	Schedule	Readout
emptyset_genesis_measurement.csv	layer_batch, slot_step	For $b = 3$, $L = 2, 3, 4$, the generated node counts are 13, 40, 121, matching $\sum_{k=0}^L 3^k$; event counts are 14, 41, 122.
level_count_measurement.csv	layer_batch, slot_step	Every level-count row has <code>ok=1</code> ; generated counts are 3^k at level $k \leq L$.
birth_linearization_invariance.csv	layer_batch	Across 27 rows, <code>edge_linf_by_address</code> , <code>edge_l1_by_address</code> , <code>node_g_linf_by_address</code> , and <code>birth_g_linf_by_address</code> are all zero; the largest orientation delta is $3.6429 \cdot 10^{-17}$.
birth_linearization_invariance.csv	slot_step	Across 27 rows, the comparison records finite schedule dependence: max edge $\ell_\infty = 9.2504 \cdot 10^{-2}$, max edge $\ell_1 = 9.2217$, max node-load $\ell_\infty = 2.5150 \cdot 10^{-1}$, max birth-weight $\ell_\infty = 2.3975 \cdot 10^{-1}$, max orientation delta $2.6895 \cdot 10^{-2}$.

8.3 Birth-relative complement meaning of CNNA

Terminology and code map. The name CNNA is used here for a *ComplemeNt Net Architecture*. The word complement refers first to the finite carrier enlargement implemented by `EmptySetGrowthModel`; the word network refers to the directed couplings inserted when that enlargement is committed. These are two distinct steps and should not be conflated.

Definition 8.16 (Birth-relative vertex complement). *Let e be an accepted non-root birth event and let v_e be its new node. Write V_e^- for the vertex set immediately before `_apply_growth_step` commits v_e , and V_e^+ for the vertex set immediately afterward. Then*

$$V_e^+ = V_e^- \dot{\cup} \{v_e\}, \quad \{v_e\} = V_e^+ \setminus V_e^-, \quad V_e^- = V_e^+ \setminus \{v_e\}. \quad (8.9)$$

The dot denotes disjoint union. The statement is relative to this finite update; it does not presuppose a fixed infinite universal carrier. The root event is the base case $V_0^- = \emptyset$, $V_0^+ = \{r\}$; it creates the first coordinate but has no prior network to which it could be coupled.

Definition 8.17 (Coordinate complement). *For a finite vertex set V , let $\mathbb{R}^V$ denote the real coordinate space with standard basis $(\delta_v)_{v \in V}$. Zero extension along the inclusion $V_e^- \hookrightarrow V_e^+$ gives an injective map*

$$\iota_e : \mathbb{R}^{V_e^-} \longrightarrow \mathbb{R}^{V_e^+}, \quad (\iota_e x)(u) = \begin{cases} x(u), & u \in V_e^-, \\ 0, & u = v_e. \end{cases} \quad (8.10)$$

The post-birth carrier has the exact direct-sum decomposition

$$\mathbb{R}^{V_e^+} = \iota_e(\mathbb{R}^{V_e^-}) \oplus \text{span}_{\mathbb{R}}\{\delta_{v_e}\}. \quad (8.11)$$

Thus the newborn node contributes one new real coordinate direction complementary to the embedded pre-birth carrier.

Proposition 8.18 (Exactness and schedule scope of the complement step). *Equation (8.11) is exact for every committed node. For `slot_step` it is also the primitive update decomposition. For `layer_batch`, let N_d be the nodes born at causal depth d from one frozen pre-commit state V_d^- . The schedule-invariant causal statement is the batch decomposition*

$$V_d^+ = V_d^- \dot{\cup} N_d, \quad \mathbb{R}^{V_d^+} = \iota_d(\mathbb{R}^{V_d^-}) \oplus \text{span}_{\mathbb{R}}\{\delta_v : v \in N_d\}. \quad (8.12)$$

The nodewise decompositions inside that batch remain exact, but their internal commit order is a technical linearization and is not promoted to an additional causal order.

Proof. Object. The objects are the address dictionary `addr_to_id`, the node list `nodes`, and the selected `GrowthStep` records. **Step 1.** Before committing a step, the source rejects any address already present in `addr_to_id`. Hence the new address is not in V^- . **Step 2.** The source appends exactly one node and enters exactly one new address, which proves Equation (8.9). **Step 3.** Every vector on V^+ splits uniquely into its restriction to V^- and its coefficient at the new standard basis vector; this proves Equation (8.11). **Step 4.** Under `layer_batch`, all `GrowthStep` records of one depth are constructed before any of their nodes is committed. Distinct open slots have distinct addresses, so the same argument applied to the complete newborn set gives Equation (8.12). **Counter-check.** The direct sum is a statement about the standard address-coordinate carrier. It is not yet a statement about orthogonality in a later response metric, GNS metric, or state-dependent inner product. **Conclusion.** The complement step is a finite exact carrier identity with an explicit schedule scope. $\square$

Definition 8.19 (Directed coupling of the complement). *Order the coordinates of one update as old vertices followed by the newborn set N . The record and live adjacency matrices have finite block forms*

$$W_{\text{record}}^+ = \begin{pmatrix} W_{\text{record}}^- & C_{\text{parent}} \\ D_{\text{parent}} & 0 \end{pmatrix}, \quad W_{\text{live}}^+ = \begin{pmatrix} W_{\text{live}}^- & C_{\text{live}} \\ D_{\text{live}} & W_{NN} \end{pmatrix}. \quad (8.13)$$

Here C_{parent} and D_{parent} contain the two directed parent-child birth weights of Equation (8.7). The live off-diagonal blocks additionally contain the implemented ancestor and sibling couplings, while W_{NN} contains the slot-rank sibling relations among nodes born in the same layer. In a one-node `slot_step` update, $W_{NN} = 0$. Because the forward and backward coefficients are generally unequal, neither block matrix is required to be symmetric.

Proposition 8.20 (The complement is coupled rather than independent). *The carrier identity in Equation (8.11) does not make the newborn line dynamically independent of the old carrier. For every non-root birth, both parent-child entries in the record and live dictionaries are non-zero and are generated from the same finite growth step. The live layer can add further directed couplings to causal predecessors and sibling slots and can update existing node loads.*

Proof. Object. The objects are `record_edges`, `live_edges`, and the node-load field g . **Step 1.** Equation (8.7) gives positive parent-to-child and child-to-parent weights for every admitted b , mode, and finite environment load generated by the source. **Step 2.** `_apply_growth_step`

inserts both weights into both edge dictionaries. **Step 3.** The ancestor loop inserts additional child-to-predecessor live entries and changes predecessor loads. **Step 4.** The sibling routines insert unequal forward/backward live entries according to slot rank. **Counter-check.** The set/direct-sum complement would remain true even if all cross-couplings were zero. Hence complementarity alone does not prove network interaction. **Conclusion.** CNNA combines an exact carrier complement with a separate generated coupling rule. $\square$

Remark 8.21 (What complement does not mean). The term does not denote the graph complement, a Boolean complement, an orthogonal complement with respect to a derived matrix S , a commutant of an operator algebra, or a claim that the newborn degree of freedom is statistically independent. Relative to the terminal carrier one also has the tautology $V_L = (V_L \setminus \{v\}) \dot{\cup} \{v\}$ for every vertex, but that terminal identity discards the birth provenance and is not the definition used here. Such stronger structures occur, if at all, only after additional definitions and tests. The primitive complement claims are the birth- and layer-relative Equations (8.9), (8.11), and (8.12).

Proposition 8.22 (Complement-to-response bridge and its non-implication). *Choose a finite selected set $B \dot{\cup} I$ after the directed complement couplings have been generated. Form the directed local matrix K^{dir} from the ordered weights and, when the interior solve is accepted, define*

$$K_{II}^{\text{dir}} X = K_{IB}^{\text{dir}}, \quad \Lambda_B^{\text{dir}} = K_{BB}^{\text{dir}} - K_{BI}^{\text{dir}} X, \quad \Lambda_B^{\text{dir}} = S_B + A_B, \quad (8.14)$$

with $S_B^T = S_B$ and $A_B^T = -A_B$ as in Equation (8.20). The carrier-complement identities do not imply $A_B \neq 0$. A non-zero skew response requires both asymmetric generated coupling data and survival of that asymmetry under the selected boundary/interior elimination and compression.

Proof. Object. The objects are the directed edge matrix, its local response matrix, and the accepted Schur response. **Step 1.** Equations (8.11) and (8.12) only enlarge the coordinate domain. **Step 2.** Equation (8.13) supplies the ordered couplings from which K^{dir} is built. **Step 3.** The Schur operation eliminates I and returns a boundary matrix. **Step 4.** Every real square matrix has the exact symmetric/skew split. **Counter-check.** If the generated weights are replaced by a symmetric control, then the local Laplacian and its accepted Schur complement are symmetric and $A_B = 0$ up to numerical residual. Conversely, a non-symmetric local matrix can still yield zero skew part on a particular compressed boundary. **Conclusion.** The logical chain is complement enlargement, directed coupling, Schur reduction, and only then symmetric/skew decomposition; none of these arrows may be reversed without an additional theorem. $\square$

8.4 Carrier and prefix projections

Code map. The module `core/projective_maps.py` defines address truncation `prefix`, terminal carrier extraction, terminal state extraction, push-forward maps, center subsets, and response-energy distributions.

Definition 8.23 (Normalized prefix push-forward). *Let Ω_L be the terminal address set at level L . For $0 \leq k \leq L$, define $\pi_k : \Omega_L \rightarrow \Omega_k$ by address truncation and let the raw fiber sum be*

$$(\pi_{k*}^{\text{raw}} \mu)(u) = \sum_{\omega: \pi_k(\omega)=u} \mu(\omega). \quad (8.15)$$

The function `push_forward` returns $\mathcal{N}_\varepsilon(\pi_{k*}^{\text{raw}} \mu)$. Thus it agrees with the ordinary push-forward for an already normalized non-negative input, but it renormalizes a general finite input and applies the uniform fallback when the raw output has no positive mass.

Definition 8.24 (Terminal carrier and terminal diagonal state). *The terminal carrier is the uniform finite distribution on Ω_L . For a terminal node v , let $I_{\text{abs}}(v)$ be the sum of absolute weights of all selected directed edges incident to v , counting incoming and outgoing incidences exactly as in the edge dictionary. The unnormalized terminal-state weight is*

$$d_s(v) = \max(\varepsilon, g(v) + 0.05 I_{\text{abs}}(v)),$$

*and the terminal diagonal state is $\mathcal{N}_\varepsilon(d_s)$. The **record** state reads **record_edges**; **live** and **sym** use the same positive proxy from **live_edges**. Consequently the terminal positive proxy does not distinguish **live** from **sym**, although their directed response readouts do. Signed current is never inserted into this positive carrier.*

Lemma 8.25 (Probability preservation for normalized inputs). *If μ is a finite non-negative probability distribution, then **push_forward** returns the ordinary prefix push-forward and its total mass is one.*

Proof. Object. The object is the dictionary passed to **push_forward**. **Step 1.** Every terminal key has exactly one truncated prefix. **Step 2.** The raw fiber sum in Equation (8.15) partitions the source support and therefore has total mass one. **Step 3.** Applying $\mathcal{N}_\varepsilon$ to a non-negative mass-one dictionary leaves it unchanged. **Counter-check.** For a non-normalized input, the implementation returns the normalized raw push-forward rather than preserving the original total mass. **Conclusion.** Exact projectivity statements apply to the normalized probability inputs used by the carrier □

Generated row families.

Table 11: Carrier and prefix readout generated under the current growth rule.

Row family	Schedule	Readout
<code>projection_grammar.csv</code>	<code>layer_batch, slot_step</code>	9 generated rows per schedule; address truncation flag is 1 in every row; status string is <code>projection_grammar_residual_le_tolerance</code> .
<code>carrier_projectivity.csv</code>	<code>layer_batch, slot_step</code>	9 generated rows per schedule; max carrier TV is 0 for both schedules.
<code>terminal_diagonal_state_measurement.csv</code>	<code>layer_batch</code>	Trace range of the generated terminal diagonal states is $[1, 1 + 2.22 \cdot 10^{-16}]$.
<code>terminal_diagonal_state_projectivity.csv</code>	<code>layer_batch</code>	Max terminal-state projectivity TV is $9.8999 \cdot 10^{-4}$.
<code>terminal_diagonal_state_projectivity.csv</code>	<code>slot_step</code>	Max terminal-state projectivity TV is $2.3796 \cdot 10^{-2}$. This comparison belongs to the schedule surface, not to the carrier identity.

8.5 Whole-network F1/F2 axes and signed current scalar

The historical row label **F2_birth_order** denotes the canonical sibling-slot rank encoded in node addresses. It is not the execution order of the event list; under **layer_batch**, births at one causal depth are evaluated from the same pre-commit state.

Code map. The module `core/signed_current.py` constructs an orthonormal F1/F2 pair and an orientation measurement from finite node-address data and a selected edge dictionary. This whole-network current is introduced before Schur–DtN response; it does not require a selected boundary/interior cut.

Definition 8.26 (Whole-network F1/F2 plane). *The function `F1_F2_vectors` assigns to a node v with address $a(v) = (r_1, \dots, r_{\ell(v)})$ the finite coordinates*

$$F_1(v) = \begin{cases} \ell(v)/L, & L > 0, \\ 0, & L = 0, \end{cases} \quad F_2(v) = \sum_{d=1}^{\ell(v)} \frac{1}{d} \frac{r_d - (b-1)/2}{\max(1, (b-1)/2)}. \quad (8.16)$$

For the root the sum is empty and hence zero. The function `orthonormal_pair` mean-centers and Gram-Schmidt orthonormalizes these two vectors when the finite rank is above the implemented fallback threshold.

Definition 8.27 (Whole-network signed-current scalar). *For a selected edge layer, `orientation_measurement` computes the directed norm, symmetric norm, skew norm, relative skew norm, and the scalar circulation in the F1/F2 plane. For `state=sym`, the skew part is set to zero at readout level.*

Proposition 8.28 (F1/F2 current is independent of Schur reduction). *The whole-network F1/F2 scalar is computed from the finite edge dictionary and node-address vectors before any Schur boundary/interior split is chosen.*

Proof. Object. The object is the output of `orientation_measurement`. **Step 1.** `F1_F2_vectors` reads only node levels and address ranks. **Step 2.** `orientation_measurement` reads `record_edges` for `record`, `live_edges` for `live`, and the symmetrized readout for `sym`. **Step 3.** No boundary list, interior list, Schur solve, or response compression is called. **Counter-check.** Regional F1/F2 role bases in later directed-response sections are different objects; they are not this whole-network scalar. **Conclusion.** The F1/F2 current is a finite whole-network measurement attached to the growth and carrier layer. $\square$

Generated row families.

Table 12: Whole-network F1/F2 readout generated under the current growth rule.

Row family	Schedule	Readout
<code>f1f2_skew_orientation_measurement.csv</code>	<code>layer_batch</code>	27 generated rows; max post-Gram-Schmidt $ F_1 \cdot F_2 = 1.3245 \cdot 10^{-17}$; max relative skew is $2.4203 \cdot 10^{-1}$ for <code>live</code> , $2.6829 \cdot 10^{-1}$ for <code>record</code> , and 0 for <code>sym</code> .
<code>f1f2_skew_orientation_measurement.csv</code>	<code>layer_batch</code>	Max absolute F1/F2 plane circulation is $3.2839 \cdot 10^{-2}$ for <code>live</code> , $3.2712 \cdot 10^{-2}$ for <code>record</code> , and 0 for <code>sym</code> .
<code>live_record_history_delta.csv</code>	<code>layer_batch</code>	9 generated rows; max directed live-minus-record Frobenius norm is $7.6674 \cdot 10^{-1}$; max terminal-state TV between live and record is $1.0644 \cdot 10^{-4}$.
<code>f1f2_skew_orientation_measurement.csv</code>	<code>slot_step</code>	Comparison route: max absolute F1/F2 plane circulation is $4.5201 \cdot 10^{-2}$ for <code>live</code> , $4.7143 \cdot 10^{-2}$ for <code>record</code> , and 0 for <code>sym</code> .

8.6 Symmetric Schur–DtN response on a selected finite cut

Code map. The module `core/schur_dtn.py` defines `SchurResponse`, `RegionSpec`, `schur_response`, `schur_response_from_model`, sibling-region construction, boundary role bases, response compression, and atom keys. The selected finite cut enters here for the first time: a boundary list B , an interior list I , and a finite local matrix K are passed to the Schur reduction.

Definition 8.29 (Finite Schur–DtN response). *Given a finite symmetric local matrix K and a disjoint boundary/interior split $B \sqcup I$, the finite response matrix is*

$$\Lambda_B = K_{BB} - K_{BI}X, \quad K_{II}X = K_{IB}. \quad (8.17)$$

when the interior block K_{II} satisfies the condition-number guard. The source uses `np.linalg.solve`; it does not form an explicit matrix inverse.

Proposition 8.30 (Accepted Schur response records a finite linear solve). *If `schur_response` returns `ok=True`, then the returned object contains the Schur matrix, the condition number of K_{II} , the solve residual $\|K_{II}X - K_{IB}\|_F / \max(\|K_{IB}\|_F, \epsilon)$, and the row-sum residual of Λ_B .*

Proof. Object. The object is `SchurResponse`. **Step 1.** The function rejects empty boundary lists, boundary/interior overlap, and interior blocks above the condition guard. **Step 2.** It solves $K_{II}X = K_{IB}$. **Step 3.** It returns the response in Equation (8.17), symmetrized as $(\Lambda_B + \Lambda_B^T)/2$. In `schur_response_from_model`, diagonal degrees include conductance from selected nodes to the full current network, while off-diagonal entries are retained only between selected nodes. Omitted external neighbors therefore remain as grounded diagonal load. **Counter-check.** The resulting open-grounded response need not satisfy $\Lambda_B \mathbf{1} = 0$; the row-sum residual measures this finite leakage. Non-accepted branches return an error string and no response matrix. **Conclusion.** The accepted row is a finite open-grounded Schur reduction with recorded numerical guards, not necessarily a conservative response of an isolated selected subgraph. $\square$

Execution command. The following command generated the row families read in this subsection and the two following subsections. It uses the current growth rule and the causal-layer schedule.

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 3 \
  --levels 4 \
  --modes linear log saturating \
  --states live record sym \
  --max-regions 2 \
  --max-boundary 600 \
  --max-interior 900 \
  --directed-frontier-cap 6 \
  --algebra-tol 1e-10 \
  --algebra-max-iter 8 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/schur_directed_j_layer_batch_reduced
```

Table 13: Generated Schur response rows under the current growth rule.

Row family	Rows	Readout
<code>regional_schur_dtn_response_measurement.csv</code>	18	All rows have <code>response_ok=1</code> . Max <code>cond_KII</code> is 2.5511, max solve residual is $1.9261 \cdot 10^{-16}$, max row-sum residual is 2.1398, and the compressed response dimension is 3.
<code>regional_schur_dtn_response_measurement.csv</code>	18	Max compressed relative skew is $1.4703 \cdot 10^{-17}$ for <code>live</code> , $9.0776 \cdot 10^{-18}$ for <code>record</code> , and $1.4703 \cdot 10^{-17}$ for <code>sym</code> .

8.7 Region specification and boundary role compression

Code map. `RegionSpec` stores the selected sibling-cone cut. `build_sibling_regions` chooses an LCA, two adjacent child cones, a boundary consisting of the LCA plus frontier nodes, and an interior inside the two cones. `boundary_role_basis` builds finite role columns and `compress_response` returns $Q^T \Lambda_B Q$.

Definition 8.31 (Boundary role compression). *For a selected region R , let Q_R be the matrix whose columns are the surviving Gram-Schmidt role vectors. The compressed response is*

$$\Lambda_R^{\text{role}} = Q_R^T \Lambda_B Q_R. \quad (8.18)$$

The current symmetric Schur rows use the role names `C_lca`, `B1_front`, and `B2_front` in the accepted compressed basis.

Proposition 8.32 (Role compression is a finite congruence). *For an accepted Schur response and a returned role basis Q_R , the compressed response is the finite matrix product $Q_R^T \Lambda_B Q_R$.*

Proof. Object. The objects are `boundary_role_basis` and `compress_response`. **Step 1.** Role indicator columns are built from boundary node lists. **Step 2.** Modified Gram-Schmidt discards columns whose residual norm is below tolerance. **Step 3.** The returned Q_R is used in the congruence of Equation (8.18). **Counter-check.** If all role columns vanish numerically, the code supplies a constant fallback column. **Conclusion.** The role layer changes finite coordinates on the selected boundary; it does not add a new response solve. $\square$

8.8 Directed response and S/A decomposition

Code map. The module `core/directed_dtn.py` constructs directed local matrices and directed Schur responses without pair-averaging the edge dictionary and without final Schur symmetrization. The whole-network F1/F2 scalar has already been introduced; this subsection uses a selected boundary/interior cut.

Definition 8.33 (Directed local response matrix). *For directed weights W_{ij} , the out-degree sign relation is*

$$K_{ii} = \sum_j W_{ij}, \quad K_{ij} = -W_{ij} \quad (i \neq j). \quad (8.19)$$

The in-degree sign relation applies the same formula to W^T . The open-grounded response keeps external out-load on the diagonal; the closed-internal response removes this external diagonal load; the environment-port response represents outside load by one additional finite boundary port.

Definition 8.34 (Symmetric/skew split). *For a directed response matrix Λ , the finite split is*

$$S = \frac{1}{2}(\Lambda + \Lambda^T), \quad A = \frac{1}{2}(\Lambda - \Lambda^T). \quad (8.20)$$

The row families record $\|S\|_F$, $\|A\|_F$, $\|A\|_F/\|\Lambda\|_F$, symmetry residuals, and the signed F1/F2 plane scalar read from A .

Proposition 8.35 (Directed response separates symmetric and skew finite parts). *For every generated directed response row in the stated command, the returned matrix decomposes as $\Lambda = S + A$, with $S^T = S$ and $A^T = -A$ up to the recorded floating-point residuals.*

Proof. Object. The object is the matrix returned by `directed_schur_response`. **Step 1.** The directed local matrix is built from ordered edge weights. **Step 2.** Schur elimination is applied without symmetrizing Λ . **Step 3.** `split_symmetric_skew` forms S and A according to Equation (8.20). **Counter-check.** The row family records separate open-grounded, closed-internal, and environment-port rows; row-sum residuals differ across these finite accounting choices. **Conclusion.** The selected-cut directed layer supplies finite symmetric and skew matrices, not a continuum operator. $\square$

Table 14: Generated directed-response rows under the current growth rule.

Row family	Rows	Readout
<code>directed_response_construction.csv</code>	54	All rows have <code>response_ok=1</code> . Max <code>cond_KII</code> is 3.0449, max solve residual is $1.9261 \cdot 10^{-16}$, max row-sum residual is 2.2103, and max relative skew is $1.1827 \cdot 10^{-1}$.
<code>directed_SA_decomposition.csv</code>	9	All rows are available. Max signed-plane relative skew is $1.9501 \cdot 10^{-3}$ for <code>live</code> , $2.0435 \cdot 10^{-3}$ for <code>record</code> , and $1.2106 \cdot 10^{-17}$ for <code>sym</code> .
<code>directed_SA_decomposition.csv</code>	9	Max $ \omega_{F1,F2} $ is $4.1050 \cdot 10^{-3}$ for <code>live</code> , $4.1128 \cdot 10^{-3}$ for <code>record</code> , and $2.4070 \cdot 10^{-17}$ for <code>sym</code> .

8.9 Finite J comparison on an S/A support

Code map. The function `j_finite_object_from_SA` reads a symmetric matrix S and a skew matrix A . It restricts to the positive spectral support of S , builds $J_{\text{raw}} = S^{-1/2}AS^{-1/2}$, extracts a polar factor by singular value decomposition, and records the residual for $J^2 = -P$ on the non-zero skew support.

Definition 8.36 (Finite support comparison). *Let P_S be the positive spectral support selected from S . On that finite support the code computes*

$$J_{\text{raw}} = S^{-1/2}AS^{-1/2}. \quad (8.21)$$

If the skew support has rank at least two, the SVD block defines a polar partial isometry $J = W_k V_k^T$ on the non-zero skew support. With $P = W_k W_k^T$, the code compares J^2 with $-P$ through $\|J^2 + P\|_F / \|P\|_F$. Unless the skew rank equals the full positive-support dimension, J is not asserted to be orthogonal on the whole S -support.

Proposition 8.37 (Finite J comparison is support-restricted). *The returned `available` field is one only when the positive support of S and the non-zero skew support of A have sufficient finite rank for the polar comparison.*

Proof. Object. The object is the dictionary returned by `j_finite_object_from_SA`. **Step 1.** The code diagonalizes the symmetric part of S and selects eigenvectors above tolerance. **Step 2.** It forms J_{raw} on that support according to Equation (8.21). **Step 3.** It computes an SVD of J_{raw} , retains singular directions above tolerance, and constructs the partial isometry and its range projector. **Counter-check.** If the positive support rank or skew support rank is below two, the function returns `available=0` with a reason string. Even skew rank is recorded but is not used as a separate acceptance gate. **Conclusion.** The finite J row tests $J^2 \simeq -P$ on the resolved non-zero skew support inside a selected matrix problem; it does not establish $J^2 = -I$ on a larger carrier. $\square$

Table 15: Generated finite-J comparison rows under the current growth rule.

Row family	Rows	Readout
directed_J_finite_object_comparison.csv	9	Six rows are available: three live and three record rows. The three sym rows return unavailable because the skew support rank is below two.
directed_J_finite_object_comparison.csv	9	Among available rows, $\max \ J^2 + P\ _F / \ P\ _F$ is 0, and $\max \text{Jraw_skew_error}$ is $2.0079 \cdot 10^{-16}$.

8.10 Finite response algebra and GNS quotient audit

Code map. The module `core/response_algebra.py` defines `AlgebraBasis`, finite closure under multiplication and transpose, response-generated generator systems, density matrices, state Gram matrices, tolerance-resolved quotient coordinates, and full-basis left-action audits. The module `derived_quantities/gns_cyclicity_controls.py` supplies independent positive and negative ambient-support cyclicity controls.

Definition 8.38 (Finite response algebra closure). *For a finite generator list $G_1, \dots, G_r \subset M_n(\mathbb{R})$, `algebra_closure` builds a numerically Frobenius-orthonormal matrix basis by repeated multiplication and transposition. A trial matrix is sequentially projected against the current basis and inserted only when the remaining Frobenius norm exceeds the algebra tolerance. Because the maintained basis is normalized by the same insertion routine, this is an orthogonal-projection residual, not a separate general least-squares solve. The loop stops when no new direction is inserted, the full matrix dimension is reached, or the iteration cap is exhausted.*

Definition 8.39 (Finite GNS Gram data and quotient coordinates). *Given a numerically orthonormal algebra basis $\{A_i\}_{i=1}^m$ and a density matrix D , the implemented state Gram matrix is*

$$G_{ij} = \text{tr}(DA_i^T A_j). \quad (8.22)$$

Let $G = U \text{diag}(\lambda) U^T$ and retain the eigenvalues satisfying $\lambda_i > \max(\varepsilon_{\text{gns}}, \varepsilon_{\text{gns}} \max(1, \lambda_{\text{max}}))$. A coefficient vector $c \in \mathbb{R}^m$ is represented on the finite quotient support by

$$q(c) = \text{diag}(\sqrt{\lambda_+}) U_+^T c.$$

For the identity coefficient vector c_I , $\Omega = q(c_I)$. If C_i is the coefficient matrix of left multiplication by A_i , the explicit cyclic-span test is

$$r_{\text{cyc}} = \text{rank}[q(C_1 c_I) \cdots q(C_m c_I)], \quad \text{cyclic exactly when } r_{\text{cyc}} = \text{rank } G. \quad (8.23)$$

Proposition 8.40 (Full-basis finite GNS audit). *For each finite algebra/density pair, `measurement_gns` computes the Gram support and audits every algebra-basis element: left-closure, transpose-adjoint compatibility, stability of the Gram nullspace under left multiplication, the identity-span residual, the equality $\pi(A_i)\Omega = [A_i]$, and the cyclic rank in Equation (8.23).*

Proof. Object. The object is `GNSData`. **Step 1.** Compute Equation (8.22) and its tolerance-resolved positive support. **Step 2.** Project the identity into the algebra basis and record its span residual and norm. **Step 3.** For every basis element, compute the left-multiplication coefficient matrix and the corresponding quotient representation. **Step 4.** Compare the represented transpose with the Euclidean adjoint, test images of Gram-null vectors, and compare $q(C_i c_I)$ with the direct class $q(e_i)$. **Step 5.** Compute the rank of all action-on- Ω columns. **Counter-check.** The separate file `finite\_gns\_cyclicity\_controls.csv` includes full-algebra positive controls and proper-subalgebra negative controls; hence the cyclicity criterion is not the tautology $\text{rank } G \leq m$. **Conclusion.** The row reports a finite left-regular GNS quotient audit over the full stored basis. No Hilbert completion, von Neumann closure, or infinite cyclicity statement is made. $\square$

Reproduction command.

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 3 \
  --levels 3 4 \
  --modes linear log saturating \
  --states live record sym \
  --max-regions 1 \
  --max-boundary 600 \
  --max-interior 900 \
  --directed-frontier-cap 6 \
  --algebra-tol 1e-10 \
  --algebra-max-iter 8 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/algebra_gns_state_entropy
```

Table 16: Generated algebra and finite-GNS rows under the current growth rule.

Row family	Rows	Readout
response_algebra_measurement.csv	54	Basis dimensions range from 3 to 9. All rows have saturated=1 . Max inclusion residuals are $7.7028 \cdot 10^{-16}$ for $A_{\text{in}} \rightarrow N$ and $2.5658 \cdot 10^{-13}$ for $N \rightarrow A_{\text{out}}$.
finite_gns_pre_representation_measurement.csv	54	GNS ranks and explicit cyclic-span ranks both range from 3 to 9; the cyclic-rank defect and Gram nullity are zero in every row. All rows have gns_psd_ok=1 , cyclic_ok=1 , representation_ok=1 , and full_basis_representation_checked=1 . Max identity-span residual is $1.4793 \cdot 10^{-13}$, max action-on-cyclic-vector residual is $2.5138 \cdot 10^{-13}$, max star-adjoint error is $1.0023 \cdot 10^{-12}$, and max left-closure residual is $2.5658 \cdot 10^{-13}$.
finite_gns_cyclicity_controls.csv	4	Both full- $M_2(\mathbb{R})$ positive controls are cyclic on their global GNS support. The diagonal/tracial and scalar/rank-one proper-subalgebra controls have cyclic ranks $2 < 4$ and $1 < 2$, respectively. All four rows have control_pass=1 .

8.11 Regional response object and local support algebras

Code map. The modules `core/region_response.py` and `core/algebra_support.py` build regional response containers, positive-support densities, role-coordinate maps, role support projectors, local generator systems, local algebra closures, joined algebras, commutator metrics, GNS support ranks, commutant bases, finite Tomita-star metrics, and split residuals.

Definition 8.41 (Local support algebra). *For a selected regional response, the code constructs role supports as index subsets of the finite boundary-role coordinate system. The local support algebra is the finite matrix algebra generated by the response-derived matrices after restriction to the selected role support.*

Proposition 8.42 (Local algebra rows are finite matrix comparisons). *The local algebra rows compare finite generated matrix algebras inside explicitly chosen ambient coordinates.*

Proof. **Object.** The objects are role-support projectors, restricted generator lists, and algebra bases. **Step 1.** Select role coordinates from the regional response object. **Step 2.** Restrict the

generator matrices to the selected support. **Step 3.** Build finite algebra bases by the same closure routine as above. **Step 4.** Compare inclusions, commutators, joins, commutants, cyclic spans, and split dimensions by finite ranks and finite norms. **Counter-check.** A non-zero residual is recorded as a finite matrix discrepancy in the selected coordinate system. **Conclusion.** The row families describe finite algebraic relations for the selected regional response. $\square$

Reproduction command.

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_local_operator_property_checks.py \
  --branching 3 \
  --levels 4 \
  --modes linear log saturating \
  --states live record sym \
  --max-regions 1 \
  --max-boundary 600 \
  --max-interior 900 \
  --directed-frontier-cap 6 \
  --algebra-tol 1e-10 \
  --rank-tol 1e-9 \
  --max-algebra-iter 8 \
  --tomita-matrix-sign_relations out_degree \
  --growth-schedule layer_batch \
  --outdir outputs/local_operator_algebras
```

Table 17: Generated local-support algebra rows under the current growth rule.

Row family	Rows	Readout
local_operator_isotony.csv	117	Max independent span residual is $7.7229 \cdot 10^{-13}$. Max relative common-ambient residual is $1.7277 \cdot 10^{-15}$.
local_operator_commutation.csv	81	Max relative commutator is 1.4142. This value is a finite norm ratio for the selected local algebras.
local_operator_additivity.csv	54	Max generated-union dimension is 9, max target-union dimension is 9, and max target-minus-generated dimension is 6. Max target-to-generated span residual is 1.
local_vacuum_cyclicity.csv	198	Local algebra dimensions range from 2 to 9. Global GNS support dimension is 9. Cyclic span ranks range from 2 to 9, and separating-kernel dimension is 0.
faithful_state.csv	81	Algebra dimensions range from 2 to 9. State Gram ranks range from 2 to 9. State Gram nullity is 0 in the generated rows.
tomita_standard_form_compatibility.csv	9	Max Tomita support dimension is 9. Max star-closure residual is $1.3174 \cdot 10^{-8}$, max star-involution error is $3.0981 \cdot 10^{-9}$, and max polar residual is $6.0951 \cdot 10^{-15}$.
commutant_duality.csv	27	Computed commutant dimensions range from 1 to 5. Complement-to-commutant residual ranges from 0 to 1, and commutant-to-complement residual ranges from $4.5776 \cdot 10^{-16}$ to 1.
split_factorization.csv	18	Max generated-join dimension is 9. Max tensor-product dimension count is 25. Max dimension residual is 16.

8.12 Compressed carrier/GNS state and projective distribution

Code map. The module `core/compressed_state.py` builds a finite compressed state object over role/address keys. The corresponding row functions in `derived_quantities/compressed_`

`state_projection_consistency.py` compare direct compression with projected compression at lower target levels.

Definition 8.43 (Compressed carrier state). *A compressed carrier state maps terminal or boundary atoms to finite role/address keys and sums the associated weights. The object contains the compressed key set, finite state weights, trace weights, response-energy weights, and residuals for the projected response density.*

Proposition 8.44 (Compressed projectivity is a route comparison). *For target levels $k < \ell$, the compressed projectivity rows compare direct compression at level k with the push-forward of compression at level ℓ .*

Proof. **Object.** The object is a pair of finite compressed distributions over role/address keys. **Step 1.** Build the compressed state at the finer target level. **Step 2.** Push it forward by address truncation. **Step 3.** Build the coarser compressed state directly. **Step 4.** Compare the two routes by total variation and sector profiles. **Counter-check.** A non-zero value records finite route dependence for the implemented compression. **Conclusion.** The row is a finite compatibility measurement for the compression map. $\square$

Table 18: Generated compressed-state rows under the current growth rule.

Row family	Rows	Readout
<code>compressed_carrier_state_objects.csv</code>	18	All rows have <code>object_available=1</code> . The compressed dimension is 3. Max projected-density residual is $6.4860 \cdot 10^{-16}$. Max <i>TV</i> between state and trace weights is $6.2347 \cdot 10^{-1}$; max <i>TV</i> between state and response-energy weights is $7.7539 \cdot 10^{-1}$.
<code>compressed_carrier_state_projectivity.csv</code>	9	Max trace-route total variation is $1.1102 \cdot 10^{-16}$. Max state-route total variation is $8.1912 \cdot 10^{-2}$. Max response-energy route total variation is $2.2005 \cdot 10^{-1}$.
<code>compressed_carrier_state_sector_decomposition.csv</code>	18	Max conditional-profile total variation is $1.8115 \cdot 10^{-3}$.
<code>compressed_carrier_state_composition.csv</code>	0	The generated file is empty for this reduced two-level run.

8.13 Refined boundary/port basis

Code map. The module `core/refined_port_basis.py` splits boundary role groups into finite subgroups, builds refined regional responses, and constructs refined support algebras.

Definition 8.45 (Refined role basis). *A refined role basis is a finite boundary-coordinate basis obtained by subdividing frontier role groups into named shells or chunks. It replaces a coarse role column by several normalized columns whose supports partition the original role support.*

Table 19: Generated refined-basis rows under the current growth rule.

Row family	Rows	Readout
<code>refined_port_basis_resolution.csv</code>	9	Coarse role dimension is 3 and max refined role dimension is 5. Max refined-role dimension gain is 2.
<code>collared_split_refined_basis.csv</code>	27	Max refined role dimension is 5. Max left and right dimensions are 10. Max generated-join dimension is 25 and max refined tensor-dimension residual is $7.5 \cdot 10^{-1}$.

8.14 Entropy functionals and record/live entropy distances

Code map. The module `core/entropy_quantities.py` defines finite entropy, finite relative entropy, Jensen–Shannon divergence, and record/live entropy metrics. The generated object-check rows evaluate these functions on densities attached to regional response and local algebra data. The same finite entropy and divergence functions are read again in Section 10.8 through the record/live row families.

Definition 8.46 (Tolerance-resolved finite entropy). *For a symmetric positive semidefinite density D , let*

$$\tau_D = \max(\varepsilon_{\text{gns}}, \varepsilon_{\text{gns}} \max(1, \lambda_{\max}(D))).$$

The function `von_neumann_entropy_no_epsilon` computes

$$S_{\tau_D}(D) = - \sum_{\lambda_i > \tau_D} \lambda_i \log \lambda_i. \quad (8.24)$$

No artificial floor is inserted inside the logarithm, but eigenvalues at or below the scale-aware cutoff are treated as numerical zero. Relative support inclusion is likewise tolerance-resolved; the routine returns an infinite value when the resolved support of ρ is not contained in that of σ .

Proposition 8.47 (The entropy routines expose their numerical support rule). *The entropy functions do not add an epsilon floor to force finite logarithms. They first resolve spectral support at the declared tolerance and test support leakage before the logarithmic trace expression is evaluated.*

Proof. Object. The objects are the spectra and support projectors computed in `entropy_quantities.py`. **Step 1.** Entropy sums eigenvalues above τ_D . **Step 2.** Relative entropy compares the trace leakage of ρ outside the resolved support projector of σ with a scale-aware tolerance. **Step 3.** Jensen–Shannon divergence uses the mixture density and the same support routine. **Counter-check.** Eigenvalues below the cutoff are not distinguished from exact zero, so the computation is not an exact symbolic-support calculation. **Conclusion.** The row values keep the implemented tolerance-resolved support relation visible without logarithmic epsilon flooring. $\square$

Table 20: Generated entropy rows under the current growth rule.

Row family	Rows	Readout
<code>record_live_relative_entropy.csv</code>	9	Max Jensen–Shannon live/record value is $6.4853 \cdot 10^{-3}$. Max live entropy is $4.1111 \cdot 10^{-1}$, and max record entropy is $2.6051 \cdot 10^{-1}$.
<code>tomita_entropy_flow_alignment.csv</code>	9	Max Tomita support dimension is 9. Max polar residual is $1.0297 \cdot 10^{-11}$. Max modular inverse-relation error is $2.7825 \cdot 10^{-1}$.
<code>multiscale_port_entropy_resolution.csv</code>	9	Max multiscale role dimension is 7. Max Jensen–Shannon live/record value on the multiscale basis is $1.8426 \cdot 10^{-2}$. Max record-tracking gain over the multiscale uniform comparison is $4.5778 \cdot 10^{-1}$.

9 Finite structural results and exact proof boundary

This section separates mathematical statements that follow from the finite definitions from numerical controls and from stronger claims that the present source does not establish. Exact statements are formulated over finite-dimensional real vector spaces and exact arithmetic. Their Python realization is additionally subject to the declared binary64 environment and tolerance rules. A passing control row checks the implementation on explicit finite cases; it is not substituted for the proof.

Table 21: Status of the six finite structural statements.

Statement	Status	Scope
Finite growth well-definedness	exact finite theorem	fixed admissible inputs, fixed sibling linearization, and the explicitly specified update rule.
Symmetric Schur positivity	exact conditional theorem	non-negative symmetrized conductances; invertible interior block. Positive definiteness requires explicit grounding.
Finite GNS quotient	exact finite theorem	a unital finite-dimensional real transpose-closed matrix algebra and a positive density matrix.
Global orientation reversal	exact conditional obstruction	sign-even data cannot distinguish the pair obtained by simultaneous reversal of the skew orientation.
Backreaction scalar separation	exact non-implication plus generated witness	distinguishes the matrix current, its norm, a difference of norms, and one oriented matrix coefficient.
Interlevel handoff	exact at address/carrier level; conditional for boundary matrices; failed for a strict operator-algebra handoff	prefix maps and uniform terminal carriers compose exactly. Current response/algebra maps retain non-zero discrepancies.

The implementation controls for this section are generated by

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 \
  package_consistency/check_finite_structure_theorems.py \
  --write-csv docs/FINITE_STRUCTURE_THEOREM_CONTROLS.csv
```

The six control rows pass in the documented environment. Their role is to detect implementation regressions in the finite hypotheses used below.

9.1 Well-definedness of the finite growth construction

Let

$$\mathcal{W}_{b,L} = \bigcup_{d=0}^L \{0, \dots, b-1\}^d$$

be the finite address set, including the empty word at depth zero. Besides $(b, L, m, \mathfrak{s})$, the source admits a fixed sibling linearization $\ell \in \{\text{rank}, \text{reverse_rank}\}$. The reference calculations use $\ell = \text{rank}$.

Theorem 9.1 (Finite growth well-definedness). *For every admissible tuple*

$$b \geq 2, \quad L \geq 0, \quad m \in \{\text{linear}, \text{log}, \text{saturating}\}, \quad \mathfrak{s} \in \{\text{slot_step}, \text{layer_batch}\},$$

and every fixed sibling linearization ℓ , the exact finite rule determines a unique tuple

$$\mathcal{G}_{b,L,m,\mathfrak{s},\ell} = (V, E_{\text{record}}, E_{\text{live}}, g, \mathcal{E}).$$

There is exactly one node v_a for each $a \in \mathcal{W}_{b,L}$, with parent v_{a-} for every non-empty word a . In particular,

$$|V| = |\mathcal{W}_{b,L}| = \sum_{d=0}^L b^d = \frac{b^{L+1} - 1}{b - 1}. \quad (9.1)$$

The event list, record-edge dictionary, live-edge dictionary, and node loads are finite and uniquely determined by the fixed inputs and rule equations.

Proof. The proof is by induction on depth. At depth zero, `_birth_root_from_empty` creates the unique empty-address node. Assume that after depth d the nodes are exactly the words of lengths at most d , with one node per address. Every depth- d word has precisely the unoccupied ranks $0, \dots, b-1$; concatenating a rank produces a distinct word of length $d+1$, and every word of length $d+1$ has exactly one such predecessor. The ordered slot list is a deterministic function of ℓ . For `slot_step`, each growth step is evaluated and committed in that order. For `layer_batch`, all step data at one depth are evaluated from the same pre-commit state and are then committed in the same deterministic slot order; the subsequent sibling relation loop is ordered by sibling rank. Every weight and load update is a finite arithmetic expression in the already determined state. Duplicate addresses are rejected explicitly. Hence the construction terminates after L depth loops and is unique. Counting the words gives Equation (9.1). $\square$

Remark 9.2 (Implementation scope). The theorem concerns the exact rule. The control file replays 96 finite configurations covering $b \in \{2, 3\}$, $L \in \{0, 1, 2, 3\}$, all three modes, both schedules, and both sibling linearizations. All 96 paired replays are exactly equal in the documented environment; address, parent, and finiteness checks have zero failures. This does not claim bitwise equality across different floating-point libraries or compiler builds.

9.2 Positivity of the symmetric finite response

For one state, let

$$c_{ij} = \frac{w_{ij} + w_{ji}}{2} \geq 0$$

be the undirected conductance used by the symmetric response route, and let L_C be its weighted graph Laplacian. The open-grounded local matrix constructed by `schur_response_from_model` is the principal block $K = (L_C)_{S,S}$ for $S = B \sqcup I$; edges from S to omitted vertices remain in its diagonal. The proof below is the finite quadratic-form proof of the standard Schur-complement positivity statement [4, 5].

Theorem 9.3 (Finite Schur positivity). *The local matrix K is positive semidefinite. If K_{II} is invertible, then it is positive definite and the exact Schur response*

$$\Lambda_B = K_{BB} - K_{BI}K_{II}^{-1}K_{IB} \quad (9.2)$$

is positive semidefinite. If every connected component of the conductance graph induced on S has a positive-conductance edge to the omitted complement, then K and Λ_B are positive definite. If no such grounding is present and a whole connected component is retained, constants on that component can remain in the kernel; positivity then holds on the quotient by that kernel.

Proof. For $x \in \mathbb{R}^S$, extension by zero outside S gives

$$x^T K x = \sum_{\substack{i < j \\ i, j \in S}} c_{ij} (x_i - x_j)^2 + \sum_{\substack{i \in S \\ j \notin S}} c_{ij} x_i^2 \geq 0. \quad (9.3)$$

Thus $K \succeq 0$. A nonsingular symmetric principal block of a positive-semidefinite matrix is positive definite, so the accepted exact interior block satisfies $K_{II} \succ 0$. For any boundary vector y , minimize the quadratic form of K over interior vectors. The unique minimizer is $z = -K_{II}^{-1}K_{IB}y$, and the minimum equals $y^T \Lambda_B y$. It is therefore non-negative. Equation (9.3) vanishes only when x is constant on each selected conductance component and zero on every component touching the omitted complement. The stated grounding condition forces $x = 0$, hence $K \succ 0$; the Schur complement of a positive-definite block matrix is positive definite. $\square$

Remark 9.4 (Directed limitation). The theorem applies only to the symmetrized conductance/Laplacian route. A directed response matrix need not be symmetric or positive semidefinite; only its symmetric part can be subjected to a separate support test. The control file evaluates 36 open-grounded symmetric responses over both schedules, all three modes, two sibling regions, and the states live, record, and sym. Every response is symmetric and positive definite in that sample; the smallest observed eigenvalue is approximately 1.2315×10^{-1} . The proof, rather than this sample, supplies the positive-semidefinite statement under the theorem's hypotheses.

9.3 Exact finite GNS quotient

The numerical routine in Section 8.10 is an audit of an exact finite-dimensional construction. The following real finite-matrix specialization is proved directly and uses the standard GNS quotient principle [20, 25]; it is independent of the tolerance-resolved eigensolver.

Theorem 9.5 (Finite real GNS construction). *Let $\mathcal{A} \subseteq M_n(\mathbb{R})$ be a finite-dimensional unital algebra closed under transpose, and let $D \succeq 0$ with $\text{tr } D = 1$. Define*

$$\varphi_D(X) = \text{tr}(DX), \quad \langle X, Y \rangle_D = \varphi_D(X^T Y),$$

and

$$\mathcal{N}_D = \{X \in \mathcal{A} : \varphi_D(X^T X) = 0\}. \quad (9.4)$$

Then $\mathcal{N}_D$ is a left ideal, the quotient

$$\mathcal{H}_D = \mathcal{A}/\mathcal{N}_D$$

is a finite-dimensional real inner-product space, and

$$\pi_D(A)[X] = [AX]$$

defines a transpose representation satisfying $\pi_D(A)^* = \pi_D(A^T)$. The vector $\Omega = [I]$ is cyclic because $\pi_D(A)\Omega = [A]$ for every $A \in \mathcal{A}$. For any basis of $\mathcal{A}$, the Gram matrix in Equation (8.22) satisfies

$$\text{rank } G = \dim \mathcal{H}_D, \quad \text{nullity } G = \dim \mathcal{N}_D. \quad (9.5)$$

Proof. Since

$$\varphi_D(X^T X) = \text{tr}(D^{1/2} X^T X D^{1/2}) = \|X D^{1/2}\|_F^2,$$

Equation (9.4) is equivalent to $X D^{1/2} = 0$. Therefore $(AX) D^{1/2} = A(X D^{1/2}) = 0$, so $\mathcal{N}_D$ is a left ideal. The sesquilinear issues of a complex space do not occur here: the real bilinear form descends to a positive-definite inner product on the quotient. Left multiplication is well-defined by the left-ideal property, and

$$\langle AX, Y \rangle_D = \text{tr}(DX^T A^T Y) = \langle X, A^T Y \rangle_D.$$

Cyclicity follows immediately from $\pi_D(A)[I] = [A]$. The Gram matrix is the matrix of the positive semidefinite form on the chosen basis, so its kernel is the coefficient representation of $\mathcal{N}_D$ and rank-nullity gives Equation (9.5). $\square$

Remark 9.6 (What the code still has to check). The exact theorem assumes a genuinely unital transpose-closed algebra. The generated basis is numerical and may be stopped by an iteration cap. Therefore the implementation separately records saturation, identity-span, multiplication-closure, transpose-adjoint, null-left-ideal, action-on- Ω , and cyclic-rank residuals. The four explicit controls include cyclic and non-cyclic ambient-support cases; all pass. No completion, irreducibility, factor property, or infinite GNS statement follows.

9.4 Orientation reversal and the unresolved absolute sign

Theorem 9.7 (Sign-reversal covariance of the finite polar object). *Let $S \succeq 0$, let $A^T = -A$, and form the support-restricted object*

$$J_{\text{raw}}(S, A) = S^{-1/2} A S^{-1/2}.$$

Let $U(S, A)$ denote its polar partial isometry on the non-zero skew support. Then

$$J_{\text{raw}}(S, -A) = -J_{\text{raw}}(S, A), \quad U(S, -A) = -U(S, A). \quad (9.6)$$

The singular values, skew rank, support projector, and square relation $U^2 = -P$ are unchanged by this reversal. Consequently, any selection rule using only S and these sign-even data cannot distinguish the two globally reversed choices.

Proof. The first identity is linearity in A . The absolute value of an operator is unchanged by multiplication by -1 , so in the polar decomposition $J_{\text{raw}} = U|J_{\text{raw}}|$, replacing J_{raw} by its negative replaces U by $-U$ on the same support. All listed diagnostics are functions of $|J_{\text{raw}}|$, the support projector, or U^2 , and are therefore invariant. $\square$

Corollary 9.8 (Relative orientation versus absolute orientation). *Once the sibling-slot $F2$ coordinate is fixed, the finite rows can compare local and global signs relative to that coordinate. Under a simultaneous reversal of every such sign, each relative product is unchanged. The present core therefore supplies relative orientation data but no sign-odd predicate that declares one simultaneous global choice to be the absolute physical sign.*

Remark 9.9 (Conditional nature of the obstruction). The obstruction does not say that the computed J is undefined for a given oriented A ; Equation (9.6) determines it. It says that a model specification invariant under simultaneous orientation reversal cannot turn the two reversed specifications into inequivalent absolute choices without adding or deriving another sign-odd datum. The control error for $U(S, -A) + U(S, A)$ is 3.14×10^{-16} ; singular values and square residuals agree within the declared tolerance.

9.5 Separation of backreaction observables

Proposition 9.10 (Backreaction non-implications). *For skew matrices A_{live} and A_{record} , define*

$$B = A_{\text{live}} - A_{\text{record}}, \quad b = \|B\|_F, \quad \beta = \|A_{\text{live}}\|_F - \|A_{\text{record}}\|_F,$$

and $\omega = F_1^T B F_2$ for fixed vectors F_1, F_2 . Then

$$B = 0 \implies b = 0, \beta = 0, \omega = 0,$$

but neither $\beta = 0 \Rightarrow B = 0$ nor $\beta = 0 \Rightarrow \omega = 0$ is valid in general.

Proof. The forward implications are immediate. For the converse, take

$$J_2 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad A_{\text{live}} = J_2, \quad A_{\text{record}} = -J_2.$$

Then $\beta = 0$, whereas $B = 2J_2$, $\|B\|_F = 2\sqrt{2}$, and with $F_1 = e_1, F_2 = e_2, \omega = -2$. Thus equality of the two skew norms does not identify the matrices or an oriented coefficient. $\square$

Corollary 9.11 (Meaning of the generated crossing). *A sign change of the sampled sequence β_L states only that the order of the two skew norms changes between the sampled levels. It does not imply an intermediate zero of a matrix-valued path B_L , because no continuous matrix interpolation is defined, and it does not imply a zero of ω_L . In the documented $b = 4, L = 4, 5, 6$ rows, β_L changes sign while $\|B_L\|_F > 0$ at every sample and $\omega_L < 0$ throughout.*

9.6 Exact prefix handoff

Theorem 9.12 (Address and carrier handoff). *For $0 \leq L \leq M \leq N$, address truncation satisfies*

$$\pi_L \circ \pi_M = \pi_L. \quad (9.7)$$

For every non-negative probability distribution μ_N on level- N addresses,

$$(\pi_L)_*((\pi_M)_*\mu_N) = (\pi_L)_*\mu_N. \quad (9.8)$$

For the uniform terminal carrier Ω_N of the full b -ary tree,

$$(\pi_L)_*\Omega_N = \Omega_L. \quad (9.9)$$

Proof. Equation (9.7) is the associativity of word truncation. Push-forward composition follows by summing over the same address fibers; normalization is inactive because a non-negative probability distribution retains total mass one under each raw push-forward. For Equation (9.9), every length- L word has exactly b^{N-L} length- N extensions, so all target fibers have equal mass. $\square$

Proposition 9.13 (Conditional composition of boundary transport matrices). *Let $R_{L \leftarrow M}$ be a boundary address-prefix matrix. If every source boundary address maps to the intermediate and target boundary sets, then the `fiber_sum` matrices compose exactly. The `fiber_average` and `fiber_sqrt` matrices also compose when the relevant fiber cardinalities factor uniformly through the intermediate boundary. Under these conditions,*

$$R_{L \leftarrow N} = R_{L \leftarrow M} R_{M \leftarrow N}. \quad (9.10)$$

Proof. Each source column has one address-determined target row. For `fiber_sum`, matrix multiplication follows the unique intermediate row and reproduces the direct unit entry. For the two normalized weightings, the composed coefficient is the product of the two inverse fiber factors; the stated factorization condition makes this product equal to the direct inverse fiber factor. $\square$

Remark 9.14 (Legacy label projection versus the new carrier map). The earlier interlevel rows use a least-squares projection between equally named generators in independently reconstructed three-role algebras. That comparison remains non-multiplicative: its maximum product-morphism residual is 1, its strict star-hom indicator is zero, its maximum GNS Gram transport error is approximately 6.82×10^{-1} , and its conditional-expectation state drift reaches approximately 1.46. Those rows are retained as a negative control for that particular label-span map. They no longer exhaust the handoff question, because the following construction introduces a genuinely nested carrier before comparing distinguished response and state data.

9.7 Growing address-contrast carriers and non-unital corner embeddings

Code map. `core/nested_multiscale.py` constructs the nested address-coordinate contrast basis, its label inclusion, the corner maps, the resolved support graph, and the common-ambient Schur comparison. `derived_quantities/nested_multiscale_handoff.py` produces the carrier, handoff, composition, state/GNS, finite- J , and common-ambient rows. The complete control is executed by

```
cd anc/CNNA_structured_core_pkg
python3 package_consistency/check_nested_multiscale_handoff.py \
    --output-dir docs
```

Definition 9.15 (First-order address-contrast carrier). *Fix one sibling-cone region and let the left and right frontier words have relative length h below their sibling roots. Let $\eta^{(1)}, \dots, \eta^{(b-1)} \in \mathbb{R}^b$ be the real Helmholtz contrasts, an orthonormal basis of the subspace orthogonal to the constant slot vector. The carrier basis contains the normalized LCA indicator, the normalized constant indicator of each sibling frontier, and, for each side $\sigma \in \{1, 2\}$, address coordinate $d = 0, \dots, h-1$, and contrast $r = 1, \dots, b-1$, the vector*

$$q_{\sigma,d,r}(a) = \begin{cases} b^{-(h-1)/2} \eta_{a_d}^{(r)}, & a \text{ belongs to frontier } \sigma, \\ 0, & \text{otherwise.} \end{cases} \quad (9.11)$$

The resulting real carrier $K_{b,h}$ has dimension

$$d_{b,h} = 3 + 2h(b-1). \quad (9.12)$$

Only first-order address-coordinate contrasts are included; products of contrasts at different address coordinates are deliberately omitted. Thus the carrier grows linearly in h , rather than exponentially with the full frontier size.

Theorem 9.16 (Exact nested carrier handoff). *Let $L < M$, let $R_{L \leftarrow M}$ be the `fiber_sqrt` boundary prefix matrix, and put $V_{M \leftarrow L} = R_{L \leftarrow M}^T$. For complete sibling-frontier fibers, let Q_L, Q_M be the bases from Definition 9.15. Then there is a label-inclusion isometry*

$$T_{M \leftarrow L} : K_L \longrightarrow K_M, \quad T_{M \leftarrow L}^T T_{M \leftarrow L} = I_{K_L}, \quad (9.13)$$

such that

$$V_{M \leftarrow L} Q_L = Q_M T_{M \leftarrow L}. \quad (9.14)$$

Consequently the map

$$\iota_{M \leftarrow L}(X) = T_{M \leftarrow L} X T_{M \leftarrow L}^T, \quad X \in M_{d_L}(\mathbb{R}), \quad (9.15)$$

is an injective transpose-preserving algebra homomorphism. Its compression

$$E_{L \leftarrow M}(Y) = T_{M \leftarrow L}^T Y T_{M \leftarrow L} \quad (9.16)$$

satisfies $E_{L \leftarrow M} \circ \iota_{M \leftarrow L} = \text{id}$. If $d_M > d_L$, the embedding is non-unital in the full target matrix algebra:

$$\iota_{M \leftarrow L}(I_{d_L}) = P_{M,L} := T_{M \leftarrow L} T_{M \leftarrow L}^T \neq I_{d_M}. \quad (9.17)$$

For $L < M < N$, the inclusions and corner projectors compose through the intermediate level.

Proof. Each coarse frontier word has exactly b^{M-L} fine extensions. The `fiber_sqrt` coefficient is the inverse square root of this fiber size. It therefore sends a normalized constant block to the corresponding normalized constant block. A contrast in Equation (9.11) depends only on one address coordinate already present at level L ; every fine extension has the same value on that coordinate, and the square-root fiber normalization again gives the level- M vector with the same label. New level- M columns depend on later address coordinates and are orthogonal to the lifted old columns. This proves Equations (9.13) and (9.14). Multiplicativity follows from

$$(TXT^T)(TYT^T) = TX(T^TT)YT^T = TXYT^T,$$

transpose preservation is immediate, and Equation (9.16) follows from $T^TT = I$. Label inclusions compose, hence so do the carrier maps. $\square$

Proposition 9.17 (Resolved response generation of the carrier algebra). *Let $\Lambda_L^{\text{ms}} = Q_L^T \Lambda_L Q_L$, let $P_i = e_i e_i^T$ be all coordinate projections of K_L , and define*

$$\mathcal{A}_L^{\text{ms}} = \text{alg}^T \left(\{P_i\}_{i=1}^{d_L} \cup \{\Lambda_L^{\text{ms}}\} \right). \quad (9.18)$$

If the undirected support graph with an edge $i \sim j$ whenever $(\Lambda_L^{\text{ms}})_{ij} \neq 0$ or $(\Lambda_L^{\text{ms}})_{ji} \neq 0$ is connected, then

$$\mathcal{A}_L^{\text{ms}} = M_{d_L}(\mathbb{R}). \quad (9.19)$$

Proof. For a supported directed entry, $P_i \Lambda_L^{\text{ms}} P_j = (\Lambda_L^{\text{ms}})_{ij} E_{ij}$. Transpose closure supplies the reverse matrix unit. Multiplying matrix units along a path in the connected support graph produces E_{ij} for every ordered pair. The matrix units span $M_{d_L}(\mathbb{R})$. $\square$

Remark 9.18 (Numerical support qualification). Equation (9.19) is an exact conditional statement. The implementation checks its premise on the support resolved at threshold 10^{-12} ; it does not convert a floating-point entry above threshold into a symbolic non-vanishing theorem. All nine tested live rows for $b = 2, 3, 4$ and $L = 4, 5, 6$ have one connected symmetric and directed support component. Their carrier dimensions are

$$7, 9, 11; \quad 11, 15, 19; \quad 15, 21, 27,$$

respectively, and the corresponding full carrier-algebra dimensions reach 121, 361, and 729 at level 6.

Proposition 9.19 (Conditional corner state and scaled GNS identity). *Let D_M be a normalized positive density on K_M , let $T = T_{M \leftarrow L}$, and define*

$$C_{M \downarrow L} = T^T D_M T, \quad \lambda_{M,L} = \text{tr}(C_{M \downarrow L}), \quad D_{M|L} = \frac{C_{M \downarrow L}}{\lambda_{M,L}} \quad (9.20)$$

whenever $\lambda_{M,L} > 0$. Then for all $X, Y \in M_{d_L}(\mathbb{R})$,

$$\langle \iota_{M \leftarrow L}(X), \iota_{M \leftarrow L}(Y) \rangle_{D_M} = \lambda_{M,L} \langle X, Y \rangle_{D_{M|L}}. \quad (9.21)$$

Thus $\lambda_{M,L}^{-1/2} \iota_{M \leftarrow L}$ induces an isometry between the conditional finite GNS pre-Hilbert spaces. It induces an isometry from the independently reconstructed level- L state only if $D_L = D_{M|L}$.

Proof. Using $T^TT = I$ and cyclicity of the finite trace,

$$\text{tr}(D_M(TXT^T)^T(TYT^T)) = \text{tr}(T^T D_M T X^T Y) = \lambda_{M,L} \text{tr}(D_{M|L} X^T Y).$$

$\square$

Theorem 9.20 (Common-ambient Schur positive control). *Fix one fine finite model, one open-grounded symmetric local matrix K_M , and a coarse retained boundary B_L . Let C be an intermediate retained set containing B_L . Whenever the required interior blocks are invertible,*

$$\text{Schur}_{B_L}(K_M) = \text{Schur}_{B_L}(\text{Schur}_C(K_M)). \quad (9.22)$$

This identity compares two elimination orders inside one ambient matrix. It does not imply equality with the response reconstructed independently from the level- L growth object.

Proof. Both sides are the block Gaussian elimination of the same coordinates of K_M . Eliminating the complement of C and then $C \setminus B_L$ gives the same Schur complement as eliminating their union in one step. $\square$

Table 22: Nested multiscale handoff readout. Exact columns report maxima over the generated rows; drift columns report observed ranges and are not acceptance tolerances.

Quantity	Readout	Status
boundary coisometry error	1.64×10^{-15}	exact finite control
role-basis intertwining error	4.59×10^{-16}	exact finite control
label-inclusion error	2.11×10^{-15}	exact finite control
corner product error	9.44×10^{-16}	exact finite control
corner left-inverse error	2.10×10^{-15}	exact finite control
carrier composition error	7.50×10^{-16}	exact finite control
common-ambient direct/staged Schur error	1.26×10^{-15}	exact finite control
conditional scaled GNS error	5.56×10^{-16}	exact finite control
independent directed-response drift	0.270–0.393	distinguished object not natural
independent density drift	0.367–0.866	independently reconstructed states differ
finite- J intertwining error	0.811–1.467	no J -natural family
finite- J leakage into new roles	0.349–0.759	old carrier not J_M -invariant

Corollary 9.21 (Record invariance and live backreaction in the common ambient). *For the tested common-ambient cases $(b, L, M) = (2, 4, 6), (3, 4, 6), (4, 4, 5)$, direct and staged elimination agree to at most 1.26×10^{-15} . The independently reconstructed record response agrees with the nested record response to at most 6.67×10^{-16} in the full boundary matrix and 4.49×10^{-16} after multiscale compression. The corresponding live compressed drift is non-zero, ranging from 0.272 to 0.657. These rows isolate immutable record extension from later live backreaction for the stated finite cases; they are not a general asymptotic theorem.*

Remark 9.22 (What the last arrow now contains). The growing carrier algebras $M_{d_L}(\mathbb{R})$ and their non-unital corner embeddings are now an exact finite directed system, and the resolved support test shows that the response together with all canonical coordinate projections generates the full carrier algebra in every sampled row. What remains absent is a natural family of the distinguished objects. The independently reconstructed responses and densities do not satisfy the corner restriction identities, the independently reconstructed GNS metrics therefore drift, and the finite- J operators neither intertwine nor preserve the old carrier subspaces. Consequently the package now separates an exact algebraic carrier handoff from the still-failed response/state/GNS/ J handoff. A norm completion or physical interpretation of the resulting growing carrier system lies outside the finite scope of this article.

Why the finite restriction is mathematically informative. Finiteness is used here as an audit condition rather than merely as a computational cutoff. It allows the provenance of every algebraic object to be traced back to explicit growth and response data, and it prevents unresolved limit operations from concealing failures of positivity, cyclicity, orientation, multiplicativity, or state compatibility. A property that fails in the finite core is not repaired merely by naming a continuum or operator-algebraic limit; a restoration would require an additional construction and a separate proof. Conversely, a property proved at every inspected finite level does not automatically survive an infinite limit. The finite results therefore serve both as constructive building blocks and as necessary obstruction tests for any later extension.

9.8 Control-row summary

Table 23: Generated finite structural control rows.

Control	Pass	Recorded readout
finite growth replay	1	96 of 96 exact paired replays; zero address, parent, or non-finite-value failures.
symmetric Schur positivity	1	36 accepted rows; zero symmetry or PSD failures; all 36 sampled open-grounded responses are positive definite.
finite GNS quotient/cyclicity	1	four of four controls pass, including two positive and two negative ambient-support cyclicity cases.
finite J reversal	1	sign-flip error 3.14×10^{-16} ; unchanged skew rank and sign-even diagnostics.
backreaction separation	1	explicit 2×2 control has $\beta = 0$, $\ B\ _F = 2\sqrt{2}$, and $\omega = -2$.
prefix handoff	1	zero prefix/push-forward/carrier failures; boundary composition errors zero, zero, and 3.24×10^{-16} for the three weightings.
nested multiscale handoff	1	exact growing carrier, corner, composition, conditional GNS, and common-ambient Schur controls; non-zero independent response/state/finite- J drift retained.

10 Derived quantities as finite row families

10.1 Baseline row families

Code map. `derived_quantities/baseline.py` provides the first rows computed from core objects: node/event genesis rows, projection grammar rows, carrier-state rows, orientation rows, and region/GNS rows.

Proposition 10.1 (Baseline rows are evaluations of core constructors). *Every baseline row can be traced to one of: `EmptySetGrowthModel`, prefix push-forward, terminal carrier/state extraction, orientation measurement, regional Schur response, or finite GNS construction.*

Proof. **Object.** The object is the row family returned by the five public baseline functions. **Step 1.** `level_models` enumerates finite approximants. **Step 2.** `genesis_rows` reads node and event counts from the growth object. **Step 3.** `projection_grammar_rows` applies prefix push-forwards. **Step 4.** `carrier_state_rows` evaluates terminal distributions. **Step 5.** `orientation_rows` and `region_and_gns_rows` call signed-current, Schur, and algebra/GNS constructors. **Counter-check.** These rows add no independent object; mismatches identify a constructor or mapping defect. **Conclusion.** Baseline is the first complete finite evaluation layer. $\square$

10.2 Rank, kernel, coverage, and compressed projectivity

Code map. `rank_kernel_conditions.py` measures local response-density rank/kernel behavior and construction coverage. `compressed_state_projection_consistency.py` measures whether compressed carrier states remain projective under target-level truncation.

Proposition 10.2 (Compressed projectivity check). *For target levels $k < \ell$, the projectivity rows compare a direct compression at level k against the push-forward of a compression at level ℓ .*

Proof. **Object.** The object is a pair of compressed distributions over role/address keys. **Step 1.** Build the compressed state at the finer target level. **Step 2.** Push it forward by address truncation. **Step 3.** Build the coarser compressed state directly. **Step 4.** Compare them by total variation and sector profiles. **Counter-check.** A non-zero discrepancy can arise from role/coarse-key mismatch and is recorded as a finite discrepancy, not as a continuum anomaly. **Conclusion.** The check decides finite compatibility of the implemented compression map. $\square$

10.3 Structural live-record ledgers

Code map. `structural_ledgers.py` records parameter purity, live-record structural calculations, history deltas, carrier effects, and state-sector decompositions.

Proposition 10.3 (Live-record history delta uses one generated approximant). *The history-delta rows are computed from live and record states generated by the same b, L, m approximant; no additional state is fitted.*

Proof. **Object.** The object is the paired row family over live and record. **Step 1.** Construct one growth model. **Step 2.** Read both edge states from that model. **Step 3.** Compute row differences and carrier effects. **Counter-check.** If a row depends on an external parameter not present in the model arguments, it is outside this construction route. **Conclusion.** The ledger is a bookkeeping layer over core state separation. $\square$

10.4 Interlevel operator maps and interface rows

Code map. `operator_stack_residuals.py` builds interlevel rows for GNS generator maps, algebra projection maps, conditional-expectation proxies, finite log-density preflow rows, and composition rows. `operator_stack_interface_residuals.py` builds seam, state-comparison, scaling, and map-naturality rows. `operator_stack_consistency_checks.py` builds finite condition rows for role dimensions, generator ranks, coordinate changes, boundary-response model choice, and skew survival.

Definition 10.4 (Interlevel residual). *Let X_M be a finite object at source level M and let X_L be the corresponding object at target level $L < M$. When both objects are represented on a common finite label set, the row family records residuals of the form*

$$\rho_{M \rightarrow L}(X) = \frac{\|\Pi_{M \rightarrow L}(X_M) - X_L\|_F}{\max(\|X_L\|_F, \varepsilon)}. \quad (10.1)$$

Here $\Pi_{M \rightarrow L}$ is the label-span projection or prefix-induced map implemented by the row function that produces the row.

Proposition 10.5 (Interlevel rows are finite discrepancies). *The interlevel rows record finite discrepancies between constructed objects at two levels. They do not construct an infinite-level object.*

Proof. **Object.** The source objects are the finite response-algebra, GNS, conditional-expectation, and log-density rows at two levels. **Step 1.** Construct the level- M and level- L rows from the same b , mode, state, region rule, and growth schedule. **Step 2.** Express the two rows in shared labels when such labels are present. **Step 3.** Evaluate Frobenius, relative, or scalar residuals. **Counter-check.** A non-zero row is retained as a finite discrepancy for that map and label route. **Conclusion.** The row family is a finite comparison layer over already constructed matrix objects. $\square$

Regenerated row command. The row values in this subsection are generated by the following command:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_cnna.py \
  --branching 4 \
  --levels 4 5 \
  --modes linear \
  --states live record sym \
  --max-regions 2 \
  --max-boundary 700 \
  --max-interior 1000 \
  --directed-frontier-cap 6 \
  --algebra-tol 1e-10 \
  --algebra-max-iter 8 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/transport_j_relation_reduced
```

Table 24: Interlevel residual readout from the regenerated row surface.

Row family / quantity	Value	Finite object readout
<code>gns_generator_transport_maps.csv</code> rows	3	live, record, and sym rows for the level-5 $\rightarrow$ 4 map.
max GNS Gram relative error	6.8185×10^{-1}	label-span transport drift in the generated GNS rows.
max cyclic-vector error	7.6898×10^{-2}	cyclic-vector drift in the same rows.
<code>algebra_projection_morphism_measurement.csv</code> rows	3	algebra projection rows for live, record, and sym.
max star-compatibility error	1.4125×10^{-16}	finite star compatibility residual.
max product-morphism residual	1	product route residual in the finite algebra projection rows.
max center-preservation error	2.2197×10^{-1}	center preservation drift in the same rows.
max CE commuting-square error	3.8206×10^{-3}	conditional-expectation square residual.
max CE state-preservation drift	1.4635	state drift in the finite conditional-expectation proxy.
max K -intertwining relative error	2.8902×10^{-1}	finite log-density preflow transport error.
max finite-preflow delta square error	3.6978×10^{-1}	commutator-delta square residual.
<code>operator_stack_residuals_classification.csv</code> rows	3	live, record, and sym classification rows.
strict star-hom indicator	0	generated indicator field for the strict star-hom residual check.

10.5 Directed transport rows

Code map. `directed_current_transport.py` builds the boundary-prefix transport matrix, transport rows, composition rows, orientation summary rows, and finite coverage rows. The transport matrix sends source boundary indicators to target prefix classes.

Proposition 10.6 (Directed transport row). *For levels $M > L$, each directed transport row compares the direct level- L directed response with the transported level- M directed response after applying the finite address-prefix transport matrix.*

Proof. Object. The objects are two directed response matrices and the boundary-prefix transport matrix. **Step 1.** Build source and target boundary lists. **Step 2.** Build the finite transport matrix from address prefixes. **Step 3.** Push the source response to target coordinates. **Step 4.** Record direct-vs-transported discrepancies and scale-fit quantities. **Counter-check.** Boundary-role mismatch and open-grounded degrees can produce non-zero rows. **Conclusion.** The rows are finite transport comparisons between two constructed directed responses. $\square$

Table 25: Directed transport readout from the regenerated row surface.

Row family / quantity	Value	Finite object readout
directed_current_transport.csv rows	9	directed transport rows for live, record, and sym in the integrated command.
J_orientation_transport_summary.csv rows	1	summary row over the directed transport surface.
directed transport coverage field	1	generated coverage indicator in the summary row.
unavailable directed transport pairs	0	generated unavailable-pair count in the summary row.
live J/A orientation transport field	1	generated residual-threshold indicator in the summary row.
A-transport composition field	0	generated composition indicator in the summary row.

10.6 A/J orientation transport and composition

Code map. `j_orientation_projectivity.py` builds A/J orientation snapshots, transport rows, composition rows, and a summary row. These rows use the skew part A on the positive support selected by the symmetric part S .

Definition 10.7 (A/J orientation row). *An A/J row contains a finite skew-plane object, its support data, and its transported or composed counterpart. The scalar errors compare the orientation after transport or composition with the orientation obtained by direct construction at the target level.*

Regenerated row command. The row values in this subsection are generated by the following command:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_j_orientation_projectivity.py \
  --branching 4 \
  --levels 4 5 6 \
  --modes linear \
  --max-regions 3 \
  --max-boundary 1200 \
  --max-interior 1800 \
  --directed-frontier-cap 6 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/j_orientation_projectivity
```

Table 26: A/J orientation transport and composition readout.

Row family / quantity	Value	Finite object readout
A_J_orientation_snapshot.csv rows	18	snapshots for levels 4, 5, 6 and states live, record.
max snapshot $J^2 + P$ error	1.5264×10^{-13}	support-level finite square residual.
A_J_orientation_transport.csv rows	54	transport rows between the represented levels.
max scale-optimal A-orientation error	6.1953×10^{-14}	scalar orientation transport residual after scale fit.
A_J_orientation_composition.csv rows	18	direct-vs-composed transport rows.
max direct-vs-composed A-plane residual	3.3697×10^{-14}	finite composition residual for the A-plane.
max direct scale-optimal A-orientation error	2.9088×10^{-14}	direct branch orientation residual.
max via scale-optimal A-orientation error	2.0043×10^{-14}	composed branch orientation residual.
summary raw A relative transport error	9.4665×10^{-1}	raw matrix relative error before scale-optimal orientation comparison.
summary scale-optimal A-orientation error	1.3444×10^{-15}	summary field for the fitted orientation scalar.

10.7 Local/global J sign relation

Code map. `local_global_j_sign_relation.py` builds global boundary sign rows and compares them with local region sign rows. The rows compare relative signs only; the simultaneous global flip remains represented by a separate field.

Proposition 10.8 (Local/global sign rows compare relative signs). *The local/global sign rows compare a local sign with a global boundary sign on the same finite construction route. The rows retain the simultaneous sign-flip field and do not select an absolute sign.*

Proof. Object. The objects are local finite J comparison rows and global boundary sign rows.

Step 1. Construct global boundary sign data. **Step 2.** Construct local region sign data. **Step 3.** Compare the local sign to the global sign. **Step 4.** Evaluate the simultaneous flip field. **Counter-check.** A zero flip-invariance error records that the comparison is relative to a simultaneous sign flip. **Conclusion.** The row family represents finite relative sign data. $\square$

Regenerated row command. The row values in this subsection are generated by the following command:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_local_global_j_sign_relation.py \
  --branching 4 \
  --levels 4 5 \
  --modes linear \
  --max-regions 2 \
  --max-boundary 700 \
  --max-interior 1000 \
  --directed-frontier-cap 6 \
  --gns-tol 1e-10 \
  --growth-schedule layer_batch \
  --outdir outputs/local_global_j_sign_relation
```

Table 27: Local/global J sign readout from regenerated rows.

Row family / quantity	Value	Finite object readout
j_sign_global_boundary_sign_relation.csv rows	16	global boundary sign rows for levels 4, 5 and states live, record.
max global $J^2 + P$ error	5.5556×10^{-15}	finite global sign square residual.
max global solve residual	2.5045×10^{-16}	Schur solve residual in the global sign rows.
max global row-sum residual	4.6731	row-sum residual for the selected global boundary construction.
local_global_J_sign_comparison.csv rows	8	local/global comparison rows over the selected regions.
local/global relative sign value	-1	generated relative sign value in all represented rows.
max local $J^2 + P$ error	3.8934×10^{-14}	local finite sign square residual.
max global $J^2 + P$ error in comparison	5.5556×10^{-15}	global square residual in comparison rows.
max flip-invariance error	0	simultaneous global flip field in the comparison rows.
local regions compared	8	count in J_sign_sign_relation_summary.csv.
absolute global sign fixed	0	generated field in the sign-relation summary row.

10.8 Live-record backreaction current

Code map. `backreaction_current.py` constructs regional rows for the live-record current defined in Equation (10.3), where $A_{\text{live},L}$ and $A_{\text{record},L}$ are the skew parts of directed Schur responses built on the same branching value, level, mode, region, boundary response model, and matrix sign relation. The same module writes growth-profile, region-sensitivity, transport, and summary rows.

Definition 10.9 (Live-record backreaction current). *For a fixed finite region R and level L , let*

$$A_{\text{live},L}^R = \frac{M_{\text{live},L}^R - (M_{\text{live},L}^R)^T}{2}, \quad (10.2)$$

$$A_{\text{record},L}^R = \frac{M_{\text{record},L}^R - (M_{\text{record},L}^R)^T}{2}.$$

The live-record backreaction current is the finite skew matrix

$$B_L^R = A_{\text{live},L}^R - A_{\text{record},L}^R. \quad (10.3)$$

Three distinct scalar readouts are used and must not be identified:

$$b_L^R = \|B_L^R\|_F, \quad \beta_L^R = \|A_{\text{live},L}^R\|_F - \|A_{\text{record},L}^R\|_F, \quad \omega_L^R = \langle F_1, B_L^R F_2 \rangle. \quad (10.4)$$

Here $b_L^R \geq 0$ is the norm of the difference matrix, β_L^R is a signed difference of two norms, and ω_L^R is the oriented scalar of the compressed F_1/F_2 plane. A zero crossing of β_L^R is neither a zero of B_L^R nor a sign change of ω_L^R .

Proposition 10.10 (Backreaction rows compare live skew response with birth record). *The current B_L^R in Equation (10.3) is computed by subtracting the record skew response from the live skew response of Equation (10.2) on the same finite row coordinates.*

Proof. Object. The inputs are two directed Schur responses with identical finite model, region, boundary, and sign-relation fields. **Step 1.** Build the live directed response and extract its skew part. **Step 2.** Build the record directed response and extract its skew part. **Step 3.** Subtract the two skew matrices. **Step 4.** Evaluate the three separate readouts in Equation (10.4). **Recorded branch.** The sign of β_L^R compares skew magnitudes; the sign of ω_L^R concerns the selected oriented plane. Neither is a matrix-order statement, and b_L^R is unsigned. **Conclusion.** The row represents a finite matrix difference between current network response and birth-record response together with explicitly separated scalar diagnostics. $\square$

Regenerated row command. The following command generates the live-record backreaction surface over three terminal levels for the displayed row values:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_live_record_backreaction_current.py \
  --branching 4 \
  --levels 4 5 6 \
  --modes linear \
  --max-regions 1 \
  --max-boundary 600 \
  --max-interior 900 \
  --directed-frontier-cap 6 \
  --growth-schedule layer_batch \
  --outdir outputs/live_record_backreaction_linear_L456_layer_batch
```

Table 28: Live-record backreaction current rows generated by the stated command.

Level	$b_L = \ B_L\ _F$	β_L	ω_L	Sign fields
4	8.7395×10^{-2}	-7.3972×10^{-2}	-3.5×10^{-5}	full-skew sign -1 , plane sign -1
5	1.4179×10^{-1}	-1.5959×10^{-2}	-6.0×10^{-5}	full-skew sign -1 , plane sign -1
6	2.4168×10^{-1}	1.3825×10^{-1}	-9.4×10^{-5}	full-skew sign 1 , plane sign -1

Table 29: Backreaction transport rows from the same generated surface.

Quantity	Value	Finite object readout
transport rows	9	three address-prefix transport routes for the level pairs in the generated surface.
available rows	9	all rows contain a constructed transported current.
raw relative error range	2.0361×10^{-1} to 1.1657×10^1	amplitude-sensitive discrepancy between transported and target current.
scale-optimal orientation error range	3.4987×10^{-14} to 5.7964×10^{-13}	residual after scalar fit of the orientation component.
orientation sign agreement	9 of 9	generated sign field comparing transported and target orientation.
fiber-average raw error range	2.0361×10^{-1} to 3.5119×10^{-1}	row subset for the fiber-average transport route.

10.9 Birth-relative backreaction rows

Code map. `backreaction_current.py` now writes the sibling-root birth level explicitly from `RegionSpec.left_root` and verifies equality with the right sibling-root level. `backreaction_birth_relative.py` reads that field, attaches a birth-relative age, and computes zero-crossing rows from the signed norm-difference sequence β_L^R . The age used for the law rows is

$$\alpha = L_{\text{frontier}} - L_{\text{birth}}. \quad (10.5)$$

Rows using coarse-frontier coverage are kept out of the law fit by the row fields.

Proposition 10.11 (Birth-relative crossing row). *For one branching value, mode, and region index, the zero-crossing row is obtained from a finite list of signed scalar backreaction rows ordered by birth-relative age.*

Proof. Object. The input is the finite row list containing B_L^R and its scalar readouts for one region family. **Step 1.** Read the explicit sibling-root birth level and attach the age in Equation (10.5). **Step 2.** Sort the eligible full-terminal rows by α . **Step 3.** Read $\beta_L^R = \text{live_minus_record_skew_norm}$. **Step 4.** If two adjacent ages carry opposite signs, record the first sign change and linearly interpolate the two sampled β -values. **Recorded branch.** If the sampled sequence contains no opposite signs, the crossing fields remain empty. **Conclusion.** The row is a finite interpolation of the skew-norm difference β_L^R ; it is not a zero of the matrix current B_L^R , whose Frobenius norm remains non-negative, and it is not a crossing of ω_L^R . $\square$

Regenerated row command. The following command generates the birth-relative rows displayed below:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_birth_relative_backreaction_current.py \
  --branching 4 \
  --levels 4 5 6 \
  --modes linear \
  --max-regions 1 \
  --max-boundary 600 \
  --max-interior 900 \
  --directed-frontier-cap 6 \
  --growth-schedule layer_batch \
  --outdir outputs/birth_relative_backreaction_linear_L456_layer_batch
```

Table 30: Birth-relative backreaction row sequence generated by the stated command.

Level	Age α	$\ A_{\text{live}}\ - \ A_{\text{record}}\ $	$\ B_L\ $	Scalar signs
4	2	-7.3972×10^{-2}	8.7395×10^{-2}	skew sign -1 , plane sign -1
5	3	-1.5959×10^{-2}	1.4179×10^{-1}	skew sign -1 , plane sign -1
6	4	1.3825×10^{-1}	2.4168×10^{-1}	skew sign 1 , plane sign -1

Birth-relative crossing fields. The file `birth\_relative\_zero\_crossing\_by\_region.csv` is generated by the stated command. The interpolated crossing concerns β_L only. In the displayed sequence $b_L > 0$ at every sampled level and $\omega_L < 0$ throughout, so neither the matrix current nor the selected oriented component crosses zero. A schedule-stability value for the interpolated β -crossing is not stated here.

10.10 Branching comparison and row aggregation

Code map. `backreaction_branching_shift.py` reads birth-relative mode/branching rows and writes curve and summary rows. `computed_status_rows.py` collects row families without constructing another matrix object.

Regenerated row command. The following command generates the branching comparison rows:

```
cd anc/CNNA_structured_core_pkg
PYTHONPATH=src python3 run_branching_zero_crossing_shift.py \
```



```

--branching 3 4 5 \
--levels 4 5 6 \
--modes linear \
--max-regions 1 \
--max-boundary 600 \
--max-interior 900 \
--directed-frontier-cap 6 \
--growth-schedule layer_batch \
--outdir outputs/branching_zero_crossing_linear_L456_layer_batch

```

Table 31: Backreaction row files generated by the stated commands.

Row family	Rows	Generated finite fields
live_record_backreaction_current.csv	3	live/record skew-response current fields at the sampled levels.
backreaction_transport.csv	9	address-prefix transport fields for the same current rows.
birth_relative_relative_ backreaction_current.csv	3	age-labelled current rows.
birth_relative_zero_crossing_by_region.csv	1	interpolation fields generated from the sampled scalar sequence; no schedule-stability value is stated in this document.
branching_zero_crossing_ branching_shift_curve.csv	3	three branching-indexed rows are generated; two contain finite interpolated crossings, while the $b = 5$ row records no finite crossing under the stated levels and caps.
branching_zero_crossing_ branching_shift_summary.csv	1	row-family summary fields generated from the branching-indexed rows.

11 Conclusion

The finite core developed here provides an explicitly checkable base layer for CNNA. It begins from an empty finite configuration and introduces no external geometric background, continuum space, spacetime, field variables, integrals, or differential operators. The rooted b -ary tree is used as a finite provenance carrier, not as a discretized physical geometry; a Sierpiński-type realization is not part of the finite derivation. Its subsequent objects are generated by a declared deterministic rule family. The branching index b , depth cutoff L , mode selector m , update schedule $\mathfrak{s}$, readout selectors, caps, and numerical tolerances are separated rather than collapsed into a claim of derivation from the empty configuration alone. This makes the model transparent in the finite sense: growth histories, edge weights, carrier push-forwards, response maps, algebraic closures, J -comparisons, local operator diagnostics, and backreaction rows can all be traced to finite combinatorial and matrix operations with explicit inputs.

The main value of the finite formulation is therefore not that it is already the most general or final form of the theory. Its value is that it displays the generative skeleton before further structures are placed on top of it. In a more developed formulation, one expects additional layers: larger region systems, deeper histories, refined operator surfaces, transport structures, limiting procedures, and eventually infinite or completed objects. Those layers may become mathematically more expressive, but they also make it harder to see which parts are genuinely derived and which parts have been added by representation. The present finite core is designed to keep that distinction visible.

The finite source now separates exact results from numerical observations. For fixed admissible inputs the growth object is well-defined; the symmetric open-grounded Schur response is positive semidefinite under its stated hypotheses; the finite real GNS quotient is a cyclic transpose representation; simultaneous orientation reversal leaves all sign-even finite- J diagnostics unchanged; the backreaction matrix and its scalar readouts obey explicit non-implications; address/carrier handoff composes exactly; and a linearly growing address-contrast carrier admits exact non-unital corner embeddings. Directed live and record surfaces still differ from the symmetrized control, skew components survive controlled decompositions, and local/operator transport diagnostics expose both satisfied and failed stronger properties. The first group consists of finite theorems. The remaining row statements are finite observations with residuals, not continuum theorems.

The discrete Dirichlet-to-Neumann construction is especially important in the finite sense used here. The DtN-type maps are Schur-complement response matrices on finite generated graphs. Boundary sets, interior sets, invertibility conditions, residuals, and compressed responses are explicit and directly inspectable. The complete finite provenance is now stated without a semantic gap: a birth adds a set-theoretic and coordinate-linear complement to the existing carrier; the generated directed weights couple that new direction to the prior network; a selected boundary/interior cut produces a directed Schur response; and only that response is decomposed into symmetric and skew parts. The complement identity by itself does not imply a skew sector. The construction is therefore a finite algebraic network response calculation, not an imported analytic boundary theory.

The conclusion is therefore conservative. The finite CNNA core demonstrates a coherent, reproducible discrete model initialized by an empty configuration and evolved by an explicit rule family. In comparisons where m , $\mathfrak{s}$, region selection, caps, and tolerances are fixed, b is the varied structural index and L the finite cutoff. The model supplies a controlled response calculus and a nontrivial hierarchy of finite diagnostics, but it does not establish an infinite theory or equivalence with established continuum frameworks. It provides a transparent base from which higher structures can be formulated without disguising fixed rules or numerical selectors as derived quantities.

12 Outlook

The next conceptual step is not to abandon the finite model, but to use it as the visible base layer for more structured extensions. The finite core should be understood as the simplest representation of the theory, not as its endpoint. Its role is comparable to a coordinate chart for the generative mechanism: it makes the construction inspectable before more complex objects, completions, or limiting procedures are introduced.

The first nontrivial carrier step has now been completed at finite level. The first-order address-contrast spaces grow with depth, intertwine exactly under the derived `fiber_sqrt` prefix map, and yield composable non-unital corner embeddings of real full matrix algebras. This closes the carrier-algebra part of the handoff without pretending that the distinguished data are already natural. The independently reconstructed responses and densities drift under the corner restriction, the corresponding independent GNS metrics are not isometric, and the finite- J objects both fail to intertwine and leak into newly added role directions. The next structural problem is therefore sharper than before: derive a live evolution or completely positive transport law for these distinguished objects, or identify a state construction compatible with the exact carrier embeddings. A norm completion and its physical interpretation remain separate from the finite results proved here.

A first direction is therefore the infinite extension. This should be treated as a higher intermediate objective, not as an automatic consequence of the finite rows. Passing from finite generated carriers and matrices to infinite or completed objects would require additional mathematical control: compatible growth embeddings, stability of carrier push-forwards, convergence or projective consistency of response maps, control of residuals under depth increase, and a precise account of which finite diagnostics survive the limit. In particular, the existence of finite J -objects, finite GNS-like structures, or finite local operator checks should not by itself be read as the existence of corresponding infinite objects. Those claims require separate construction.

A second direction is deeper comparison with the discrete models separated in Section 2. The current comparison distinguishes carrier growth from occupation growth, sequential from synchronous event order, preservation from rearrangement of prior genealogy, forward response from inverse reconstruction, and finite combinatorial networks from metric quantum graphs. Future work should sharpen those bridges by proving explicit correspondences or incompatibility results, not by relying on shared terminology. The present model still introduces no manifold, smooth function, measure, integral, differential operator, or analytic boundary value problem. Likewise, a tree-Sierpiński correspondence is not assumed: it remains a separate finite gluing or duality problem.

A third direction is structural enrichment above the finite core. The present code exposes live, record, and sym surfaces, finite boundary responses, directed decompositions, local operator diagnostics, sign transport, and backreaction currents. Higher layers could add more refined region systems, larger multi-scale carrier hierarchies, stronger consistency constraints between local and global objects, categorical or functorial descriptions of growth and projection, or completion procedures that preserve the derived-only character of the construction. The key constraint should remain that no new primitive is introduced silently. If an additional object is added, its status should be explicit: derived finite object, diagnostic row, limiting candidate, external comparison structure, or genuine new axiom.

The long-term theoretical question is whether this empty-initialized finite hierarchy can support a controlled path toward richer mathematical structures without losing its auditability. The finite core shows that substantial response, orientation, algebraic, and backreaction structure can arise after fixing an explicit deterministic rule family. Future work must determine which features are stable under changes of m , $\mathfrak{s}$, region system, and cutoff, rather than assuming those selectors are irrelevant. An infinite extension would then test whether compatible objects can be stabilized

beyond finite depth. Connections to established theories become meaningful only after the relevant bridges have been built: finite-to-infinite consistency, response-map convergence, operator-domain control, and preservation of the live/record distinction.

Thus the outlook is intentionally cautious. The present work does not claim that the finite model already is a completed infinite theory or a final higher-level structure. It claims something narrower and testable: before such higher structures are invoked, there exists a reproducible discrete-finite CNNA core with an empty initial configuration, an explicit deterministic rule family, and declared finite selectors, in which the main objects can be computed, inspected, and challenged directly.

Software availability and assistance disclosure

The complete Python source boundary documented in this article is included as arXiv ancillary material under `anc/CNNA_structured_core_pkg`. Generative AI assistance was used for drafting, review, code inspection, and documentation support. The author is solely responsible for the mathematical content, the implementation, the reported claims, and their interpretation.

13 References

The following references provide standard discrete, graph-theoretic, matrix-analytic, Schur-complement, finite network-response, and finite-dimensional operator-algebraic background, together with the recent model comparators discussed in Sections 1 and 2. Historical primary sources are retained where they are mathematically foundational; recent references are used to locate the finite construction within current tree-growth and network-response work. They do not supply primitives for the CNNA construction.

References

- [1] F. Harary, *Graph Theory*, Addison–Wesley, 1969, ISBN 978-0-201-02787-7, <https://search.worldcat.org/title/64676>.
- [2] B. Bollobás, *Modern Graph Theory*, Graduate Texts in Mathematics, vol. 184, Springer, 1998, doi:10.1007/978-1-4612-0619-4.
- [3] C. Godsil and G. Royle, *Algebraic Graph Theory*, Graduate Texts in Mathematics, vol. 207, Springer, 2001, doi:10.1007/978-1-4613-0163-9.
- [4] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd edition, Cambridge University Press, 2013, ISBN 978-0-521-83940-2, <https://www.cambridge.org/core/books/matrix-analysis/9CF2CB491C9E97948B15FAD835EF9A8B>.
- [5] F. Zhang, editor, *The Schur Complement and Its Applications*, Numerical Methods and Algorithms, Springer, 2005, doi:10.1007/b105056.
- [6] E. B. Curtis and J. A. Morrow, “The Dirichlet to Neumann Map for a Resistor Network,” *SIAM Journal on Applied Mathematics*, 51(4), 1011–1029, 1991, doi:10.1137/0151051.
- [7] E. B. Curtis, D. Ingerman, and J. A. Morrow, “Circular planar graphs and resistor networks,” *Linear Algebra and its Applications*, 283(1–3), 115–150, 1998, doi:10.1016/S0024-3795(98)10087-3.

- [8] J.-F. Marckert, “Growing Random Uniform d -ary Trees,” *Annals of Combinatorics*, 27, 51–66, 2023, [doi:10.1007/s00026-022-00621-3](#), [arXiv:2105.05513](#).
- [9] L. Fredes and J.-F. Marckert, “Models of Random Subtrees of a Graph,” *Probability Surveys*, 20, 722–801, 2023, [doi:10.1214/23-PS22](#), [arXiv:2102.12738](#).
- [10] F. Dunlop and A. Mardin, “Galton–Watson Trees with First Ancestor Interaction,” *Journal of Statistical Physics*, 189, article 38, 2022, [doi:10.1007/s10955-022-03000-7](#), [arXiv:2203.02253](#).
- [11] O. Bodini, A. Genitrini, C. Mailler, and M. Naima, “Strict Monotonic Trees Arising from Evolutionary Processes: Combinatorial and Probabilistic Study,” *Advances in Applied Mathematics*, 133, article 102284, 2022, [doi:10.1016/j.aam.2021.102284](#), [https://hal.science/hal-02865198](#).
- [12] O. Bodini, A. Genitrini, and K. Nurligareev, “Growing Binary Trees,” *Electronic Proceedings in Theoretical Computer Science*, 445, 47–56, 2026, [doi:10.4204/EPTCS.445.7](#), [arXiv:2603.25972](#).
- [13] H. Gernandt and J. Rohleder, “A Calderón Type Inverse Problem for Tree Graphs,” *Linear Algebra and its Applications*, 646, 29–42, 2022, [doi:10.1016/j.laa.2022.03.018](#), [arXiv:2002.03670](#).
- [14] K. Beard, A. Stefan, R. Viator, and A. Welters, “Effective Operators and Their Variational Principles for Discrete Electrical Network Problems,” *Journal of Mathematical Physics*, 64(7), 073501, 2023, [doi:10.1063/5.0130429](#), [arXiv:2210.05761](#).
- [15] R. Peng and Z. Song, “Sparsified Block Elimination for Directed Laplacians,” in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*, 557–567, 2022, [doi:10.1145/3519935.3520053](#), [arXiv:2111.10257](#).
- [16] S. A. Avdonin, K. V. Khmelnytskaya, and V. V. Kravchenko, “Synthesis of a Quantum Tree Weyl Matrix,” *Boletín de la Sociedad Matemática Mexicana*, 29, article 82, 2023, [doi:10.1007/s40590-023-00561-9](#), [arXiv:2308.01911](#).
- [17] J. Kigami, “Harmonic Calculus on p.c.f. Self-Similar Sets,” *Transactions of the American Mathematical Society*, 335(2), 721–755, 1993, [doi:10.1090/S0002-9947-1993-1076617-1](#).
- [18] J. Kigami, *Analysis on Fractals*, Cambridge Tracts in Mathematics, vol. 143, Cambridge University Press, 2001, [doi:10.1017/CB09780511470943](#).
- [19] R. S. Strichartz, *Differential Equations on Fractals: A Tutorial*, Princeton University Press, 2006, [doi:10.1515/9780691186832](#).
- [20] I. M. Gelfand and M. A. Naimark, “On the Imbedding of Normed Rings into the Ring of Operators on Hilbert Space,” *Matematicheskii Sbornik*, 12(2), 197–217, 1943, [https://www.mathnet.ru/eng/sm6155](#).
- [21] R. Haag and D. Kastler, “An Algebraic Approach to Quantum Field Theory,” *Journal of Mathematical Physics*, 5(7), 848–861, 1964, [doi:10.1063/1.1704187](#).
- [22] O. Bratteli, “Inductive Limits of Finite Dimensional C^* -Algebras,” *Transactions of the American Mathematical Society*, 171, 195–234, 1972, [doi:10.1090/S0002-9947-1972-0312282-2](#).

- [23] M. Fannes, B. Nachtergaele, and R. F. Werner, “Finitely Correlated States on Quantum Spin Chains,” *Communications in Mathematical Physics*, 144(3), 443–490, 1992, [doi:10.1007/BF02099178](https://doi.org/10.1007/BF02099178).
- [24] F. Finster and N. Kamran, “Complex Structures on Jet Spaces and Bosonic Fock Space Dynamics for Causal Variational Principles,” *Pure and Applied Mathematics Quarterly*, 17(1), 55–140, 2021, [doi:10.4310/PAMQ.2021.v17.n1.a3](https://doi.org/10.4310/PAMQ.2021.v17.n1.a3), [arXiv:1808.03177](https://arxiv.org/abs/1808.03177).
- [25] I. E. Segal, “Irreducible Representations of Operator Algebras,” *Bulletin of the American Mathematical Society*, 53, 73–88, 1947, [doi:10.1090/S0002-9904-1947-08742-5](https://doi.org/10.1090/S0002-9904-1947-08742-5).

14 Source-tree object boundary

The Python source constructs finite objects at fixed values of b , L , mode, state, region selectors, finite caps, and tolerances. The constructed objects include finite trees with explicit open slots, finite growth-step records, finite edge dictionaries, finite distributions, finite Schur responses, finite directed responses, finite matrix algebras, finite GNS support quotients, finite transports, finite sign comparisons, finite backreaction-current rows, and row-family tables computed from those objects.

Table 32: Objects constructed by the inspected source tree.

Object group	Constructed finite objects
Growth and schedule	Node , Event , OpenBirthSlot , GrowthStep , EmptySetGrowthModel , record/live edge dictionaries, and schedule-dependent generated rows.
Carrier and prefix maps	terminal address sets, uniform carrier, terminal diagonal state, prefix push-forward, center/non-center split, finite total-variation rows, nested address-contrast role carriers, label inclusions, and non-unital corner embeddings.
Response matrices	symmetric Schur–DtN response matrices, directed Schur response matrices, symmetric/skew decomposition, role-compressed response matrices, and finite residual fields.
Finite algebra layer	finite matrix-basis closure, positive density from response support, finite Gram matrices, tolerance-resolved quotient support, full-basis left-action audits, explicit cyclic-span ranks, and finite GNS data.
Regional/local layer	regional response records, local generator systems, support projectors, restricted finite algebras, joined algebras, commutator matrices, finite commutants, Tomita star-map matrices, and split residual rows.
State and entropy rows	compressed carrier/GNS-state objects, refined boundary-role bases, finite entropy, finite relative entropy, Jensen–Shannon divergence, and record/live entropy rows.
Transport and orientation rows	interlevel projection residuals, seam/interface rows, directed-current transport, A/J orientation snapshots, orientation transport, local/global sign comparisons, and finite sign summaries.
Backreaction rows	live/record skew-response matrix $B_L = A_L^{\text{live}} - A_L^{\text{record}}$, its norm b_L , skew-norm difference β_L , oriented scalar ω_L , transported rows, explicit sibling-root birth levels, β -crossing rows, and branching comparisons.

Table 33: Objects absent from the inspected source tree.

Object name	Status in this source tree
Infinite-level object	The source constructs a finite directed system of growing carrier algebras and corner embeddings, but no norm completion, inductive-limit object, or projective-limit object over all L is present.
Von Neumann algebra closure	The source constructs finite matrix algebras and finite commutant bases; it contains no weak-operator or strong-operator closure.
Type-III result	The source contains finite rank, nullity, support, and residual rows; it contains no Type-III classification object.

Object name	Status in this source tree
Continuum spacetime model	The source contains finite growth steps, finite responses, and finite transport rows; it contains no continuum metric field.
Lorentz signature or local eigen-time	The source records causal depth, linearization index, and response/lapse weights; it contains no Lorentzian tangent structure or eigen-time field.
Unique global complex orientation	The source contains finite local and global sign-comparison rows; it contains no constructor selecting one absolute global sign of J .
Asymptotic theorem over L	The source emits rows at specified finite levels; it contains no theorem over all L or a convergence proof.

15 Generated row-file index

The table groups the row files whose values are read in the numerical paragraphs of this document. Each entry is tied to a command shown in the section where the corresponding object is discussed. The growth-schedule argument is part of the command label.

Table 34: Row-file groups generated from the current source tree.

Object layer	Command label	Row files
growth, carrier, and whole-network F1/F2	integrated / layer batch	emptyset_genesis_measurement.csv level_count_measurement.csv eventlog_hashes.csv projection_grammar.csv carrier_projectivity.csv terminal_diagonal_state_measurement.csv terminal_diagonal_state_projectivity.csv birth_linearization_invariance.csv f1f2_skew_orientation_measurement.csv live_record_history_delta.csv
slot-step comparison rows	integrated / slot step	birth_linearization_invariance.csv carrier_projectivity.csv terminal_diagonal_state_projectivity.csv f1f2_skew_orientation_measurement.csv
Schur response, directed response, and finite J	integrated / layer batch	regional_schur_dtn_response_measurement.csv directed_response_construction.csv directed_SA_decomposition.csv directed_skew_survival.csv directed_J_finite_object_comparison.csv
response algebra, GNS, and compressed state rows	integrated / layer batch	response_algebra_measurement.csv finite_gns_pre_representation_measurement.csv finite_gns_cyclicity_controls.csv compressed_carrier_state_projectivity.csv
local support algebra, Tomita, split, refined-basis, and entropy rows	local-operator run	local_operator_isotony.csv local_operator_commutation.csv local_operator_additivity.csv local_vacuum_cyclicity.csv faithful_state.csv commutant_duality.csv tomita_standard_form_compatibility.csv split_factorization.csv refined_port_basis_resolution.csv collared_split_refined_basis.csv record_live_relative_entropy.csv tomita_entropy_flow_alignment.csv multiscale_port_entropy_resolution.csv
interlevel transport and A/J orientation rows	integrated; A/J run	gns_generator_transport_maps.csv algebra_projection_morphism_measurement.csv conditional_expectation_commuting_square.csv operator_stack_residuals_classification.csv A_J_orientation_snapshot.csv A_J_orientation_transport.csv A_J_orientation_composition.csv
local/global J sign rows	local/global sign run	j_sign_global_boundary_sign_relation.csv local_global_J_sign_comparison.csv J_sign_sign_relation_summary.csv
live-record backreaction current	backreaction run	live_record_backreaction_current.csv backreaction_growth_profile.csv backreaction_transport.csv backreaction_current_summary.csv
birth-relative backreaction rows	birth-relative run	birth_relative_relative_backreaction_current.csv birth_relative_zero_crossing_by_region.csv birth_relative_mode_branching_comparison.csv birth_relative_backreaction_law_summary.csv
branching crossing rows	branching run	branching_zero_crossing_branching_shift_curve.csv branching_zero_crossing_branching_shift_summary.csv
finite structural theorem controls	package-consistency run	docs/FINITE_STRUCTURE_THEOREM_CONTROLS.csv

Object layer	Command label	Row files
nested multiscale handoff	nested handoff run	<code>docs/NESTED_MULTISCALE_CARRIER.csv</code>
		<code>docs/NESTED_MULTISCALE_HANDOFF.csv</code>
		<code>docs/NESTED_MULTISCALE_COMPOSITION.csv</code>
		<code>docs/NESTED_MULTISCALE_COMMON_AMBIENT_SCHUR.csv</code>
		<code>docs/NESTED_MULTISCALE_SUMMARY.csv</code>

16 Function-level source inventory

The table lists top-level classes and functions in the inspected Python source subtree. The line number is the definition line in the current source archive.

Table 35: Top-level code objects in the documented Python source surface.

Source file	Code object	Line	Object type
analysis_entrypoints/run_cnna_analysis.py	parse_args	21	finite function
analysis_entrypoints/run_cnna_analysis.py	main	40	finite function
core/algebra_support.py	_unit_projector	32	finite function
core/algebra_support.py	_sandwich	39	finite function
core/algebra_support.py	_symmetric_coupling	43	finite function
core/algebra_support.py	_skew_coupling	50	finite function
core/algebra_support.py	operator_scale_metadata	57	finite function
core/algebra_support.py	_nonzero_or_empty	80	finite function
core/algebra_support.py	local_operator_generators	85	finite function
core/algebra_support.py	role_coordinate_mapping	141	finite function
core/algebra_support.py	role_support_indices	169	finite function
core/algebra_support.py	role_support_projector	213	finite function
core/algebra_support.py	independently_restricted_operator_algebra	221	finite function
core/algebra_support.py	independent_embedding_metrics	275	finite function
core/algebra_support.py	local_operator_algebras	319	finite function
core/algebra_support.py	max_span_residual	326	finite function
core/algebra_support.py	max_star_residual	332	finite function
core/algebra_support.py	joined_algebra	338	finite function
core/algebra_support.py	commutator_metrics	345	finite function
core/algebra_support.py	state_commutator_metric	360	finite function
core/algebra_support.py	matrix_rank_from_columns	368	finite function
core/algebra_support.py	gns_support_data	381	finite function
core/algebra_support.py	gns_cyclic_rank	391	finite function
core/algebra_support.py	gns_separating_nullity	401	finite function
core/algebra_support.py	nullspace	417	finite function
core/algebra_support.py	commutant_algebra_basis	426	finite function
core/algebra_support.py	basis_to_algebra	448	finite function
core/algebra_support.py	star_map_matrix	452	finite function
core/algebra_support.py	finite_tomita_star_metrics	462	finite function
core/algebra_support.py	state_pair_backreaction_algebras	639	finite function
core/algebra_support.py	live_record_backreaction_algebras	686	finite function
core/algebra_support.py	identical_history_backreaction_algebras	690	finite function
core/algebra_support.py	_node_address_is_prefix	694	finite function
core/algebra_support.py	_descendants_at_exact_level	698	finite function
core/algebra_support.py	_lca_id_for_nodes	706	finite function
core/algebra_support.py	_cone_interior_to_frontier	715	finite function
core/algebra_support.py	node_level_cut_for_operator_system	726	finite function
core/algebra_support.py	node_level_schur_embedding_metrics	783	finite function
core/algebra_support.py	relative_common_ambient_schur_embedding_metrics	881	finite function
core/algebra_support.py	build_genealogical_corridor_regions	1035	finite function
core/algebra_support.py	genealogical_corridor_metadata	1110	finite function
core/compressed_state.py	compressed_state_object	24	finite function
core/compressed_state.py	compressed_projective_distribution	95	finite function
core/compressed_state.py	values_for_keys	120	finite function
core/compressed_state.py	key_is_center	124	finite function
core/compressed_state.py	sector_profile_tv	128	finite function
core/directed_dtn.py	DirectedSchurResponse	16	finite class
core/directed_dtn.py	state_directed_edges	27	finite function
core/directed_dtn.py	directed_local_matrix	50	finite function
core/directed_dtn.py	directed_schur_response	98	finite function
core/directed_dtn.py	directed_schur_response_from_model	139	finite function
core/directed_dtn.py	directed_environment_port_response_from_model	169	finite function

Source file	Code object	Line	Object type
core/directed_dtn.py	_descendants_at_level	235	finite function
core/directed_dtn.py	build_coarse_frontier_sibling_regions	243	finite function
core/directed_dtn.py	region_frontier_level	352	finite function
core/directed_dtn.py	region_is_coarse_frontier	357	finite function
core/directed_dtn.py	signed_role_basis_F1F2	360	finite function
core/directed_dtn.py	split_symmetric_skew	402	finite function
core/directed_dtn.py	j_finite_object_from_SA	408	finite function
core/emptyset_growth.py	RuleCoefficients	20	finite class
core/emptyset_growth.py	rule_coefficients	41	finite function
core/emptyset_growth.py	Node	59	finite class
core/emptyset_growth.py	Event	76	finite class
core/emptyset_growth.py	OpenBirthSlot	100	finite class
core/emptyset_growth.py	GrowthStep	110	finite class
core/emptyset_growth.py	EmptySetGrowthModel	121	finite class
core/entropy_quantities.py	_support_projector_from_density	12	finite function
core/entropy_quantities.py	von_neumann_entropy_no_epsilon	23	finite function
core/entropy_quantities.py	quantum_relative_entropy_no_epsilon	37	finite function
core/entropy_quantities.py	jensen_shannon_divergence_no_epsilon	76	finite function
core/entropy_quantities.py	record_live_entropy_metrics	94	finite function
core/math_utils.py	fro	19	finite function
core/math_utils.py	tv	23	finite function
core/math_utils.py	normalize_dict	28	finite function
core/math_utils.py	normalize_distribution	36	finite function
core/math_utils.py	tv_arr	47	finite function
core/math_utils.py	_float_field	58	finite function
core/math_utils.py	_max_metric	68	finite function
core/nested_multiscale.py	NestedRoleBasis	15	finite class
core/nested_multiscale.py	CommonAmbientSchurResult	24	finite class
core/nested_multiscale.py	helmert_slot_contrasts	34	finite function
core/nested_multiscale.py	_normalized_indicator	47	finite function
core/nested_multiscale.py	_relative_depths	59	finite function
core/nested_multiscale.py	nested_address_contrast_basis	64	finite function
core/nested_multiscale.py	label_inclusion_matrix	122	finite function
core/nested_multiscale.py	role_intertwining_metrics	133	finite function
core/nested_multiscale.py	corner_embed	153	finite function
core/nested_multiscale.py	corner_restrict	157	finite function
core/nested_multiscale.py	symmetric_open_grounded_matrix	161	finite function
core/nested_multiscale.py	_nodes_at_level_under	185	finite function
core/nested_multiscale.py	_cone_nodes	196	finite function
core/nested_multiscale.py	common_ambient_interlevel_schur	209	finite function
core/nested_multiscale.py	resolved_support_graph_metrics	258	finite function
core/operator_test_config.py	OperatorTestConfig	10	finite class
core/operator_test_config.py	iter_growth_models	26	finite function
core/projective_maps.py	prefix	21	finite function
core/projective_maps.py	terminal_carrier	27	finite function
core/projective_maps.py	terminal_state	32	finite function
core/projective_maps.py	push_forward	56	finite function
core/projective_maps.py	center_subset_prefix	64	finite function
core/projective_maps.py	split_distribution	72	finite function
core/projective_maps.py	response_energy_distribution	87	finite function
core/refined_port_basis.py	_normalized_indicator_vector	17	finite function
core/refined_port_basis.py	_split_frontier_group	28	finite function
core/refined_port_basis.py	refined_boundary_role_basis	36	finite function
core/refined_port_basis.py	refined_regional_response	75	finite function
core/refined_port_basis.py	refined_support_algebra	112	finite function
core/region_response.py	sibling_pair_regions	25	finite function
core/region_response.py	region_size_meta	35	finite function
core/region_response.py	positive_support_density_from_response	52	finite function
core/region_response.py	RegionalResponse	101	finite class
core/region_response.py	regional_response	119	finite function
core/region_response.py	signed_orientation_metrics	154	finite function
core/response_algebra.py	AlgebraBasis	22	finite class
core/response_algebra.py	rank_one	30	finite function
core/response_algebra.py	add_to_basis	34	finite function

Source file	Code object	Line	Object type
core/response_algebra.py	algebra_closure	45	finite function
core/response_algebra.py	project_coeffs	70	finite function
core/response_algebra.py	reconstruct	74	finite function
core/response_algebra.py	span_residual	81	finite function
core/response_algebra.py	positive_density_from_response	88	finite function
core/response_algebra.py	gens_from_response	105	finite function
core/response_algebra.py	build_algebras	144	finite function
core/response_algebra.py	state_value	155	finite function
core/response_algebra.py	GNSData	160	finite class
core/response_algebra.py	state_gram	184	finite function
core/response_algebra.py	quotient_support	195	finite function
core/response_algebra.py	left_coeff_matrix	204	finite function
core/response_algebra.py	_column_rank	216	finite function
core/response_algebra.py	measurement_gns	229	finite function
core/response_algebra.py	positive_trace_normalize_matrix	329	finite function
core/response_algebra.py	project_density_to_algebra_span	358	finite function
core/schur_dtn.py	SchurResponse	23	finite class
core/schur_dtn.py	RegionSpec	33	finite class
core/schur_dtn.py	schur_response	45	finite function
core/schur_dtn.py	schur_response_from_model	73	finite function
core/schur_dtn.py	build_sibling_regions	115	finite function
core/schur_dtn.py	normalized_indicator	159	finite function
core/schur_dtn.py	boundary_role_basis	169	finite function
core/schur_dtn.py	compress_response	197	finite function
core/schur_dtn.py	role_for_boundary_node	201	finite function
core/schur_dtn.py	atom_key	211	finite function
core/signed_current.py	orthonormal_pair	22	finite function
core/signed_current.py	F1_F2_vectors	44	finite function
core/signed_current.py	orientation_measurement	65	finite function
derived_quantities/backreaction_	_to_int	12	finite function
birth_relative.py			
derived_quantities/backreaction_	_to_float	21	finite function
birth_relative.py			
derived_quantities/backreaction_	_sign	30	finite function
birth_relative.py			
derived_quantities/backreaction_	_region_birth_level	36	finite function
birth_relative.py			
derived_quantities/backreaction_	birth_relative_backreaction_rows	52	finite function
birth_relative.py			
derived_quantities/backreaction_	_linear_zero_crossing_age	102	finite function
birth_relative.py			
derived_quantities/backreaction_	birth_relative_zero_crossing_rows	118	finite function
birth_relative.py			
derived_quantities/backreaction_	birth_relative_mode_branching_rows	179	finite function
birth_relative.py			
derived_quantities/backreaction_	birth_relative_region_family_rows	213	finite function
birth_relative.py			
derived_quantities/backreaction_	birth_relative_summary_rows	240	finite function
birth_relative.py			
derived_quantities/backreaction_	write_birth_relative_backreaction_	285	finite function
birth_relative.py	current_report		
derived_quantities/backreaction_	_to_int	13	finite function
branching_shift.py			
derived_quantities/backreaction_	_to_float	22	finite function
branching_shift.py			
derived_quantities/backreaction_	read_csv_rows	31	finite function
branching_shift.py			
derived_quantities/backreaction_	_mode_branching_key	36	finite function
branching_shift.py			
derived_quantities/backreaction_	deduplicate_mode_branching_rows	40	finite function
branching_shift.py			
derived_quantities/backreaction_	branching_shift_curve_rows	63	finite function
branching_shift.py			
derived_quantities/backreaction_	_linear_fit_slope	92	finite function
branching_shift.py			

Source file	Code object	Line	Object type
derived_quantities/backreaction_branching_shift.py	branching_shift_summary_rows	102	finite function
derived_quantities/backreaction_branching_shift.py	write_branching_zero_crossing_shift_report	147	finite function
derived_quantities/backreaction_current.py	_regions	29	finite function
derived_quantities/backreaction_current.py	_directed_parts	39	finite function
derived_quantities/backreaction_current.py	_state_pair_snapshot	57	finite function
derived_quantities/backreaction_current.py	_omega	63	finite function
derived_quantities/backreaction_current.py	_sign	69	finite function
derived_quantities/backreaction_current.py	_rel	75	finite function
derived_quantities/backreaction_current.py	_row_sum_condition_note	79	finite function
derived_quantities/backreaction_current.py	backreaction_current_rows	89	finite function
derived_quantities/backreaction_current.py	backreaction_growth_profile_rows	226	finite function
derived_quantities/backreaction_current.py	backreaction_region_parameter_sensitivity_rows	291	finite function
derived_quantities/backreaction_current.py	_snapshot_for_region	337	finite function
derived_quantities/backreaction_current.py	backreaction_transport_rows	363	finite function
derived_quantities/backreaction_current.py	backreaction_current_summary_rows	443	finite function
derived_quantities/backreaction_current.py	write_live_record_backreaction_current_report	506	finite function
derived_quantities/baseline.py	level_models	21	finite function
derived_quantities/baseline.py	genesis_rows	37	finite function
derived_quantities/baseline.py	projection_grammar_rows	77	finite function
derived_quantities/baseline.py	carrier_state_rows	106	finite function
derived_quantities/baseline.py	orientation_rows	156	finite function
derived_quantities/baseline.py	region_and_gns_rows	174	finite function
derived_quantities/birth_linearization_invariance.py	_edge_weights_by_address	11	finite function
derived_quantities/birth_linearization_invariance.py	_dict_linf	29	finite function
derived_quantities/birth_linearization_invariance.py	_dict_l1	36	finite function
derived_quantities/birth_linearization_invariance.py	_node_field_by_address	41	finite function
derived_quantities/birth_linearization_invariance.py	_rank_reverse_pair	45	finite function
derived_quantities/birth_linearization_invariance.py	birth_linearization_invariance_rows	65	finite function
derived_quantities/compressed_state_projection_consistency.py	compressed_carrier_state_rows	21	finite function
derived_quantities/compressed_state_projection_consistency.py	state_construction_track_ledger_rows	159	finite function
derived_quantities/computed_status_rows.py	operator_stack_residuals_classification_rows	26	finite function
derived_quantities/computed_status_rows.py	final_computed_status_rows_full	96	finite function
derived_quantities/computed_status_rows.py	write_markdown_full	152	finite function
derived_quantities/directed_current_transport.py	_first_transport_region	15	finite function
derived_quantities/directed_current_transport.py	_transport_region_meta	31	finite function
derived_quantities/directed_current_transport.py	_open_out_directed_snapshot	40	finite function

Source file	Code object	Line	Object type
derived_quantities/directed_current_transport.py	_address_at_target	61	finite function
derived_quantities/directed_current_transport.py	_boundary_prefix_transport_matrix	68	finite function
derived_quantities/directed_current_transport.py	_safe_rel_err	115	finite function
derived_quantities/directed_current_transport.py	_scale_fit_error	119	finite function
derived_quantities/directed_current_transport.py	_skew_orientation_sign	128	finite function
derived_quantities/directed_current_transport.py	_j_orientation_metrics	135	finite function
derived_quantities/directed_current_transport.py	_snapshot_cache	157	finite function
derived_quantities/directed_current_transport.py	_pair_records	176	finite function
derived_quantities/directed_current_transport.py	directed_current_transport_rows	191	finite function
derived_quantities/directed_current_transport.py	directed_current_composition_rows	276	finite function
derived_quantities/directed_current_transport.py	directed_j_orientation_summary_rows	344	finite function
derived_quantities/directed_current_transport.py	write_directed_current_transport_report	398	finite function
derived_quantities/directed_current_transport.py	directed_large_L_region_coverage_rows	460	finite function
derived_quantities/directed_current_transport.py	_safe_region_matrix_metrics	549	finite function
derived_quantities/directed_current_transport.py	_safe_region_plane_metrics	566	finite function
derived_quantities/directed_current_transport.py	directed_large_L_coverage_summary_rows	583	finite function
derived_quantities/directed_current_transport.py	write_directed_region_coverage_report	639	finite function
derived_quantities/directed_operator_stack.py	_first_directed_region	15	finite function
derived_quantities/directed_operator_stack.py	_region_meta	31	finite function
derived_quantities/directed_operator_stack.py	_response_for_relations	42	finite function
derived_quantities/directed_operator_stack.py	_matrix_metrics	61	finite function
derived_quantities/directed_operator_stack.py	_plane_metrics	97	finite function
derived_quantities/directed_operator_stack.py	_reference_open_out_snapshot	124	finite function
derived_quantities/directed_operator_stack.py	directed_response_construction_rows	133	finite function
derived_quantities/directed_operator_stack.py	directed_sa_decomposition_rows	171	finite function
derived_quantities/directed_operator_stack.py	directed_signed_role_survival_rows	198	finite function
derived_quantities/directed_operator_stack.py	directed_boundary_response_model_comparison_rows	232	finite function
derived_quantities/directed_operator_stack.py	directed_skew_survival_rows	271	finite function
derived_quantities/directed_operator_stack.py	directed_operator_transport_rows	304	finite function
derived_quantities/directed_operator_stack.py	directed_j_finite_object_comparison_rows	365	finite function
derived_quantities/directed_operator_stack.py	directed_operator_stack_summary_rows	394	finite function
derived_quantities/directed_operator_stack.py	write_directed_operator_stack_report	452	finite function
derived_quantities/finite_structure_theorem_controls.py	_growth_signature	21	finite function

Source file	Code object	Line	Object type
derived_quantities/finite_structure_theorem_controls.py	_expected_addresses	66	finite function
derived_quantities/finite_structure_theorem_controls.py	growth_well_defined_control	73	finite function
derived_quantities/finite_structure_theorem_controls.py	symmetric_schur_positivity_control	133	finite function
derived_quantities/finite_structure_theorem_controls.py	gns_quotient_control	178	finite function
derived_quantities/finite_structure_theorem_controls.py	orientation_flip_control	194	finite function
derived_quantities/finite_structure_theorem_controls.py	backreaction_separation_control	234	finite function
derived_quantities/finite_structure_theorem_controls.py	prefix_handoff_control	255	finite function
derived_quantities/finite_structure_theorem_controls.py	finite_structure_theorem_control_rows	332	finite function
derived_quantities/gns_cyclicity_controls.py	_control_algebras	11	finite function
derived_quantities/gns_cyclicity_controls.py	finite_gns_cyclicity_control_rows	22	finite function
derived_quantities/j_orientation_projectivity.py	_memo_key	20	finite function
derived_quantities/j_orientation_projectivity.py	_regions	24	finite function
derived_quantities/j_orientation_projectivity.py	_regions_by_id	34	finite function
derived_quantities/j_orientation_projectivity.py	_region_meta	38	finite function
derived_quantities/j_orientation_projectivity.py	_safe_eigs	51	finite function
derived_quantities/j_orientation_projectivity.py	_response_parts	64	finite function
derived_quantities/j_orientation_projectivity.py	_metric_choice	79	finite function
derived_quantities/j_orientation_projectivity.py	_orientation_sign	89	finite function
derived_quantities/j_orientation_projectivity.py	_snapshot_for_region	98	finite function
derived_quantities/j_orientation_projectivity.py	_snapshot_cache	143	finite function
derived_quantities/j_orientation_projectivity.py	j_orientation_snapshot_rows	159	finite function
derived_quantities/j_orientation_projectivity.py	j_orientation_transport_rows	207	finite function
derived_quantities/j_orientation_projectivity.py	j_orientation_composition_rows	288	finite function
derived_quantities/j_orientation_projectivity.py	j_orientation_sign_scope_rows	363	finite function
derived_quantities/j_orientation_projectivity.py	j_orientation_projectivity_summary_rows	388	finite function
derived_quantities/j_orientation_projectivity.py	write_b_j_orientation_projectivity_report	442	finite function
derived_quantities/j_orientation_projectivity.py	_snapshot_A_for_state	530	finite function
derived_quantities/j_orientation_projectivity.py	_snapshot_A_cache	570	finite function
derived_quantities/j_orientation_projectivity.py	a_j_orientation_snapshot_rows	585	finite function
derived_quantities/j_orientation_projectivity.py	a_j_orientation_transport_rows	631	finite function
derived_quantities/j_orientation_projectivity.py	a_j_orientation_composition_rows	697	finite function
derived_quantities/j_orientation_projectivity.py	a_j_orientation_projectivity_summary_rows	734	finite function
derived_quantities/j_orientation_projectivity.py	write_j_orientation_a_orientation_projectivity_report	763	finite function

Source file	Code object	Line	Object type
derived_quantities/local_global_j_sign_relation.py	_frontier_nodes	32	finite function
derived_quantities/local_global_j_sign_relation.py	_choose_global_frontier_level	36	finite function
derived_quantities/local_global_j_sign_relation.py	_global_boundary_sets	51	finite function
derived_quantities/local_global_j_sign_relation.py	_global_signed_basis	72	finite function
derived_quantities/local_global_j_sign_relation.py	_safe_response	119	finite function
derived_quantities/local_global_j_sign_relation.py	global_j_boundary_sign_relation_rows	139	finite function
derived_quantities/local_global_j_sign_relation.py	local_global_j_sign_comparison_rows	242	finite function
derived_quantities/local_global_j_sign_relation.py	local_global_j_sign_summary_rows	296	finite function
derived_quantities/local_global_j_sign_relation.py	write_local_global_j_sign_relation_report	343	finite function
derived_quantities/operator_stack_consistency_checks.py	RegionMeasurementSnapshot	25	finite class
derived_quantities/operator_stack_consistency_checks.py	_state_edges	47	finite function
derived_quantities/operator_stack_consistency_checks.py	_selected_schur_response	51	finite function
derived_quantities/operator_stack_consistency_checks.py	_first_snapshot	94	finite function
derived_quantities/operator_stack_consistency_checks.py	_snapshots	130	finite function
derived_quantities/operator_stack_consistency_checks.py	_span_rank	140	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_role_dimension_condition_note_rows	156	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_generator_rank_condition_note_rows	200	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_coordinate_gauge_condition_note_rows	239	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_boundary_response_model_condition_note_rows	313	finite function
derived_quantities/operator_stack_consistency_checks.py	_orientation_lookup	354	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_skew_survival_condition_note_rows	364	finite function
derived_quantities/operator_stack_consistency_checks.py	operator_stack_consistency_checks_summary_rows	405	finite function
derived_quantities/operator_stack_consistency_checks.py	write_operator_stack_condition_consistency_report	452	finite function
derived_quantities/operator_stack_interface_residuals.py	_finite	11	finite function
derived_quantities/operator_stack_interface_residuals.py	_key_pair	19	finite function
derived_quantities/operator_stack_interface_residuals.py	_index	29	finite function
derived_quantities/operator_stack_interface_residuals.py	_composition_index	38	finite function
derived_quantities/operator_stack_interface_residuals.py	_metric	57	finite function
derived_quantities/operator_stack_interface_residuals.py	_dominant_seam	63	finite function
derived_quantities/operator_stack_interface_residuals.py	_seam_computed_status	80	finite function
derived_quantities/operator_stack_interface_residuals.py	operator_stack_seam_localization_rows	93	finite function
derived_quantities/operator_stack_interface_residuals.py	operator_stack_live_record_sym_comparison_rows	178	finite function
derived_quantities/operator_stack_interface_residuals.py	operator_stack_level_scaling_rows	225	finite function

Source file	Code object	Line	Object type
derived_quantities/operator_stack_interface_residuals.py	operator_stack_map_naturality_measurement_rows	277	finite function
derived_quantities/operator_stack_interface_residuals.py	write_operator_stack_residual_report	317	finite function
derived_quantities/nested_multiscale_handoff.py	_ambient_J_from_SA	34	finite function
derived_quantities/nested_multiscale_handoff.py	NestedSnapshot	72	finite class
derived_quantities/nested_multiscale_handoff.py	_first_terminal_region	88	finite function
derived_quantities/nested_multiscale_handoff.py	build_nested_snapshot	93	finite function
derived_quantities/nested_multiscale_handoff.py	_relative_error	131	finite function
derived_quantities/nested_multiscale_handoff.py	_deterministic_test_matrices	135	finite function
derived_quantities/nested_multiscale_handoff.py	_state_inner_product	146	finite function
derived_quantities/nested_multiscale_handoff.py	_address_sequence	150	finite function
derived_quantities/nested_multiscale_handoff.py	_permutation_from_addresses	154	finite function
derived_quantities/nested_multiscale_handoff.py	nested_carrier_rows	171	finite function
derived_quantities/nested_multiscale_handoff.py	nested_handoff_rows	236	finite function
derived_quantities/nested_multiscale_handoff.py	common_ambient_schur_rows	398	finite function
derived_quantities/nested_multiscale_handoff.py	nested_multiscale_summary_rows	496	finite function
derived_quantities/nested_multiscale_handoff.py	nested_multiscale_all_rows	593	finite function
derived_quantities/operator_stack_residuals.py	FullRegionData	23	finite class
derived_quantities/operator_stack_residuals.py	build_first_region_data	37	finite function
derived_quantities/operator_stack_residuals.py	common_labels	54	finite function
derived_quantities/operator_stack_residuals.py	_basis_matrix	58	finite function
derived_quantities/operator_stack_residuals.py	project_to_label_span	64	finite function
derived_quantities/operator_stack_residuals.py	reconstruct_from_labels	72	finite function
derived_quantities/operator_stack_residuals.py	pi_role_project	83	finite function
derived_quantities/operator_stack_residuals.py	role_projection_residual	88	finite function
derived_quantities/operator_stack_residuals.py	matrix_log_spd	96	finite function
derived_quantities/operator_stack_residuals.py	commutator_delta	102	finite function
derived_quantities/operator_stack_residuals.py	center_expectation	106	finite function
derived_quantities/operator_stack_residuals.py	generator_list	115	finite function
derived_quantities/operator_stack_residuals.py	generator_gram	119	finite function
derived_quantities/operator_stack_residuals.py	interlevel_operator_rows	125	finite function
derived_quantities/rank_kernel_conditions.py	local_s_response_density	22	finite function
derived_quantities/rank_kernel_conditions.py	measure_rank_kernel_condition_rows	47	finite function
derived_quantities/rank_kernel_conditions.py	construction_coverage_rows	122	finite function

Source file	Code object	Line	Object type
derived_quantities/rank_kernel_conditions.py	fixed_rule_constants_measurement_rows	143	finite function
derived_quantities/rank_kernel_conditions.py	regional_rank_kernel_condition_rows	162	finite function
derived_quantities/rank_kernel_conditions.py	terminal_density_state_measurement_rows	178	finite function
derived_quantities/structural_ledgers.py	is_reference_model	21	finite function
derived_quantities/structural_ledgers.py	parameter_purity_rows	25	finite function
derived_quantities/structural_ledgers.py	live_record_structural_calculation_ledger_rows	50	finite function
derived_quantities/structural_ledgers.py	live_record_history_delta_ledger_rows	114	finite function
derived_quantities/structural_ledgers.py	carrier_effect_operator_rows	183	finite function
derived_quantities/structural_ledgers.py	state_sector_decomposition_rows	235	finite function
io/csv_writer.py	write_csv	17	finite function
io/csv_writer.py	sha256_json	36	finite function
object_checks/backreaction_current_localization.py	_projectors	24	finite function
object_checks/backreaction_current_localization.py	_row_computed_status	34	finite function
object_checks/backreaction_current_localization.py	_track_key	44	finite function
object_checks/backreaction_current_localization.py	_summarize_track	48	finite function
object_checks/backreaction_current_localization.py	backreaction_current_localization_rows	101	finite function
object_checks/backreaction_current_localization.py	backreaction_current_localization_report	170	finite function
object_checks/bianconi_simplicial_statistics_bridge.py	bianconi_statistics_bridge_condition_rows	57	finite function
object_checks/bianconi_simplicial_statistics_bridge.py	bianconi_branching_sector_expectation_rows	61	finite function
object_checks/bianconi_simplicial_statistics_bridge.py	bianconi_statistics_bridge_report	91	finite function
object_checks/collared_split_refined_basis.py	_median	25	finite function
object_checks/collared_split_refined_basis.py	_track_key	30	finite function
object_checks/collared_split_refined_basis.py	_summarize	34	finite function
object_checks/collared_split_refined_basis.py	collared_split_refined_basis_rows	62	finite function
object_checks/collared_split_refined_basis.py	collared_split_refined_basis_report	121	finite function
object_checks/commutant_duality.py	_computed_status	29	finite function
object_checks/commutant_duality.py	commutant_duality_rows	39	finite function
object_checks/commutant_duality.py	commutant_report	82	finite function
object_checks/commutator_decay_by_genealogical_distance.py	_finite	31	finite function
object_checks/commutator_decay_by_genealogical_distance.py	_track_key	39	finite function
object_checks/commutator_decay_by_genealogical_distance.py	_summarize_track	49	finite function
object_checks/commutator_decay_by_genealogical_distance.py	commutator_decay_by_genealogical_distance_rows	112	finite function
object_checks/commutator_decay_by_genealogical_distance.py	commutator_decay_by_genealogical_distance_report	179	finite function
object_checks/core_construction_dependencies.py	core_theory_construction_rows	146	finite function

Source file	Code object	Line	Object type
object_checks/core_construction_dependencies.py	presentation_mainline_rows	150	finite function
object_checks/core_construction_dependencies.py	core_theory_construction_report	154	finite function
object_checks/core_construction_dependencies.py	presentation_mainline_report	196	finite function
object_checks/cyclicity_dimension_rank_bound.py	cyclicity_dimension_rank_bound_rows	31	finite function
object_checks/cyclicity_dimension_rank_bound.py	cyclicity_dimension_rank_bound_report	77	finite function
object_checks/decomposed_entropy_flow.py	_finite	8	finite function
object_checks/decomposed_entropy_flow.py	_median	16	finite function
object_checks/decomposed_entropy_flow.py	_pearson	21	finite function
object_checks/decomposed_entropy_flow.py	_track_key	33	finite function
object_checks/decomposed_entropy_flow.py	_same_direction	37	finite function
object_checks/decomposed_entropy_flow.py	decomposed_entropy_flow_rows	41	finite function
object_checks/decomposed_entropy_flow.py	decomposed_entropy_flow_report	123	finite function
object_checks/decomposed_geometry_quantities.py	_finite	8	finite function
object_checks/decomposed_geometry_quantities.py	_median	16	finite function
object_checks/decomposed_geometry_quantities.py	_key	21	finite function
object_checks/decomposed_geometry_quantities.py	_component_index	25	finite function
object_checks/decomposed_geometry_quantities.py	_ratio	29	finite function
object_checks/decomposed_geometry_quantities.py	_within	33	finite function
object_checks/decomposed_geometry_quantities.py	_dominant_explanation	37	finite function
object_checks/decomposed_geometry_quantities.py	_surface_track_key	59	finite function
object_checks/decomposed_geometry_quantities.py	_summarize_track	63	finite function
object_checks/decomposed_geometry_quantities.py	_base_row	109	finite function
object_checks/decomposed_geometry_quantities.py	decomposed_geometry_quantities_rows	171	finite function
object_checks/decomposed_geometry_quantities.py	decomposed_geometry_quantities_report	211	finite function
object_checks/deformed_fractal_bridge.py	deformed_fractal_bridge_rows	6	finite function
object_checks/deformed_fractal_bridge.py	deformed_fractal_bridge_report	62	finite function
object_checks/destroyed_history_oriented_geometry.py	_finite	8	finite function
object_checks/destroyed_history_oriented_geometry.py	_median	16	finite function
object_checks/destroyed_history_oriented_geometry.py	_key	21	finite function
object_checks/destroyed_history_oriented_geometry.py	_idx	25	finite function
object_checks/destroyed_history_oriented_geometry.py	_ratio	29	finite function
object_checks/destroyed_history_oriented_geometry.py	_control_measures	33	finite function
object_checks/destroyed_history_oriented_geometry.py	destroyed_history_oriented_geometry_rows	49	finite function

Source file	Code object	Line	Object type
object_checks/destroyed_history_oriented_geometry.py	destroyed_history_oriented_geometry_report	138	finite function
object_checks/entropy_gradient_current.py	_finite	8	finite function
object_checks/entropy_gradient_current.py	_pearson	16	finite function
object_checks/entropy_gradient_current.py	_median	28	finite function
object_checks/entropy_gradient_current.py	_track_key	33	finite function
object_checks/entropy_gradient_current.py	entropy_gradient_current_rows	37	finite function
object_checks/entropy_gradient_current.py	entropy_gradient_current_report	110	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_finite	26	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_median	34	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_js	39	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_swap_left_right_permutation	44	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_density_key	60	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_region_shape_key	69	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_base_key_without_L	85	finite function
object_checks/entropy_measure_parameter_sensitivity.py	_base_key_without_mode	89	finite function
object_checks/entropy_measure_parameter_sensitivity.py	entropy_measure_parameter_sensitivity_rows	93	finite function
object_checks/entropy_measure_parameter_sensitivity.py	entropy_measure_parameter_sensitivity_report	209	finite function
object_checks/entropy_normalized_geometry.py	_finite	8	finite function
object_checks/entropy_normalized_geometry.py	_hist_index	16	finite function
object_checks/entropy_normalized_geometry.py	_base_key	26	finite function
object_checks/entropy_normalized_geometry.py	_normed	30	finite function
object_checks/entropy_normalized_geometry.py	entropy_normalized_geometry_rows	34	finite function
object_checks/entropy_normalized_geometry.py	entropy_normalized_geometry_report	151	finite function
object_checks/entropy_ordered_complement.py	_finite	8	finite function
object_checks/entropy_ordered_complement.py	_hist_index	16	finite function
object_checks/entropy_ordered_complement.py	entropy_ordered_complement_rows	24	finite function
object_checks/entropy_ordered_complement.py	entropy_ordered_complement_report	88	finite function
object_checks/entropy_ordered_locality.py	_finite	8	finite function
object_checks/entropy_ordered_locality.py	_median	16	finite function
object_checks/entropy_ordered_locality.py	_hist_index	21	finite function
object_checks/entropy_ordered_locality.py	_track_key	31	finite function
object_checks/entropy_ordered_locality.py	_summarize_track	35	finite function
object_checks/entropy_ordered_locality.py	entropy_ordered_locality_rows	77	finite function

Source file	Code object	Line	Object type
object_checks/entropy_ordered_locality.py	entropy_ordered_locality_report	137	finite function
object_checks/entropy_ordered_split.py	_finite	8	finite function
object_checks/entropy_ordered_split.py	_median	16	finite function
object_checks/entropy_ordered_split.py	_hist_index	21	finite function
object_checks/entropy_ordered_split.py	_track_key	29	finite function
object_checks/entropy_ordered_split.py	_summarize_track	33	finite function
object_checks/entropy_ordered_split.py	entropy_ordered_split_rows	67	finite function
object_checks/entropy_ordered_split.py	entropy_ordered_split_report	115	finite function
object_checks/extended_entropy_level_trajectory.py	_finite	8	finite function
object_checks/extended_entropy_level_trajectory.py	_median	16	finite function
object_checks/extended_entropy_level_trajectory.py	_track_key	21	finite function
object_checks/extended_entropy_level_trajectory.py	_trend	25	finite function
object_checks/extended_entropy_level_trajectory.py	extended_entropy_level_trajectory_rows	41	finite function
object_checks/extended_entropy_level_trajectory.py	extended_entropy_level_trajectory_report	105	finite function
object_checks/f1_f2_orientation_resolution.py	_unique	28	finite function
object_checks/f1_f2_orientation_resolution.py	_path_from_lca_to_node	32	finite function
object_checks/f1_f2_orientation_resolution.py	_chunks_by_birth_time	45	finite function
object_checks/f1_f2_orientation_resolution.py	_indicator	59	finite function
object_checks/f1_f2_orientation_resolution.py	_gram_schmidt	70	finite function
object_checks/f1_f2_orientation_resolution.py	_paired_shells	88	finite function
object_checks/f1_f2_orientation_resolution.py	_multishell_f1f2_basis	113	finite function
object_checks/f1_f2_orientation_resolution.py	_profile	150	finite function
object_checks/f1_f2_orientation_resolution.py	_response	158	finite function
object_checks/f1_f2_orientation_resolution.py	_js	202	finite function
object_checks/f1_f2_orientation_resolution.py	_profile_gap	207	finite function
object_checks/f1_f2_orientation_resolution.py	_oriented_distance	214	finite function
object_checks/f1_f2_orientation_resolution.py	_transform_matrix	220	finite function
object_checks/f1_f2_orientation_resolution.py	_apply_transform	248	finite function
object_checks/f1_f2_orientation_resolution.py	_uniform_density	255	finite function
object_checks/f1_f2_orientation_resolution.py	_cosine	259	finite function
object_checks/f1_f2_orientation_resolution.py	_meta_key_without_L	268	finite function
object_checks/f1_f2_orientation_resolution.py	_meta_key_without_mode	272	finite function
object_checks/f1_f2_orientation_resolution.py	f1_f2_orientation_resolution_rows	276	finite function

Source file	Code object	Line	Object type
object_checks/f1_f2_orientation_resolution.py	f1_f2_orientation_resolution_report	419	finite function
object_checks/f1_f2_oriented_history_measure.py	_finite	27	finite function
object_checks/f1_f2_oriented_history_measure.py	_js	35	finite function
object_checks/f1_f2_oriented_history_measure.py	_coupling	40	finite function
object_checks/f1_f2_oriented_history_measure.py	_sign	46	finite function
object_checks/f1_f2_oriented_history_measure.py	_orientation_gap	52	finite function
object_checks/f1_f2_oriented_history_measure.py	_oriented_distance	59	finite function
object_checks/f1_f2_oriented_history_measure.py	_f1f2_plane_response	70	finite function
object_checks/f1_f2_oriented_history_measure.py	_plane_transform	111	finite function
object_checks/f1_f2_oriented_history_measure.py	_apply_transform	125	finite function
object_checks/f1_f2_oriented_history_measure.py	_uniform_plane_density	134	finite function
object_checks/f1_f2_oriented_history_measure.py	_meta_key_without_L	140	finite function
object_checks/f1_f2_oriented_history_measure.py	_meta_key_without_mode	148	finite function
object_checks/f1_f2_oriented_history_measure.py	f1_f2_oriented_history_measure_rows	156	finite function
object_checks/f1_f2_oriented_history_measure.py	f1_f2_oriented_history_measure_report	328	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_finite	8	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_median	16	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_key	21	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_surface_key	31	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_f1f2_index	42	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_measure_index	46	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_bool01	50	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_ratio	57	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_naive_tensor_status	61	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_replacement_rule	85	finite function
object_checks/f1f2_oriented_geometry_replacement.py	f1f2_oriented_geometry_replacement_rows	125	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_summary_computed_status	193	finite function
object_checks/f1f2_oriented_geometry_replacement.py	f1f2_oriented_geometry_replacement_summary_rows	210	finite function
object_checks/f1f2_oriented_geometry_replacement.py	_summary_row	227	finite function
object_checks/f1f2_oriented_geometry_replacement.py	f1f2_oriented_geometry_replacement_report	253	finite function
object_checks/f1f2_signature_mode_comparison.py	_finite	24	finite function
object_checks/f1f2_signature_mode_comparison.py	_median	32	finite function
object_checks/f1f2_signature_mode_comparison.py	_signature	37	finite function

Source file	Code object	Line	Object type
object_checks/f1f2_signature_mode_comparison.py	_signature_gap	45	finite function
object_checks/f1f2_signature_mode_comparison.py	_cosine	51	finite function
object_checks/f1f2_signature_mode_comparison.py	_strength_range	60	finite function
object_checks/f1f2_signature_mode_comparison.py	_track_key_from_meta	70	finite function
object_checks/f1f2_signature_mode_comparison.py	_pairwise_state_stats	74	finite function
object_checks/f1f2_signature_mode_comparison.py	f1f2_signature_mode_comparison_rows	135	finite function
object_checks/f1f2_signature_mode_comparison.py	mode_mode_stable_f1f2_signature_summary_rows	264	finite function
object_checks/f1f2_signature_mode_comparison.py	f1f2_signature_mode_comparison_report	308	finite function
object_checks/f1f2_signature_mode_comparison.py	mode_mode_stable_f1f2_signature_summary_report	327	finite function
object_checks/faithful_state.py	_computed_status	38	finite function
object_checks/faithful_state.py	faithful_state_rows	50	finite function
object_checks/faithful_state.py	faithful_state_report	95	finite function
object_checks/finite_complement_relations.py	finite_complement_relations_rows	31	finite function
object_checks/finite_complement_relations.py	finite_complement_relations_report	86	finite function
object_checks/finite_property_inventory.py	finite_finite_property_inventory_rows	6	finite function
object_checks/finite_property_inventory.py	previous_reproduction_measurement_rows	132	finite function
object_checks/finite_property_inventory.py	finite_property_inventory_report	165	finite function
object_checks/finite_region_net.py	FiniteRegionRecord	21	finite class
object_checks/finite_region_net.py	_descendants_until	33	finite function
object_checks/finite_region_net.py	finite_region_records	43	finite function
object_checks/finite_region_net.py	region_grammar_rows	80	finite function
object_checks/finite_region_net.py	region_inclusion_rows	106	finite function
object_checks/finite_region_net.py	region_poset_summary	133	finite function
object_checks/finite_region_net.py	region_net_report	166	finite function
object_checks/fixed_measure_ratio_summary.py	_finite	8	finite function
object_checks/fixed_measure_ratio_summary.py	_median	16	finite function
object_checks/fixed_measure_ratio_summary.py	fixed_measure_ratio_summary_rows	21	finite function
object_checks/fixed_measure_ratio_summary.py	fixed_measure_ratio_summary_report	67	finite function
object_checks/fixed_oriented_history_measure.py	_finite	8	finite function
object_checks/fixed_oriented_history_measure.py	_median	16	finite function
object_checks/fixed_oriented_history_measure.py	_key	21	finite function
object_checks/fixed_oriented_history_measure.py	fixed_oriented_history_measure_rows	25	finite function
object_checks/fixed_oriented_history_measure.py	fixed_oriented_history_measure_report	104	finite function
object_checks/fock_aqft_bridge.py	fock_aqft_bridge_comparison_register_rows	76	finite function

Source file	Code object	Line	Object type
object_checks/fock_aqft_bridge.py	_dominance_ratio	80	finite function
object_checks/fock_aqft_bridge.py	fock_aqft_one_particle_comparison_rows	96	finite function
object_checks/fock_aqft_bridge.py	fock_aqft_bridge_report	155	finite function
object_checks/four_path_transition_matrix.py	four_path_transition_matrix_rows	70	finite function
object_checks/four_path_transition_matrix.py	four_path_dependency_table_priority_rows	74	finite function
object_checks/four_path_transition_matrix.py	four_path_literature_bridge_rows	93	finite function
object_checks/four_path_transition_matrix.py	closed_finite_results_rows	130	finite function
object_checks/four_path_transition_matrix.py	external_measurement_assessment_rows	171	finite function
object_checks/four_path_transition_matrix.py	consistency_prerequisite_rows	218	finite function
object_checks/four_path_transition_matrix.py	presentation_selection_rows	255	finite function
object_checks/four_path_transition_matrix.py	four_path_transition_report	284	finite function
object_checks/four_path_transition_matrix.py	closed_results_report	309	finite function
object_checks/genealogical_corridor_commutation.py	_computed_status	27	finite function
object_checks/genealogical_corridor_commutation.py	genealogical_corridor_commutator_rows	41	finite function
object_checks/genealogical_corridor_commutation.py	genealogical_corridor_commutation_report	81	finite function
object_checks/genealogical_corridor_split.py	_computed_status	25	finite function
object_checks/genealogical_corridor_split.py	genealogical_corridor_split_rows	39	finite function
object_checks/genealogical_corridor_split.py	genealogical_corridor_split_report	90	finite function
object_checks/history_live_common_measure.py	_finite	34	finite function
object_checks/history_live_common_measure.py	_median	42	finite function
object_checks/history_live_common_measure.py	_pearson	47	finite function
object_checks/history_live_common_measure.py	inherited_uniform_node_density_for_role_basis	59	finite function
object_checks/history_live_common_measure.py	_distance_metrics	121	finite function
object_checks/history_live_common_measure.py	_track_key	147	finite function
object_checks/history_live_common_measure.py	_augment_track_summaries	151	finite function
object_checks/history_live_common_measure.py	history_live_common_measure_rows	210	finite function
object_checks/history_live_common_measure.py	history_live_common_measure_report	313	finite function
object_checks/history_measure_level_mode_dependence.py	_finite	28	finite function
object_checks/history_measure_level_mode_dependence.py	_median	36	finite function
object_checks/history_measure_level_mode_dependence.py	_track_key	41	finite function
object_checks/history_measure_level_mode_dependence.py	_score	52	finite function
object_checks/history_measure_level_mode_dependence.py	_row_for_control	58	finite function
object_checks/history_measure_level_mode_dependence.py	history_measure_level_mode_dependence_rows	135	finite function

Source file	Code object	Line	Object type
object_checks/history_measure_level_mode_dependence.py	history_measure_level_mode_dependence_report	256	finite function
object_checks/level_mode_dependence_summary.py	_finite	8	finite function
object_checks/level_mode_dependence_summary.py	_median	16	finite function
object_checks/level_mode_dependence_summary.py	level_mode_dependence_summary_rows	21	finite function
object_checks/level_mode_dependence_summary.py	level_mode_dependence_summary_report	82	finite function
object_checks/live_current_cluster_scaling.py	_finite	25	finite function
object_checks/live_current_cluster_scaling.py	_median	33	finite function
object_checks/live_current_cluster_scaling.py	_track_key	38	finite function
object_checks/live_current_cluster_scaling.py	_summarize	42	finite function
object_checks/live_current_cluster_scaling.py	live_current_cluster_scaling_rows	74	finite function
object_checks/live_current_cluster_scaling.py	live_current_cluster_scaling_report	131	finite function
object_checks/live_current_property_summary.py	_count	10	finite function
object_checks/live_current_property_summary.py	_append_counts	19	finite function
object_checks/live_current_property_summary.py	live_current_property_summary_rows	35	finite function
object_checks/live_current_property_summary.py	live_current_property_summary_report	60	finite function
object_checks/local_operator_additivity.py	_computed_status	32	finite function
object_checks/local_operator_additivity.py	local_operator_additivity_rows	44	finite function
object_checks/local_operator_additivity.py	additivity_report	92	finite function
object_checks/local_operator_commutation.py	_computed_status	33	finite function
object_checks/local_operator_commutation.py	_append_commutator_row	49	finite function
object_checks/local_operator_commutation.py	local_operator_commutator_rows	68	finite function
object_checks/local_operator_commutation.py	commutation_report	161	finite function
object_checks/local_operator_isotony.py	_is_current_system	43	finite function
object_checks/local_operator_isotony.py	_computed_status	51	finite function
object_checks/local_operator_isotony.py	_construction_overlap	69	finite function
object_checks/local_operator_isotony.py	local_operator_isotony_rows	77	finite function
object_checks/local_operator_isotony.py	isotony_report	159	finite function
object_checks/local_vacuum_cyclicity.py	_computed_status	41	finite function
object_checks/local_vacuum_cyclicity.py	_append_row	57	finite function
object_checks/local_vacuum_cyclicity.py	_graded_closures	89	finite function
object_checks/local_vacuum_cyclicity.py	_annotate_first_cyclic_depth	121	finite function
object_checks/local_vacuum_cyclicity.py	local_vacuum_cyclicity_rows	142	finite function
object_checks/local_vacuum_cyclicity.py	cyclicity_report	164	finite function

Source file	Code object	Line	Object type
object_checks/mode_ablated_measure_summary.py	_finite	8	finite function
object_checks/mode_ablated_measure_summary.py	_median	16	finite function
object_checks/mode_ablated_measure_summary.py	mode_ablated_measure_summary_rows	21	finite function
object_checks/mode_ablated_measure_summary.py	mode_ablated_measure_summary_report	93	finite function
object_checks/mode_ablation_signature.py	_finite	23	finite function
object_checks/mode_ablation_signature.py	_median	31	finite function
object_checks/mode_ablation_signature.py	_signature	36	finite function
object_checks/mode_ablation_signature.py	_cosine	44	finite function
object_checks/mode_ablation_signature.py	_signature_gap	53	finite function
object_checks/mode_ablation_signature.py	_amplitude_gap	59	finite function
object_checks/mode_ablation_signature.py	_combined	64	finite function
object_checks/mode_ablation_signature.py	_mode_signature_row	70	finite function
object_checks/mode_ablation_signature.py	mode_ablation_signature_rows	129	finite function
object_checks/mode_ablation_signature.py	mode_ablation_signature_report	218	finite function
object_checks/modular_entropy_backreaction_direction.py	_finite	8	finite function
object_checks/modular_entropy_backreaction_direction.py	_median	16	finite function
object_checks/modular_entropy_backreaction_direction.py	_hist_index	21	finite function
object_checks/modular_entropy_backreaction_direction.py	_track_key	42	finite function
object_checks/modular_entropy_backreaction_direction.py	_summarize	46	finite function
object_checks/modular_entropy_backreaction_direction.py	modular_entropy_backreaction_direction_rows	95	finite function
object_checks/modular_entropy_backreaction_direction.py	modular_entropy_backreaction_direction_report	149	finite function
object_checks/modular_ratio_trajectory_by_level.py	_f	12	finite function
object_checks/modular_ratio_trajectory_by_level.py	_median	20	finite function
object_checks/modular_ratio_trajectory_by_level.py	_track_key	25	finite function
object_checks/modular_ratio_trajectory_by_level.py	_lambda_trend	32	finite function
object_checks/modular_ratio_trajectory_by_level.py	_ratio_cloud_computed_status	46	finite function
object_checks/modular_ratio_trajectory_by_level.py	modular_ratio_trajectory_by_level_rows	59	finite function
object_checks/modular_ratio_trajectory_by_level.py	modular_ratio_trajectory_by_level_report	152	finite function
object_checks/modular_spectrum_trajectory.py	_parse_floats	22	finite function
object_checks/modular_spectrum_trajectory.py	_lambda_hint	32	finite function
object_checks/modular_spectrum_trajectory.py	modular_spectrum_trajectory_rows	47	finite function
object_checks/modular_spectrum_trajectory.py	modular_spectrum_trajectory_report	123	finite function
object_checks/multiscale_port_entropy_resolution.py	_chunks	36	finite function

Source file	Code object	Line	Object type
object_checks/multiscale_port_entropy_resolution.py	_indicator	49	finite function
object_checks/multiscale_port_entropy_resolution.py	_multiscale_basis	60	finite function
object_checks/multiscale_port_entropy_resolution.py	_response	85	finite function
object_checks/multiscale_port_entropy_resolution.py	_simple_side_dim	98	finite function
object_checks/multiscale_port_entropy_resolution.py	multiscale_port_entropy_resolution_rows	121	finite function
object_checks/multiscale_port_entropy_resolution.py	multiscale_port_entropy_resolution_report	174	finite function
object_checks/observable_state_rows.py	state_physical_role	16	finite function
object_checks/observable_state_rows.py	property_statement_scope	27	finite function
object_checks/observable_state_rows.py	support_loss_interpretation	38	finite function
object_checks/observable_state_rows.py	state_relations_fields	52	finite function
object_checks/observable_state_rows.py	observable_state_rows_rows	64	finite function
object_checks/observable_state_rows.py	observable_state_rows_report	101	finite function
object_checks/oriented_geometry_level_mode_dependence.py	_finite	8	finite function
object_checks/oriented_geometry_level_mode_dependence.py	_median	16	finite function
object_checks/oriented_geometry_level_mode_dependence.py	_key	21	finite function
object_checks/oriented_geometry_level_mode_dependence.py	_ratio	31	finite function
object_checks/oriented_geometry_level_mode_dependence.py	oriented_geometry_level_mode_dependence_rows	35	finite function
object_checks/oriented_geometry_level_mode_dependence.py	oriented_geometry_level_mode_dependence_report	102	finite function
object_checks/oriented_history_geometry.py	_finite	8	finite function
object_checks/oriented_history_geometry.py	_median	16	finite function
object_checks/oriented_history_geometry.py	_measure_key	21	finite function
object_checks/oriented_history_geometry.py	_measure_index	27	finite function
object_checks/oriented_history_geometry.py	_per	31	finite function
object_checks/oriented_history_geometry.py	_track_key	35	finite function
object_checks/oriented_history_geometry.py	_summarize_track	39	finite function
object_checks/oriented_history_geometry.py	oriented_history_geometry_rows	80	finite function
object_checks/oriented_history_geometry.py	oriented_history_geometry_report	206	finite function
object_checks/oriented_history_measure_geometry_summary.py	oriented_history_measure_geometry_summary_rows	7	finite function
object_checks/oriented_history_measure_geometry_summary.py	oriented_history_measure_geometry_summary_report	21	finite function
object_checks/record_anchor_consistency.py	_support_basis	20	finite function
object_checks/record_anchor_consistency.py	_density_support_overlap	30	finite function
object_checks/record_anchor_consistency.py	_skew_alignment	60	finite function
object_checks/record_anchor_consistency.py	_computed_status	77	finite function

Source file	Code object	Line	Object type
object_checks/record_anchor_consistency.py	record_anchor_consistency_rows	93	finite function
object_checks/record_anchor_consistency.py	record_anchor_consistency_report	138	finite function
object_checks/record_live_relative_entropy.py	_finite	25	finite function
object_checks/record_live_relative_entropy.py	_median	33	finite function
object_checks/record_live_relative_entropy.py	_track_key	38	finite function
object_checks/record_live_relative_entropy.py	_summarize_track	42	finite function
object_checks/record_live_relative_entropy.py	record_live_relative_entropy_rows	82	finite function
object_checks/record_live_relative_entropy.py	record_live_relative_entropy_report	141	finite function
object_checks/refined_port_basis_resolution.py	refined_port_basis_resolution_rows	25	finite function
object_checks/refined_port_basis_resolution.py	refined_port_basis_resolution_report	71	finite function
object_checks/refined_port_coarse_consistency.py	_finite	26	finite function
object_checks/refined_port_coarse_consistency.py	refined_port_coarse_consistency_rows	34	finite function
object_checks/refined_port_coarse_consistency.py	refined_port_coarse_consistency_report	101	finite function
object_checks/relative_common_ambient_schur_isotony.py	relative_common_ambient_schur_isotony_rows	33	finite function
object_checks/relative_common_ambient_schur_isotony.py	relative_common_ambient_schur_isotony_report	62	finite function
object_checks/signature_strength_decomposed_measure.py	_finite	8	finite function
object_checks/signature_strength_decomposed_measure.py	_median	16	finite function
object_checks/signature_strength_decomposed_measure.py	_pearson	21	finite function
object_checks/signature_strength_decomposed_measure.py	_key	33	finite function
object_checks/signature_strength_decomposed_measure.py	_track_key	43	finite function
object_checks/signature_strength_decomposed_measure.py	_history_index	52	finite function
object_checks/signature_strength_decomposed_measure.py	_mode_ablation_true_index	60	finite function
object_checks/signature_strength_decomposed_measure.py	_f1f2_index	77	finite function
object_checks/signature_strength_decomposed_measure.py	_dominance	85	finite function
object_checks/signature_strength_decomposed_measure.py	_augment_tracks	102	finite function
object_checks/signature_strength_decomposed_measure.py	signature_strength_decomposed_measure_rows	157	finite function
object_checks/signature_strength_decomposed_measure.py	signature_strength_decomposed_measure_report	241	finite function
object_checks/signature_strength_geometry_summary.py	signature_strength_geometry_summary_rows	7	finite function
object_checks/signature_strength_geometry_summary.py	signature_strength_geometry_summary_report	21	finite function
object_checks/split_factorization.py	_computed_status	28	finite function
object_checks/split_factorization.py	split_factorization_rows	38	finite function
object_checks/split_factorization.py	split_report	83	finite function
object_checks/split_residual_by_collar_width.py	_track_key	31	finite function

Source file	Code object	Line	Object type
object_checks/split_residual_by_collar_width.py	_median	35	finite function
object_checks/split_residual_by_collar_width.py	_summarize_track	40	finite function
object_checks/split_residual_by_collar_width.py	split_residual_by_collar_width_rows	87	finite function
object_checks/split_residual_by_collar_width.py	split_residual_by_collar_width_report	166	finite function
object_checks/stack_dependency_conditions.py	topology_precedence_rows	195	finite function
object_checks/stack_dependency_conditions.py	stack_layer_rows	199	finite function
object_checks/stack_dependency_conditions.py	stack_transition_rows	203	finite function
object_checks/stack_dependency_conditions.py	condition_register_rows	207	finite function
object_checks/stack_dependency_conditions.py	cfs_cross_axis_rows	225	finite function
object_checks/stack_dependency_conditions.py	input_exclusion_rule_rows	229	finite function
object_checks/stack_dependency_conditions.py	stack_architecture_report	233	finite function
object_checks/stack_dependency_conditions.py	condition_register_report	274	finite function
object_checks/support_rank_trajectory.py	_mean	18	finite function
object_checks/support_rank_trajectory.py	_track_key	23	finite function
object_checks/support_rank_trajectory.py	_trend	27	finite function
object_checks/support_rank_trajectory.py	support_rank_trajectory_rows	38	finite function
object_checks/support_rank_trajectory.py	support_rank_trajectory_report	142	finite function
object_checks/support_state_summary.py	_scope	17	finite function
object_checks/support_state_summary.py	_condition_not_met	28	finite function
object_checks/support_state_summary.py	support_state_summary_rows	43	finite function
object_checks/support_state_summary.py	support_state_summary_report	76	finite function
object_checks/tomita_entropy_flow_alignment.py	_finite	26	finite function
object_checks/tomita_entropy_flow_alignment.py	_track_key	34	finite function
object_checks/tomita_entropy_flow_alignment.py	_summarize	38	finite function
object_checks/tomita_entropy_flow_alignment.py	tomita_entropy_flow_alignment_rows	80	finite function
object_checks/tomita_entropy_flow_alignment.py	tomita_entropy_flow_alignment_report	126	finite function
object_checks/tomita_standard_form.py	_computed_status	23	finite function
object_checks/tomita_standard_form.py	tomita_standard_form_rows	58	finite function
object_checks/tomita_standard_form.py	tomita_report	104	finite function
object_checks/two_component_history_measure.py	_finite	8	finite function
object_checks/two_component_history_measure.py	_median	16	finite function
object_checks/two_component_history_measure.py	_pearson	21	finite function
object_checks/two_component_history_measure.py	_key	33	finite function

Source file	Code object	Line	Object type
object_checks/two_component_history_measure.py	_history_index	43	finite function
object_checks/two_component_history_measure.py	_f1f2_index	51	finite function
object_checks/two_component_history_measure.py	_track_key	55	finite function
object_checks/two_component_history_measure.py	_augment_tracks	64	finite function
object_checks/two_component_history_measure.py	two_component_history_measure_rows	116	finite function
object_checks/two_component_history_measure.py	two_component_history_measure_report	185	finite function
run_birth_relative_backreaction_current.py	parse_args	30	finite function
run_birth_relative_backreaction_current.py	main	44	finite function
run_branching_zero_crossing_shift.py	parse_args	30	finite function
run_branching_zero_crossing_shift.py	main	46	finite function
run_directed_region_coverage.py	parse_args	27	finite function
run_directed_region_coverage.py	main	43	finite function
run_finite_emergence_dependency_table.py	write_csv	9	finite function
run_finite_emergence_dependency_table.py	main	20	finite function
run_j_orientation_projectivity.py	parse_args	25	finite function
run_j_orientation_projectivity.py	main	41	finite function
run_live_record_backreaction_current.py	parse_args	26	finite function
run_live_record_backreaction_current.py	main	40	finite function
run_local_global_j_sign_relation.py	parse_args	18	finite function
run_local_global_j_sign_relation.py	main	33	finite function
run_local_operator_property_checks.py	_ints	81	finite function
run_local_operator_property_checks.py	_strings	85	finite function
run_local_operator_property_checks.py	parse_args	89	finite function
run_local_operator_property_checks.py	main	108	finite function